

LoRaWAN Security: An Evolvable Survey on Vulnerabilities, Attacks and their Systematic Mitigation

FRANK HESSEL, LARS ALMON, and MATTHIAS HOLLICK, Technical University of Darmstadt

The changing vulnerability and threat landscape constantly challenge the security of wireless communication standards and protocols. For the Internet of Things (IoT), LoRaWAN is one of the dominant technologies for urban environments, industrial settings, or critical infrastructures due to its low-power and long-range capabilities. LoRaWAN IoT deployments are expected to operate for multiple years or even decades. Hence, it is imperative to maintain operational security at all times while continuously evolving the security of the standard and its implementations. We survey LoRaWAN security and follow a systematic and evolvable approach that can be dynamically updated. To this end, we propose a novel methodology to create evolvable surveys which relate the analyzed critical security concepts, thus allowing IT security experts to reason about LoRaWAN security properties over time. With this, we provide a tool to hardware manufacturers, software developers and providers, and network operators to achieve sustainable security for IoT deployments.

CCS Concepts: • **General and reference** → **Surveys and overviews**; • **Computer systems organization** → **Sensor networks**; • **Networks** → *Network security*; *Mobile networks*; • **Security and privacy** → **Mobile and wireless security**;

Additional Key Words and Phrases: LoRaWAN, LoRa, LPWAN, Internet of Things, taxonomy

ACM Reference format:

Frank Hessel, Lars Almon, and Matthias Hollick. 2023. LoRaWAN Security: An Evolvable Survey on Vulnerabilities, Attacks and their Systematic Mitigation. *ACM Trans. Sen. Netw.* 18, 4, Article 70 (March 2023), 55 pages.

<https://doi.org/10.1145/3561973>

1 INTRODUCTION

The **Internet-of-Things (IoT)** pervades all areas of everyday life, and we witness increasing IoT applications in urban environments, industrial settings, or critical infrastructures. For these application domains, wireless communication standards need to offer a long communication range and support **end devices' (EDs)** lifetimes of months or years of unattended operation. NB-IoT and LoRaWAN are the predominant technologies in such **low-power wide-area networks (LPWAN)**

This work has been co-funded by the LOEWE initiative (Hessen State Ministry for Higher Education, Research and the Arts, Germany) within the emergenCITY centre and by the German Federal Ministry of Education and Research and the Hessen State Ministry for Higher Education, Research and the Arts within their joint support of the National Research Center for Applied Cybersecurity ATHENE.

Authors' address: F. Hessel, L. Almon, and M. Hollick, Technical University of Darmstadt, Secure Mobile Networking Lab, Pankratiusstr. 2, Darmstadt 64289, Germany; emails: {fhessel, lalmon, mhollick}@seemoo.tu-darmstadt.de.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.

1550-4859/2023/03-ART70 \$15.00

<https://doi.org/10.1145/3561973>

[59], with LoRaWAN being the more widely deployed and mature standard. LoRaWAN IoT deployments are expected to operate for multiple years or even decades, which makes a long-term, sustainably secure operation imperative. Wireless security generally does not induce confidence for long-term secure operation because the history of most wireless standards is riddled with security flaws. That includes cellular systems such as GSM/UMTS/LTE [9, 81] and short-range wireless technologies such as Wi-Fi [94] and Bluetooth [79]. Since the first release of the LoRaWAN specification in 2015, security has been an integral part of its design. However, similar to other standards, LoRaWAN had several security issues from the start [62].

To support LoRaWAN's secure long-term operation in the application domains mentioned above, the evolution of security mechanisms must adapt to the constantly changing vulnerability and threat landscape. That is particularly challenging given the complexity of the standard and its ongoing refinement. In practice, the security of standardized mechanisms has to go hand in hand with secure implementations of these mechanisms and operational security. That includes the radio link, which features unique characteristics in LoRaWAN, the network's edge with its EDs and **gateways (GWs)**, and the backend infrastructure.

LoRaWAN has recently attracted the focus of the security research community, resulting in various publications of both theoretical and practical work. The results can inform the LoRaWAN security evolution. However, the changing specifications and increasingly heterogeneous hardware and software landscape have led to a situation in which publications differ broadly in their respective system and adversary model, application scenarios, and general assumptions. Hence, ensuring complete coverage of security issues is challenging, and tools to aid in systematically collecting, tracking, and visualizing them are missing.

Several earlier studies have given an overview of publications on LoRaWAN security, often with a focus on specific applications or areas of the specification. Haxhibeqiri et al. [34] provide a comprehensive overview of LoRaWAN as a protocol and list early work on LoRaWAN 1.0.x security. Butun et al. [11] collect a threat catalog for LoRaWAN 1.1 to perform a risk assessment by ETSI guidelines, assigning each attack a value for likelihood and impact. Philip et al. [72] focus on security-relevant changes in LoRaWAN 1.1 and concisely assess which vulnerabilities it addresses. Dönmez and Nigussie [24] discuss several vulnerabilities and attacks in backward compatibility scenarios between LoRaWAN 1.0.2 and 1.1, focusing on the **over-the-air activation (OTAA)** join process. Ingham et al. [40] give an overview of **denial-of-service (DoS)** attacks based on LoRaWAN 1.0.2 to validate them experimentally. Noura et al. [69] provide a security-focused meta-study on most of the work presented in this section, but without introducing a unifying taxonomy and differentiation between protocol versions. Kuntke et al. [44] approach LoRaWAN security with the specific use case of IoT in agriculture. They provide a comprehensive overview of relevant threats and guidance on reducing the risk for this application. Loukil et al. [55] present security measures implemented by LoRaWAN 1.0.x and 1.1, also covering hybrid deployments. Their focus lies on security context generation and its usage to create and validate data messages. Goulart et al. [31] give an overview of fundamental security concepts for LPWANs, namely NB-IoT, Sigfox, and LoRaWAN, and present lightweight alternatives which could be considered in this area. Rizzardi et al. [77] set an even broader scope and discuss authentication, confidentiality, integrity, and authorization for IoT protocols, including LoRaWAN and higher-layer protocols like CoAP and MQTT. Their overview is based on selected literature.

We survey LoRaWAN security and provide a comprehensive and structured overview of its current state. Our methodology follows a systematic and tool-based approach to get a current picture of LoRaWAN security and evolve it over time. Integration of new insights allows reasoning about LoRaWAN security properties and quickly adapting to and assessing changes in the protocol specification. Combining knowledge from related publications, we identify root causes for each

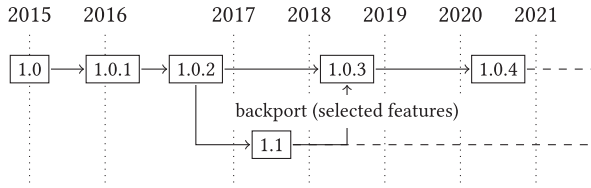


Fig. 1. Version history of the LoRaWAN specification.

problem and suitable countermeasures, revealing structural dependencies. Thereby, we provide a novel instrument to standardization bodies, manufacturers, and network operators to facilitate sustainable security for IoT deployments. We make the following contributions:

- We provide a security-focused introduction to LoRaWAN (Section 2) and the physical layer LoRa (Section 3).
- We design a systematic and evolvable survey methodology for relating and substantiating vulnerabilities, attacks, and countermeasures. We include attributes like the LoRaWAN version to keep track of issues over time (Section 4).
- We apply this methodology to identify security issues under three adversary models: attackers with physical access to EDs and GWs (Section 5), attackers targeting the LoRa radio link (Section 6), and attackers focusing on the backend infrastructure (Section 7). In each part, we discuss the current state of LoRaWAN security and point out open issues and possible solutions.

We conclude our work with a discussion and outlook (Sections 8 and 9).

2 LORAWAN

LoRaWAN is a **medium access control (MAC)** layer protocol on top of the LoRa wireless physical layer, which uses **chirp spread spectrum (CSS)** and sometimes **frequency-shift keying (FSK)** modulation. The specification of the LoRaWAN MAC layer is maintained and published openly by the LoRa Alliance, a consortium of hardware manufacturers, software providers, and network operators. This section covers the characteristics and the operation of the open LoRaWAN MAC layer protocol before Section 3 introduces the proprietary LoRa physical layer.

We provide comprehensive background knowledge focusing on all security aspects relevant for further discussion. We start by giving an overview of the specification and its development in Section 2.1, then discuss the network architecture in Section 2.2, the device activation in Section 2.3, data transfer in Section 2.4, and MAC layer management in Section 2.5. Section 3 covers the LoRa modulation and its peculiarities. Readers familiar with the domain may skip the background information and proceed to Section 4.

2.1 The LoRaWAN Specification

Since its first publication as LoRaWAN 1.0 in 2015 [83], the LoRa Alliance repeatedly revised the LoRaWAN specification. Figure 1 shows the separate branches resulting from this: LoRaWAN 1.0.x, of which all versions are interoperable, and LoRaWAN 1.1, which introduces new features and security measures at the cost of losing direct interoperability with the 1.0.x branch of the specification.

LoRaWAN 1.0.1 [84] and LoRaWAN 1.0.2 [85] mainly contain clarifications and corrections rather than introducing new features. Besides improving security, LoRaWAN 1.1 finalizes unspecified feature drafts from the former specification like additional device classes, multicast messaging, and over-the-air firmware updates. From an architectural point of view, the more explicit

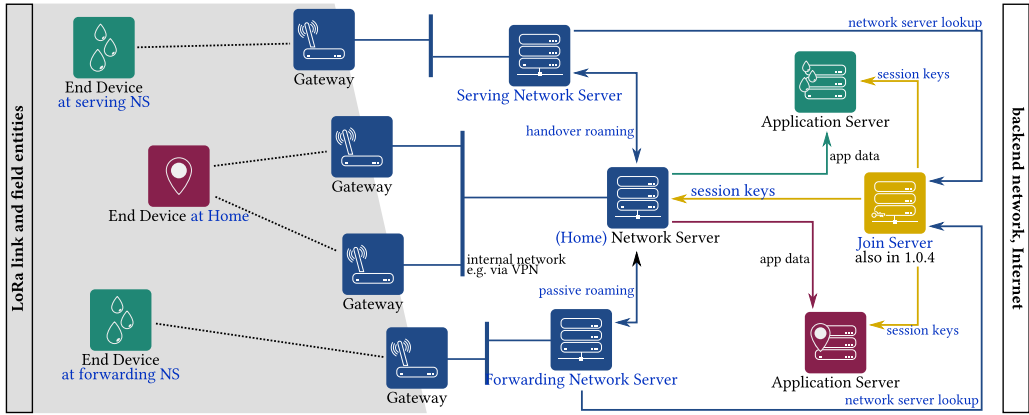


Fig. 2. Architecture of LoRaWAN networks. Highlighted entities have been added in LoRaWAN 1.1.

distinction between network management and application as well as the introduction of the join server as a trusted third-party represent the most significant change in LoRaWAN 1.1. Whereas some functional aspects of LoRaWAN 1.1 found their way back to the 1.0-branch in LoRaWAN 1.0.3 [50], the incompatible security features have not been back-ported, allowing for interoperability within each branch.

After the first findings from security research, it took over two years for their adaption in LoRaWAN 1.1. Meanwhile, the nature of real-world LoRaWAN networks and the development process for LoRaWAN devices may maintain LoRaWAN-1.0.x-compatible networks for much longer. Sensors operate unmaintained for up to 10 years without an option for firmware updates. Even if the network infrastructure eventually advances to a more secure LoRaWAN specification version, support for old sensors and their security issues will remain beyond that date.

Furthermore, the LoRa Alliance advises a certification process assuring protocol and regulatory compliance for all deployed devices but exacerbating the adaption to a new specification during development. In practice, the LoRa Alliance lists no LoRaWAN EDs to be officially certified for LoRaWAN 1.1 as of March 2022, which emphasizes the importance of analyzing and addressing security issues for older versions of the protocol. A reason for the slow adaption in EDs could be an erratum [22], which has not been addressed in an officially released version.

Apart from the core specification, technical recommendation documents further define the operation of LoRaWAN networks. These documents exist for the backend infrastructure [48], certain functionalities like firmware updates [29], and regional characteristics [54]. LoRaWAN operates on the license-free **industrial, scientific, and medical (ISM)** bands, which differ in available frequencies and regulatory limitations.

2.2 Network Architecture and System Model

The LoRaWAN specification defines the network's architecture based on distinct entities and their responsibilities. Figure 2 illustrates how those entities are related and how the architecture evolved in the transition to LoRaWAN 1.1.

ED initiate the transmission in all versions of LoRaWAN. The term uplink communication refers to messages from the ED to the network. As LoRaWAN supports device mobility, all GWs within reach opportunistically pick up the messages and forward them without further validation or filtering to their associated **network server (NS)**. Thereby, NS, GWs, and ED form a star-of-stars topology. The NS deduplicates messages received by multiple GWs, validates their integrity

Table 1. ED Lifetime Device Data [49, Section 6.1.1]

LoRaWAN 1.0	LoRaWAN 1.1	Confid.	App/Nwk	Size	Purpose
DevEUI	DevEUI	⊖	both	8 Byte (EUI)	Globally unique identifier for an ED
AppEUI	JoinEUI	⊖	both	8 Byte (EUI)	Identifier for the application server (AS) (1.0) or JS (1.1) to use with the ED
AppKey	NwkKey	●	Nwk	16 Byte	Derivation of network management session keys
AppKey	AppKey ^a	●	App	16 Byte	Derivation of application session keys
–	JSIntKey	●	JS	16 Byte	MIC in Join Accept message (derived from NwkKey)
–	SEncKey	●	JS	16 Byte	Join Accept encryption (derived from NwkKey)

^aA LoRaWAN 1.1 device in backward-compatibility mode uses the NwkKey as single root key [49, Section 6.1.1.3].

cryptographically, and forwards them to the AS of the ED. The NS takes care of all MAC layer management. On the **application server (AS)**, the domain-specific application accepts encrypted data from the network, decrypts, and processes it. Then, it may decide to queue an encrypted downlink payload for the ED. The AS sends that to the NS, which determines the GW and receive window for transmissions.

LoRaWAN 1.1 introduces a roaming concept, in which the NS of one operator can cooperate with NSs of different operators. In such a scenario, each involved NS may take one or more specific roles for an ED:

- The **Home Network Server (HNS)** manages the ED profile and communicates with the AS.
- The **Serving Network Server (SNS)** manages the MAC layer on behalf of the HNS, called handover roaming.
- The **Forwarding Network Server (FNS)** connects to the GW and forwards data to the SNS, called passive roaming.

We assume this configuration when we refer to a NS in the context of LoRaWAN 1.1 in the remainder of this work.

To enable roaming, the SNS and FNS need a way to identify and address the HNS for an ED, which is where the **join server (JS)** steps in. If the ED has already joined a network, the NetID, a prefix of the DevAddr, fulfills that purpose. While LoRaWAN 1.0.x uses a single root key to derive application and **network session keys (NwkSKey)** on the NS, LoRaWAN 1.1 assigns this task to the JS so that the NS no longer needs access to the application keys. LoRaWAN 1.0.4 supports a JS but keeps the key derivation scheme based on a single root key.

2.3 Device Activation

An end device (ED) can only communicate with a LoRaWAN network after being activated in a so-called join process. Each ED has lifetime device data, as shown in Table 1, consisting of device identifiers and root keys. By activating a device, the ED and NS agree on the device-related session data shown in Table 2.

The number and usage of keys significantly differ between LoRaWAN 1.0.x and LoRaWAN 1.1. In LoRaWAN 1.0.x, only the **application root key (AppKey)** existed to derive the **application session key (AppSKey)** and NwkSKey, removing the independence of network and application keys and requiring the application provider's full trust in the network operator. LoRaWAN 1.1, therefore, introduces the NwkKey as a second root key. Both root keys are known to the join server (JS) that performs the key derivation during the join process.

LoRaWAN supports two ways of activating an ED: **activation by personalization (ABP)** and OTAA. ABP happens once during device commissioning by configuring static session data on the ED. Therefore session data is available upon boot, the ED is activated directly and can start

Table 2. ED Session Data After Joining the Network [49, Section 6.1.2.6]

LoRaWAN 1.0	LoRaWAN 1.1	Confid.	known to	Size	Purpose	Derived from
DevAddr	DevAddr	⊗	NS, AS	4 Byte	Device identifier within network	
FCntUp	FCntUp	⊗	NS, AS	4 Byte	Uplink frame counter	
FCntDown	NFCntDown	⊗	NS	4 Byte	Downlink frame counter (network management)	
FCntDown	AFCntDown	⊗	AS	4 Byte	Downlink frame counter (application payload)	
AppSKey	AppSKey	●	NS ^a , AS, JS	16 Byte	Encryption of application payload	AppKey
–	NwkSEncKey	●	NS	16 Byte	Encryption of network management traffic	NwkKey
–	SNwSIntKey	●	NS	16 Byte	Integrity protection on Serving Network Server (SNS)	NwkKey
NwkSKey	FNwSIntKey	⊕	NS, AS ^a , JS	16 Byte	Integrity protection on Forwarding Network Server (fNS)	NwkKey

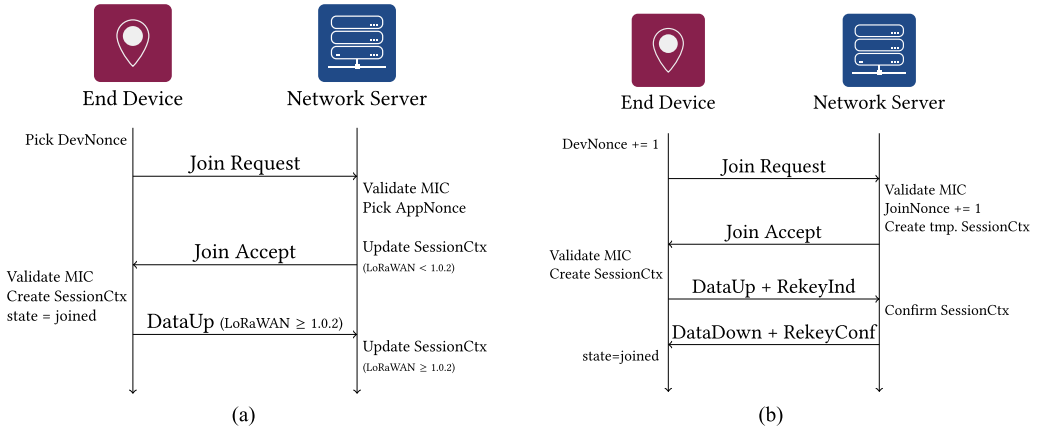
^aonly in LoRaWAN 1.0.x.

Fig. 3. (a) OTAA join process in LoRaWAN 1.0.x. (b) OTAA join process in LoRaWAN 1.1.

communicating. OTAA is more flexible by deriving session data from lifetime data, which allows for rekeying, roaming, and handling saturated frame counters.

Figure 3(a) shows the OTAA join process for LoRaWAN 1.0.x. As GW forward messages transparently, we consider only the ED and NS. The ED starts by selecting its *DevNonce*, a previously unused random 2-byte number. Figure 4 shows how the payload of a Join Request is a concatenation of the globally unique DevEUI, the AS-designating AppEUI, and the DevNonce. This payload is not encrypted, but the so-called **message integrity code (MIC)** is calculated as a **cypher-based message authentication code (CMAC)** based on $\text{aes}_{128}\text{cmac}$, using the AppKey as a secret. As the DevNonce is the only mutable parameter in the MIC calculation, assuring its non-repeatability is decisive for the security of the join process.

When receiving the Join Request, the NS looks up the ED and AS based on the DevEUI and AppEUI. It verifies the freshness of the DevNonce, and the MIC. If both checks pass, the NS randomly picks a 3-byte unique AppNonce. This nonce, concatenated with network ID, the assigned DevAddr, and network configuration parameters forms the Join Accept message. The encrypted Join Accept message is sent to the ED, which decrypts and verifies it. If successful, the ED proceeds with session initialization and key derivation:

$$\text{NwkSKey} = \text{aes}_{128}\text{encrypt}(\text{AppKey}, 0x01|\text{AppNonce}|\text{NetID}|\text{DevNonce}|\text{pad}_{16}), \quad (1)$$

$$\text{AppSKey} = \text{aes}_{128}\text{encrypt}(\text{AppKey}, 0x02|\text{AppNonce}|\text{NetID}|\text{DevNonce}|\text{pad}_{16}). \quad (2)$$

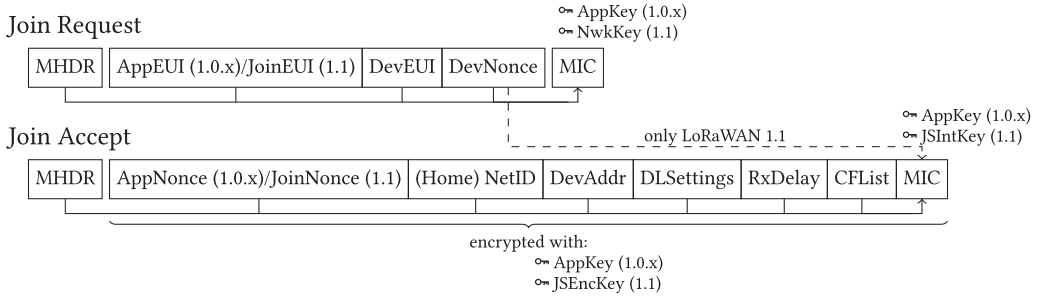


Fig. 4. Messages structure used in the OTAA join process [49, Section 6.2], [85, Section 6.2].

Then, the ED considers itself to be activated. On the NS side, the exact LoRaWAN version defines when the new session becomes active, according to [85, Section 6.2.4, Note 1]:

Figure 3(b) shows how LoRaWAN 1.1 specifies the join process, with three major changes: Nonces are now counters, the session context switch is confirmed bidirectionally, and the Join Accept MIC depends on the Join Request message. Using counters for DevNonce and JoinNonce (formerly called AppNonce) removes the requirement for tracking all previously used values, which turns out to be infeasible, particularly for the JoinNonce on small sensor nodes. This change is sound, as the nonces are employed to assure freshness but not to introduce randomness. The context switch confirmation uses the **rekey indication (RekeyInd)** and **rekey confirmation (RekeyConf)** MAC command pair. The ED is required to confirm the Join Accept message with an uplink message containing the RekeyInd command so that the network server (NS) has proof of successful activation. The NS confirms this command in a downlink message containing a rekey confirmation (RekeyConf) command so that both devices can safely assume that the new context is valid. If the ED does not receive a RekeyConf, it starts over with a new Join Request.

Key derivation in LoRaWAN 1.1 takes place on the JS based on the following scheme [49, Section 6.2.3]:¹

$$\text{FNwkSIntKey} = \text{aes}_{128}\text{encrypt}(\text{NwkKey}, 0 \times 01 \parallel \text{JoinNonce} \parallel \text{JoinEUI} \parallel \text{DevNonce} \parallel \text{pad}_{16}), \quad (3)$$

$$\text{SNwkSIntKey} = \text{aes}_{128}\text{encrypt}(\text{NwkKey}, 0 \times 03 \parallel \text{JoinNonce} \parallel \text{JoinEUI} \parallel \text{DevNonce} \parallel \text{pad}_{16}), \quad (4)$$

$$\text{NwkSEncKey} = \text{aes}_{128}\text{encrypt}(\text{NwkKey}, 0 \times 04 \parallel \text{JoinNonce} \parallel \text{JoinEUI} \parallel \text{DevNonce} \parallel \text{pad}_{16}), \quad (5)$$

$$\text{AppSKey} = \text{aes}_{128}\text{encrypt}(\text{AppKey}, 0 \times 02 \parallel \text{JoinNonce} \parallel \text{JoinEUI} \parallel \text{DevNonce} \parallel \text{pad}_{16}). \quad (6)$$

The main differences in key derivation are the usage of the additional root key and the separation of the NwkSKey to enable roaming.

2.4 Data Transfer

Once activated, an ED can use its session to transmit data to and receive data from the NS. Those messages are called DataUp or DataDown, respectively. Not requiring acknowledgment is referred to as sending unconfirmed messages, called UnconfDataUp and UnconfDataDown. Confirmed messages may be used to request MAC-layer acknowledgment, then called ConfDataUp and ConfDataDown, respectively.

LoRaWAN knows three modes of ED operation, called classes, as shown in Figure 5. Class A is the default, in which the ED initiates all communication by transmitting an uplink frame at an

¹Changes to 1.0.x are highlighted.

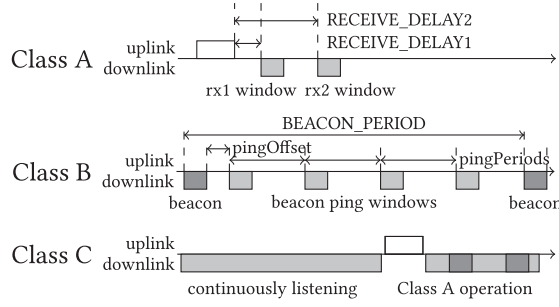


Fig. 5. Classes for ED operation.

Table 3. FCtrl Flags in LoRaWAN Data Messages

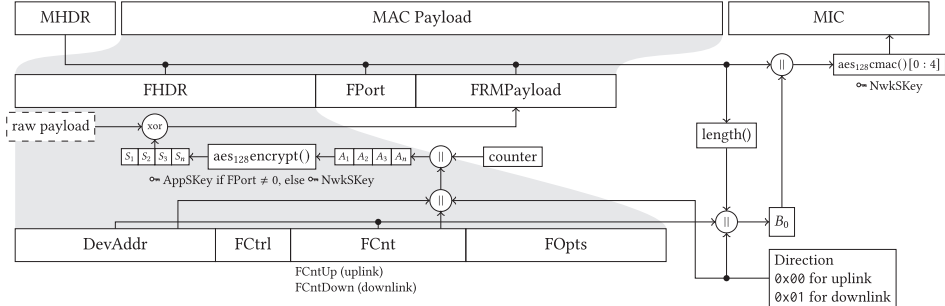
Flag	Up/Down	Purpose
ADR	both	ED (in DataUp) or NS (in DataDown) supports ADR.
ADRACKReq	up	ED requests downlink to check connectivity while using ADR.
ACK	both	A preceding confirmed data message is acknowledged.
FPending	down	NS has more pending downlink messages.
ClassB	up	ED operates as Class B device (LoRaWAN $\geq$ 1.0.3/1.1 only).

arbitrary time. A short period after finishing the transmission, defined by `RECEIVE_DELAY_1`, a first receive window (“rx1”) opens on a channel related to the uplink transmission’s channel. One second later, after `RECEIVE_DELAY_2` has passed, a second window opens on a preconfigured, fixed channel. Class B EDs search for periodically transmitted beacons (all 128s, called the `BEACON_PERIOD`) and schedule additional so-called beacon ping windows for downlink between two beacon transmissions. When Class C is enabled, EDs continuously listen on a specific frequency so that downlink latency is minimized at the cost of significantly increased energy consumption.

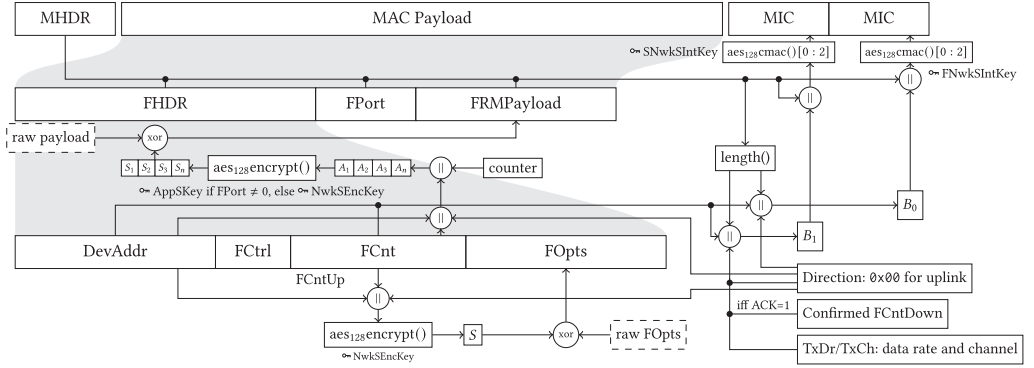
Figure 6 shows the overall message structure for data messages. The message type is encoded in the message header (MHDR) field at the beginning of every LoRaWAN message structure. The remaining message structure is the same for all data messages, with the available flags in the FCtrl block (cf. Table 3) being an exception. The frame header (FHDR) contains medium access control (MAC) layer information, starting with the DevAddr of the transmitting or addressed ED. The FCtrl block follows the address and contains the flags listed in Table 3 for ADR (cf. Section 2.5), Class B mode, and confirmed traffic as well as the four-bit length field for the FOpts field. The frame counter field FCnt contains the 16 **least significant bits (LSBs)** of a 32-bit frame counter, the **most significant bits (MSBs)** are assumed to be tracked on ED and NS and reset to 0 during activation. In LoRaWAN 1.0.x, frame counters are separated for each communication direction (FCntUp and FCntDown). LoRaWAN 1.1 splits the FCntDown into NFCntDown for network messages and AFCntDown for application messages. The FPort field defines which counter to use: a value of 0 denotes network messages, and all other values belong to application messages. In network messages, FRMPayload contains so-called MAC commands, used for network management. Even though MAC commands can be piggy-backed in the FOpts field of messages with application payload in the FRMPayload field, sending a dedicated network message allows sending more than 15 bytes of MAC commands at once.

Whereas the message structure remains unchanged by communication direction and protocol version, the construction of the message integrity code (MIC) for integrity protection and payload

(a) Data Messages in LoRaWAN 1.0.x (Up/Down, Confirmed/Unconfirmed)



(b) DataUp Messages in LoRaWAN 1.1 (Confirmed/Unconfirmed)



(c) DataDown Messages in LoRaWAN 1.1 (Confirmed/Unconfirmed)

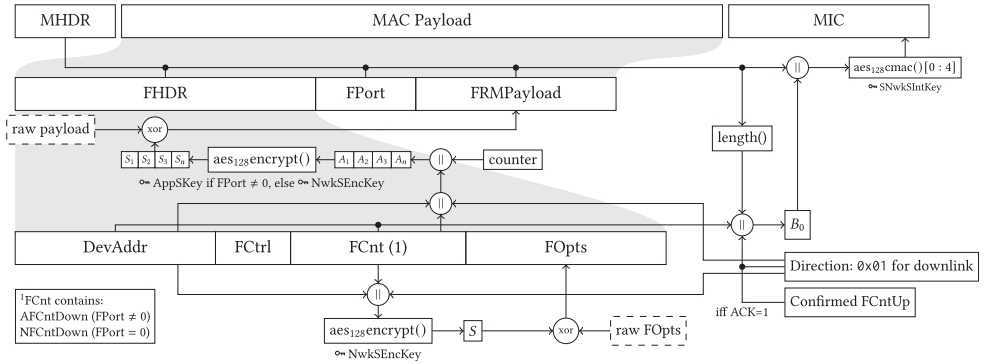


Fig. 6. Structure and cryptographic protection of LoRaWAN data messages. Constants and padding in MIC calculation are omitted.

encryption vary. Figure 6(a) shows the MIC calculation and payload encryption for LoRaWAN 1.0. AES is used in counter mode to generate a keystream from application session key (AppSKey) or network session key (NwkSKey) to encrypt the payload so that arbitrary lengths can be transmitted without adding expensive padding. The keystream generation considers the frame counter to assure freshness. The MIC is calculated using aes128cmac based on an initial block B_0 and the entire message content. The cypher-based message authentication code (CMAC) is truncated to the first four bytes. The MIC is created using the NwkSKey, meaning the NS must verify it.

In LoRaWAN 1.1, payload encryption works similar to LoRaWAN 1.0.x but is based on the network session encryption key (NwkSEncKey) if the payload contains network configuration.

Table 4. MAC Commands in LoRaWAN (Excerpt)

Commands	Introduced	Initiator	Purpose
DeviceTimeReq, DeviceTimeAns	v1.1	ED	ED requests absolute time reference from network.
LinkADDRReq, LinkADDRAns	v1.0	NS	The NS configures the ED's data rate (DR) and tx power.
LinkCheckReq, LinkCheckAns	v1.0	ED	ED requests information about link quality
RekeyInd, RekeyConf	v1.1	ED	ED and NS confirm the context switch after joining.

For downlink messages (Figure 6(c)), the MIC is calculated based on the serving network session integrity key (SNwkSIntKey). It now includes the FCnt value of an acknowledged frame if the ACK flag is set, creating a direct relation between confirmed and acknowledging messages. The MIC in uplink messages (Figure 6(b)) consists of two parts: The first one is created with the SNwkSIntKey for the sNS to verify, and the second is based on the forwarding network session integrity key (FNwkSIntKey), so that a fNS can verify the MIC partly but not forge a valid message. The part for the sNS considers frame metadata, which includes **data rate (DR)** and channel used for the uplink. The related downlink FCnt is included if the frame is acknowledging, similar to the uplink frame.

The FOpts field is encrypted in LoRaWAN 1.1 by a similar mechanism as the network configuration in the FRMPayload field, but only with a single block (counter of 0) since FOpts cannot exceed 16 bytes. However, the specification bases this keystream generation for downlink on NFCntDown [49, Section 4.3.1.6], which is not part of application downlink messages and, therefore, cannot be implemented this way. A change request [22] has been published and implemented in most of today's LoRaWAN 1.1 software.

2.5 Network Management

With LoRaWAN being a MAC layer protocol for low-power wide-area networks (LPWANs), it requires mechanisms for network and device management and data structures to exchange configuration information. Most fundamental properties like device addresses, frame counters, or acknowledgments are mapped directly to ever-present fields in the message structure. Additional functionality comes in in so-called MAC commands, each consisting of a single-byte type identifier and a payload of type-dependent length. Table 4 contains the subset of MAC command pairs for LoRaWAN 1.1 [49, Section 5] which play a role in our security analysis.

The ED initiates an exchange of rekey indication (RekeyInd) and RekeyConf after receiving a Join Accept message during over-the-air activation (OTAA) (cf. Section 2.3) to mutually confirm session context switch. EDs with no or imprecise clock use DeviceTimeReq and DeviceTimeAns to request date and time from the network [49, Section 5.12]. The time can serve the application, e.g., to record timed measurements. The ED can issue a LinkCheckReq to get information about the connection quality from the network's perspective.

The remaining command pair in Table 4, LinkADDRReq and LinkADDRAns, relates to a specific mechanism called **adaptive data rate (ADR)**, which aims at assigning an optimized DR to each ED to save energy and improve the utilization of the medium. This information allows defining a strategy to optimize DR, meaning **bandwidth (BW)** and **Spreading Factor (SF)**, and **transmission power (TP)**, either per ED or network-wide. After an ED signals its demand for ADR

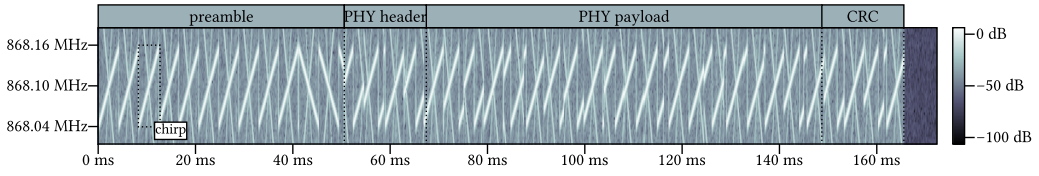


Fig. 7. LoRa frame (SF9, 125 kHz BW).

configuration by setting the ADR flag, the NS may send a LinkADRReq command containing the determined optimal transmission parameters, which are applied and acknowledged or declined in the LinkADRRAns command [49, Section 5.3]. ADR-adjusted EDs run connectivity checks periodically by setting the ADRACKReq flag to request a downlink and autonomously and gradually revert their DR when no answer is received [49, Section 4.3.1.1].

3 LORA PHYSICAL LAYER

LoRaWAN achieves its characteristic low energy profile and long range mainly by employing the LoRa modulation on the PHY. LoRa is a chirp spread spectrum (CSS) modulation, so it uses an ascending or descending signal around a center frequency f_c . It is limited by the channel bandwidth bw and wraps around when reaching $f_c \pm \frac{bw}{2}$. A physical layer (PHY) symbol, called a chirp, consists of a full ascend or descend, with the frequency offset at its start denoting the symbol's value. Two parameters mainly influence the shape of a chirp: bandwidth (BW) and Spreading Factor (SF). The Spreading Factor sf determines the number of symbol values a chirp can represent as 2^{sf} . LoRaWAN uses values from 7 to 12 for sf and 125 kHz, 250 kHz, and 500 kHz for bw [54].

With both parameters determining the length and width of a symbol, they also determine the rate of frequency change over time. Chirps for which those rates differ are orthogonal, allowing for creating multiple virtual channels around the same center frequency f_c . Combining several chirps creates a LoRa frame, as depicted in Figure 7. Each frame starts with a preamble for synchronization, terminated by the easily identifiable 2.25 inverse chirps at its end. The optional PHY header follows, specifying the frame length and properties like whether the frame has a physical layer **cyclic redundancy check (CRC)** checksum at the end. Without a header, the receiving side must know this information in advance. Next is the actual payload, followed by a CRC for error detection at the frame's end.

SF and BW together form a so-called data rate (DR) in LoRaWAN terminology. Each region may define up to 16 DRs, indexed starting with the lowest bitrate and highest robustness against interference. The receiver sensitivity and airtime calculation example show the tradeoff made with each step of DR increase: transmission time roughly halves while sacrificing 2.5 dBm to 3 dBm of sensitivity.

3.1 Co-Existence and Interference

LoRa claims to be robust against interference from other signals through its relatively high BW and low DR of the CSS modulation, allowing it to coexist with other technology in the unlicensed industrial, scientific, and medical (ISM) bands. Reynders et al. [75] compare the cross-technology interference between LoRa and **ultra narrow band (UNB)** modulation schemes employed by Sigfox. Their results show that for collocated CSS and ultra narrow band (UNB) networks, CSS modulation like LoRa is better in keeping its robustness promise close to gateways, there allowing for higher throughput.

Effects of interference between two LoRa signals depend on signal polarity, orthogonality through SF and BW, relative frame offset in time, and **signal-to-interference ratio (SIR)**. Goursaud and Gorce [32] first analyzed SF orthogonality by identifying the critical signal-to-interference ratio (SIR) threshold for successful reception for different SFs combinations. They show that channel orthogonality is not perfect, with SIRs of -16 dB to -29 dB for higher and -24 dB to -36 dB for lower interfering SFs. For signals and interferers of the same SF, a beneficial capture effect is observable for a signal of 6 dB. Croce et al. [19] revise these findings using a MATLAB model and experiments with a LoRa receiver, reporting average values of -16 dB and a worst-case SIR of only -8 dB between SFs. They estimate the capture effect threshold at around 3 dB for a 2% **packet error rate (PER)**. Therefore, modeling a LoRa network as independent, virtual channels for each SF is questionable, and near-far situations on the same SF may be adverse for the distant node. Haxhibeqiri et al. [35] support these findings: In a setup with controllable signal strength and temporal offset of the interfering frame, they show that an equally strong interferer using the same SF cannot overcome the capture effect on a synchronized receiver. With a 6 dB to 14 dB stronger interferer, however, symbols are corrupted, and the cyclic redundancy check (CRC) at the receiver fails. Rahmadhani and Kuipers [74] examine the impact of the temporal offset between colliding frames with a higher granularity, emphasizing the importance of a frame synchronization of at least four preamble symbols. They note that PHY header collisions also lead to receiver unlocking, while collisions during payload raise CRC failures.

3.2 Regulatory Limitations

Besides interference, regulatory limitations are important in determining the performance of LoRaWAN networks. In the EU863-870 region, each transmitter has to follow a 1% duty cycle (0.1% or 10% on some sub-bands), which is most critical for long frames on low DRs. Adelantado et al. [1] show how these restrictions affect high-throughput nodes but that dense networks reach their actual throughput peak even earlier by collisions becoming the main limiting factor. Van den Abeele et al. [91] simulate large networks to study the effect of confirmed traffic. They find the duty cycle limitation, which also applies to the gateway, as a major bottleneck when networks become large, have a low gateway density, or strongly use confirmed uplink traffic, draining the duty cycle budget. Other regions like US902-928 come with a dwell time of 400 ms, which allows for more frequent transmissions. However, as downlink disrupts all uplink traffic, gateway capacity remains a bottleneck in large networks.

4 METHODOLOGY

This section presents the methodological framework that sets the foundation for our evolvable security analysis. We define the terminology and taxonomy for classifying our findings (Section 4.1) and describe our systematic approach for literature retrieval (Section 4.2).

4.1 Terminology

To represent knowledge about LoRaWAN vulnerabilities, attacks, and their mitigation in a structured manner, we apply the terminology from Figure 8 to all literature considered for this study. We define the terms used in the figure as follows:

A *Publication* is an article on the topic of LoRaWAN security matching the selection criteria discussed in Section 4.2. Each publication relates to different security concepts valid for a specific protocol versions. We differentiate between the following research methods to further categorize the contribution of a specific publication:

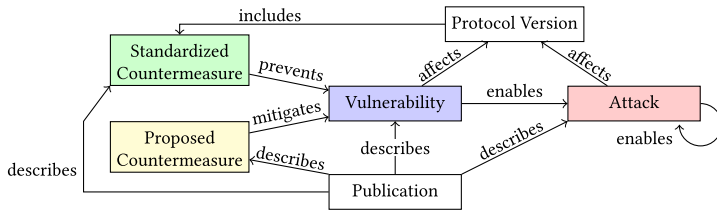


Fig. 8. Terminology used in this survey. Colors are used as the key to identify the different concept types in concept graphs.

- *General Discussion*: The concept is explained with a level of detail that allows verifying the idea. If the publication extends a concept from related literature, it adds new insights to the discussion.
- *Formal Analysis* (F): The concept is modeled formally so that claims about it can be formally verified or falsified.
- *Simulation* (S): A simulator demonstrates the concept and generates qualitative or quantitative data.
- *Proof of Concept* (PoC): The concept is shown qualitatively in a real-world environment.
- *Experiment* (E): The concept is shown quantitatively in a real-world environment.

We define a *vulnerability* (depicted in blue in this document) as a specific property, behavior, or non-behavior of the protocol that allows for actions contradicting the defined security policies of the system. A vulnerability can exist directly within the specification or may emerge from a specific way of specification-compliant implementation. Without a threat, vulnerabilities do not compromise the security of the system.

Vulnerabilities combined with a threat enable *attacks* (in red). They represent a way of exploiting one or more vulnerabilities or other, *enabling* attacks to achieve a goal contrary to the system's security policies.

LoRaWAN comes with different *protocol versions* that define the operation of the protocol. Changes in the specification directly affect the protocol's behavior, so the existence of vulnerabilities and the applicability of attacks must be seen in the context of the *affected* protocol version.

A protocol version can include *standardized countermeasures* (in green) to specific vulnerabilities. Implementing these countermeasures directly eliminates these vulnerabilities or *prevents* attackers from exploiting them. It also transitively prevents attacks enabled by this vulnerability.

Publications on vulnerabilities and attacks often discuss *proposed countermeasures* (in yellow), which can also mitigate vulnerabilities and may or may not be compatible with the specification. Implementing incompatible changes requires an out-of-band agreement between the device and network side to deviate from the standard protocol. Proposed countermeasures later adopted by the official specification are modeled as standardized countermeasures in this study.

Structuring the knowledge in this well-defined taxonomy comes with major advantages: We can apply formal rule-based reasoning to predict missing data points that research has not covered yet. Assessing the security of LoRaWAN 1.0.4 while requiring only little additional literature is an example of that ability. As vulnerabilities and attacks are tightly bound to countermeasures, adding the new specification to the corpus to identify applicable attacks based on the implemented and missing countermeasures is sufficient. On the other hand, we can classify research papers based on their methods and, by aggregating results for certain concepts, improve the confidence of findings. New research can be added continuously, helping to evolve the knowledge about the security of LoRaWAN.

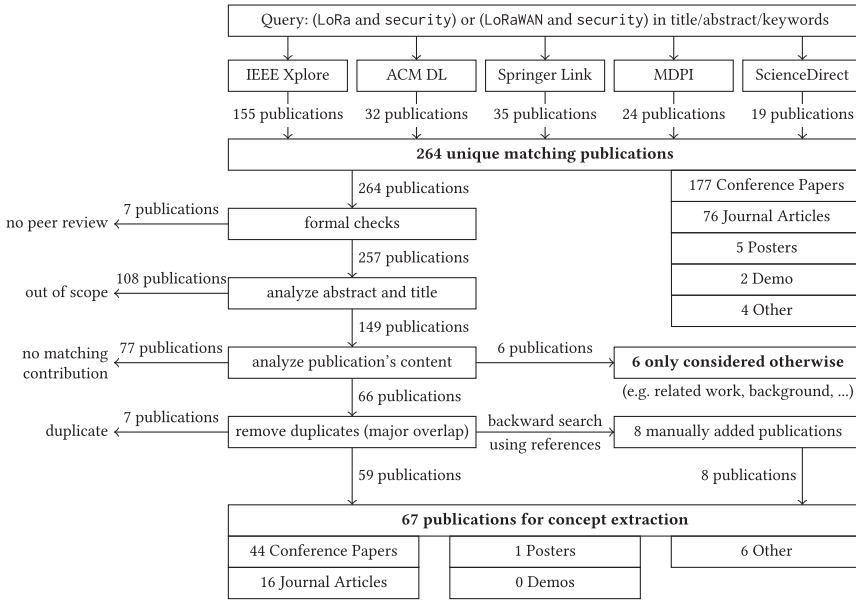


Fig. 9. Creation of the literature corpus and staged filtering of publications.

To facilitate knowledge management, we created scripts to organize and visualize our findings while working on this survey. Data of vulnerabilities, attacks, and countermeasures are stored as linked data. We generate overview tables in this manuscript automatically and can update them based on the latest revision of the dataset. The included graphs have been laid out manually with the help of an interactive viewer based on the same data source. The whole process and all tools support collaboration and are generalized to enable analyzing other protocols.

4.2 Paper Selection and Corpus Creation

This survey follows a systematic approach for creating a corpus of considered publications. As shown in Figure 9, the whole process starts with a broad search query we used to retrieve publications that contain the keywords *LoRa* and *security* or *LoRaWAN* and *security* either in their title, abstract, or keywords. We run this query against the catalogs of five publishers: IEEE Xplore, the ACM Digital Library, Springer Link, the **Multidisciplinary Digital Publishing Institute (MDPI)**, and Elsevier’s Science Direct, giving us a total of 264 unique publications. Publications listed in multiple catalogs are reduced to a single entry in this step. The corpus presented in this survey was collected on June 7th, 2022. For the collected publications, we first run formal checks to assure accessibility and quality:

- The publication’s language must be English.
- It must appear in a journal or conference proceedings with a formal peer-review process.

In the next step, each publication’s title and abstract are manually reviewed to ensure they match the scope of the survey, and the keywords are used in relation to each other. Promising publications then undergo a more thorough investigation, during which we apply the following criteria:

- Publications must focus on LoRa and LoRaWAN, not analyze the security of custom protocols.

- If the specification explicitly mentions a particular issue and its implications as a design decision, we do not consider it a vulnerability.
- Publications must either present new vulnerabilities, attacks, or countermeasures or add a previously unknown aspect or new data to the discussion, e.g., by using a new method for verification or adding empirical results.
- If concepts affect a wide variety of LPWANs or network protocols, we only include those that analyze the situation specifically in the context of LoRaWAN.
- Vulnerabilities and attacks must be valid and proposed countermeasures must be technically sound.
- We do not consider vulnerabilities and attacks of specific LoRaWAN software or those which arise from operational or organizational issues of a certain software or software library.

The last filtering step is to remove duplicates of publications that have a major overlap with each other. In these cases, we keep the more elaborate variant, or the earlier version if both provide a similar level of detail.

After this process, we consider 59 publications for extracting concepts based on the taxonomy introduced in Figure 8. To identify a concept's first introduction, we perform a backward search using references of the selected publications when they add a new concept to the taxonomy but base it on related work not covered in our systematic search. That adds eight more publications for concept extraction, giving a total of 67 publications. The additional publications are security whitepapers [13, 62] and theses [38, 92, 98] that first describe and evaluate certain issues, lay the foundation for future research, or provide large empirical datasets. Furthermore, we add publications for which our keyword-based approach failed [1, 19, 57], mainly because they discuss frame collisions under a performance aspect and miss the "security" keyword. However, they provide insights relevant to discussing jamming in Section 6. From all 67 publications, we extracted 40 vulnerabilities, 47 attacks, 26 proposed countermeasures, and 14 standardized countermeasures.

5 PHYSICAL THREATS AND MITIGATION

LoRaWAN provides connectivity to a large area even with a low infrastructure density. EDs can be deployed at unfrequented locations, making them an easy target for hardware manipulation. LoRaWAN gateways (GWs) can be placed at locations with restricted access to provide physical protection but come with their own risks if compromised.

This section covers attacks relying on physical access to LoRaWAN devices and infrastructure in the field. Like all the following sections, we group vulnerabilities, attacks, and countermeasures for each focus area and present them in a graph. We summarize each section by a tabular overview that connects the concepts to LoRaWAN versions, research methodology, and corresponding literature.

5.1 Compromised End Devices

EDs in the field are the weakest part of LoRaWAN infrastructure due to their limited physical protection. We show the related concepts as a graph in Figure 10, again with vulnerabilities in blue, attacks in red, standardized countermeasures from the specification in green, and proposed countermeasures in yellow. Dashed lines represent mitigation in the specification, helping visually identify resolved issues and remaining problems. Graphs with many dashed lines represent areas covered well by the specification. An attacker who gains physical access to an ED can proceed in two directions: disabling the device to achieve denial-of-service (DoS) or trying to access and modify data, e.g., by extracting keys.

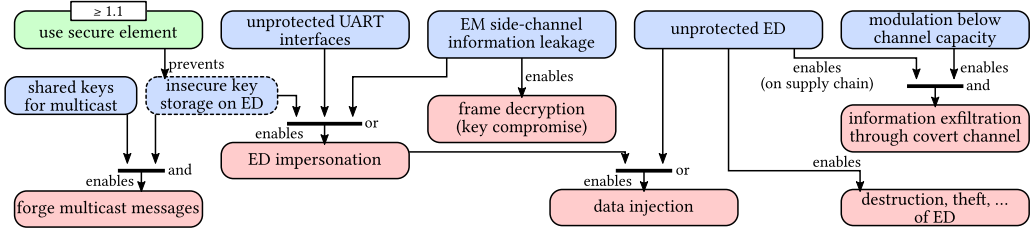


Fig. 10. Vulnerabilities and attacks requiring physical access to an ED.

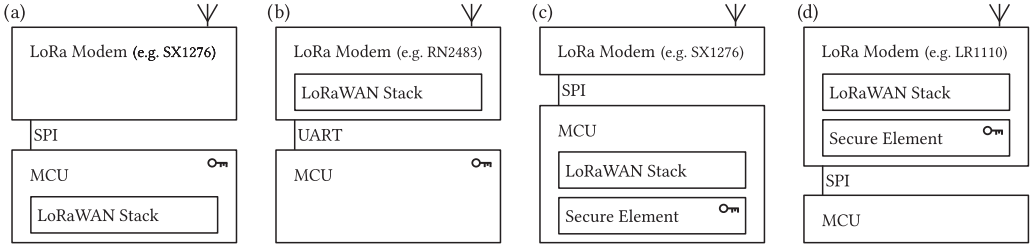


Fig. 11. ED key storage: (a) key and LoRaWAN stack in MCU program memory (b) key in MCU program memory, LoRaWAN stack on dedicated chip, (c) key in SE of MCU, LoRaWAN stack on MCU, (d) key in SE and LoRaWAN stack on dedicated chip.

For our taxonomy, we model access to an *unprotected EDs* as an enabling vulnerability, inherent to how the EDs are deployed and used. Like with most of the concepts in this section, the vulnerability affects all protocol versions equally, as options for defining requirements on physical ED security in the specification are limited. Kloibhofer et al. [43] discuss this issue for an agricultural application, in which sensors are widespread and without supervision. In [39], Hou and Zheng discuss supply chain attacks on EDs as an additional threat.

The most trivial attack resulting from missing physical protection is the *destruction or theft of EDs*. As Butun et al. [11] show, both options create a risk to the availability of the service, but the scope of an attack is limited to a single ED. Theft of devices can also affect the authenticity of sensor readings, e.g., by bringing a tracking device to a new location. Kloibhofer et al. [43] propose to counter this attack by securing the ED so that it cannot be used after tampering with or stealing it.

Even without key extraction, an attacker possessing an ED can cause *data injection* on the application layer by replacing sensors with their own equipment. An attacker can trigger or suppress communication in event-based applications which compare sensor readings to thresholds and transmit only on their exceedance. Miller [62] sketches such an attack for ED architectures in which the LoRaWAN stack and keys are on a separate chip (cf. Figure 11(b)). Data is sent to the chip via UART, which an attacker can intercept. In [2], Aras et al. demonstrate a **proof of concept (PoC)** for such an attack based on a LoRaWAN-enabled mousetrap using the RN2483 transceiver. Butun et al. [11] call this attack type “rogue ED” and assess major risks from this attack for all security goals, except for a critical risk to authenticity. Monjur et al. [63] propose local data injection at the ED by taking a **machine-in-the-middle (MITM)** position between sensor and **microcontroller unit (MCU)** and modifying sensor values on the bus.

We define *ED impersonation* as a more general concept also affecting the network layer. That usually implies application *data injection* as well, meaning that the attacker can mimic the ED

completely and inject arbitrary data. While the ED's application usually runs on its microcontroller unit (MCU), the network layer is implemented either on the same chip or as part of the LoRa transceiver, as Figure 11 shows. This architectural decision determines the attacker's chances for impersonation through key extraction. If the LoRaWAN stack runs on a dedicated MCU, authentication and encryption are done before the frame is passed to the LoRa transceiver, meaning that the interface between both chips reveals the same data as the radio interface would do (Figure 11(a), (c)). With a LoRaWAN stack as part of the transceiver, the interface to the MCU reveals application data and some network management instructions (Figure 11(b), (d)). Without **non-volatile memory (NVM)** and a **secure element (SE)** on the transceiver, the interface also reveals the keys during transceiver initialization (Figure 11(b)). Cerrudo et al. [13] discuss threats to the keys of LoRaWAN EDs. An impersonation attack through physical access only is enabled either by unprotected interfaces on the PCB, insecure key storage, or **electromagnetic spectrum (EM)** side-channel leakage.

Key extraction through *unprotected UART interfaces* exists as a vulnerability only in ED designs requiring key transfer from one component to another. If that is the case, the attack will apply to all inter-chip communication interfaces. Aras et al. [2] show a proof of concept (PoC) for key extraction on a device based on the Microchip RN2483, a modem with an integrated LoRaWAN 1.0.2 stack. In this specific case, the RN2483 is (re-)initialized each time the main MCU resets, exposing the AppKey on the UART interface. Miller [62] discusses a variation of this attack. In his example, the RN2483 is only initialized once with keys by the legitimate MCU. However, the attacker replaces the MCU with a rogue one, using the already configured LoRaWAN stack to impersonate the ED.

An attacker could also extract keys from *insecure key storage on the ED*. If the key is stored as part of the firmware image or in EEPROM during commissioning without a secure element (SE) (Figure 11(a), (b)), it might be easy to extract. Coman et al. [15] mention this issue for LPWAN protocols in general, and suggest the usage of a SE as mitigation. Navarro-Ortiz et al. [68] present key extraction as a major risk for remotely deployed EDs in LoRaWAN 1.0.2. They propose a solution around USIM modules for 3GPP networks, and demonstrate it in a PoC. However, they use AKA mutual authentication through challenge-response, making their approach incompatible with OTAA. In their risk assessment, Butun et al. [11] predict a high likelihood of exploitation of weak key storage. Kloibhofer et al. [43] try to prevent key extraction in their PoC of agricultural sensors by introducing trip wires to ED's case. Key erasure will trigger immediately if these wires break while forcefully opening the device.

The LoRaWAN specification recommends to *using a SEs* or **hardware security modules (HSM)** in version 1.1 [49, Section 6.1.1.3] to counter key extraction. A SE must be part of the chip running LoRaWAN (cf. Figure 11(b), (d)). Coman et al. [15] and Naoui et al. [66] discuss ED security for LoRaWAN 1.0.2 and recommend deploying secure hardware to protect against physical attacks. Kloibhofer et al. [43] use a Zymkey hardware security module (HSM) in their PoC of a secure agricultural sensor. Coppolino et al. [16] provide a PoC of secure key storage on the ED using ARM TrustZone on a Raspberry Pi. While searching for cheap alternatives for SEs, Navarro-Ortiz et al. [68] present a PoC for generating session keys for LoRaWAN 1.0.2 on USIMs.

The third vulnerability enabling ED impersonation is *electromagnetic spectrum (EM) side-channel leakage*, which allows extracting keys by measuring EM emanations of the devices. Xu et al. [97] show a PoC for LoRaWAN 1.0.3, which needs EM traces of the encryption of 25 to 100 frames to restore the AppKey. The PoC assumes knowledge of the encrypted payload. Fukushima et al. [28] extract the single AppKey root key of a LoRaWAN 1.0.2 ED by observing its emissions.

Forging LoRaWAN messages always requires access to the network keys, as they are required for calculating the MIC. If attackers only obtain application keys, their actions are limited to *frame*

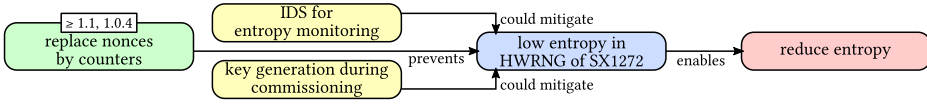


Fig. 12. Entropy issues of the SX1272 transceiver.

decryption. As LoRaWAN 1.0.x has only one root key, frame decryption without full device compromise can only happen by obtaining the AppSKey, and thus only affects a single session. In LoRaWAN 1.1, an additional attack path is obtaining the AppKey root key, which then allows deriving all session keys and affects traffic for the ED's lifetime. We discuss lifetime effects and key management issues also in Section 6.10. Xu et al. [97] and Fukushima et al. [28] show frame decryption by obtaining the application key as a consequence of information leakage. Dönmez and Nigussie [25] discuss how personal devices left unsupervised or shared between users can suffer from long-lasting privacy implications if their keys should be compromised.

Multicast keys are special because they encrypt data sent to multiple EDs at once over class B. *Multicast keys are shared* between all EDs within a multicast group. Consequently, *forging multicast messages* is possible by accessing a single ED within the multicast group. Miller [62] first mentions the issue but refers to an early draft, unfinalized in the examined LoRaWAN 1.0.

LoRa is a *modulation below channel capacity*, trading the throughput for a longer range. In principle, this allows modulating additional information onto the LoRa signal, e.g., by slightly modifying the amplitude during transmission. As commercial off-the-shelf (COTS) LoRa transceivers cannot demodulate this information, an attacker can use it for *information exfiltration through a covert channel*. Hou and Zheng [39] describe a scenario in which an ED is prepared with special firmware to send device keys over such a covert channel after being commissioned by the new owner. In their experiments, they use a slightly modified COTS LoRa transceiver and a **software defined radio (SDR)** on the receiving side. The authors report successfully receiving the covert channel over 250 m.

5.2 Entropy Reduction

An issue exploitable in proximity, but without physical access to a specific ED is the *low entropy in the hardware random number generator (HWRNG) of the SX1272*, a LoRa transceiver for EDs. Figure 12 shows the nexus around this issue.

A comprehensive work on this topic is the thesis by Zulian [101]. After an analytical discussion of the SX1272 receiver model and identifying influential factors to its **received signal strength indication (RSSI)** measurements, an experiment shows that *reducing entropy* as an attack is possible. The follow-up publication [88] discusses its worsening effect on the DevNonce saturation vulnerability, further discussed in Section 6.6. Danish et al. [20] experiment with a jammer interfering with the join process of LoRaWAN 1.0.2. They show that the jammer not only affects the OTAA message exchange directly but also causes DevNonce repetition and rejection of the Join Requests by the NS.

McPherson and Irvine [58] propose a scheme for *key generation during commissioning* in which a smartphone creates the root keys. Those are then transmitted using flashlight pulses to the ED equipped with a photoresistor and uploaded to the NS off-band using the smartphone's cellular connection. Danish et al. [20] observe a decrease in the Hamming distance between consecutive 16 bit nonces from 8 to around 3 during jammer activity. Based on their findings, they show how an *intrusion detection system (IDS) for entropy monitoring* can be integrated into the NS to detect this condition.

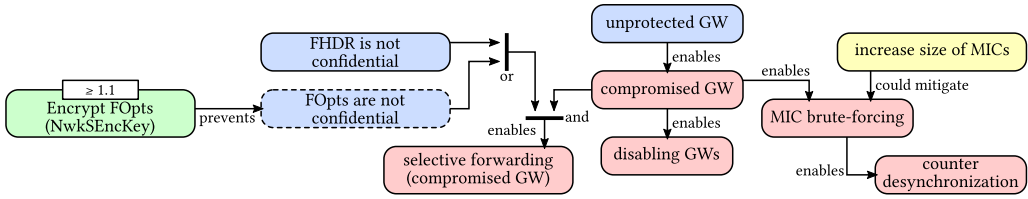


Fig. 13. Vulnerabilities and attacks requiring physical access to an GW.

Both of these proposed countermeasures do not adequately address the root cause of the vulnerability, as nonce creation still requires an entropy source. Tomasin et al. and Zulian [88, 101] already note that OTAA needs only the freshness but not the randomness of values, suggesting *replacing nonces with counters*. Naoui et al. [66] propose a variation of this mitigation using timestamps, which comes with the downside of requiring an additional **real-time clock (RTC)** on EDs. The specification adopts the replacement by simple counters in LoRaWAN 1.0.4 [52, Section 6.2.4] and LoRaWAN 1.1 [49, Section 6.2.2]. An ED using this modification is backward-compatible with an older LoRaWAN network, as the counter does not repeat. The NS, however, is not directly backward-compatible with older EDs if it enforces strict monotonicity of the DevNonce. We discuss further implications on security coming from this change in Section 6.6.

5.3 Compromised Gateways

LoRaWAN GWs are the components that terminate the LoRa link and forward frames transparently to and from the infrastructure, forming a bridge between the wireless network and the IP-based backhaul. From the specification's perspective, the GW's role is "simply to decode uplink radio packets from the air and forward them unprocessed to the NS" [53, Section 3], leaving secure connection and protocols open for the implementer. In practice, the GW also adds metadata like received signal strength indication (RSSI), the received channel and DR, and local timestamps to the frames before forwarding them. Due to the heterogeneity of gateways and their means of network connectivity, we assume the possibility of a *compromised GW* attack based on a generic *unprotected GW* vulnerability, as shown in Figure 13. Protection may be missing either physically or on the network interfaces. An attacker's motivation for this may lie in (1) having the same capabilities as attacking the radio link, but with more stable results by removing the impact of the channel, (2) being able to manipulate frame metadata directly, and (3) accessing the operator's network infrastructure, e.g., through stored VPN credentials.

Oniga et al. [70] discuss requirements for a secure LoRaWAN 1.0 infrastructure and propose best practices for improving operational security. They conclude that a compromised GW is a risk if the GW's credentials allow for access to the core network. Beltrán-García et al. [7] show a PoC for getting control of the GW by penetration testing its local network interfaces and exploiting an insecure configuration. Coman et al. [15] note that a GW provides much higher BW in the direction of the NS than the relatively slow LoRa radio link. An attacker with such resources can run DoS attacks by flooding the NS with uplink frames.

Another PoC presented by Coman et al. [15] for LoRaWAN 1.0.2 is *MIC brute-forcing*. Using the high bandwidth between GW and NS, an attacker can reasonably try to guess the correct MIC for a message. By selecting a message with a high frame counter, the authors extend the attack to achieve the goal of *counter desynchronization*. If successful, the NS no longer accepts the ED's messages due to a small frame counter value. We give more details on counter-related issues in Section 6.8. Canard and Ferreira [12] identify the small size of the MIC as problematic and propose *increasing MIC size* to make brute-forcing impractical.

Table 5. Vulnerabilities Requiring Physical Access to EDs or GWs

Vulnerability	1.0/1.0.1	1.0.2/1.0.3	1.0.4	1.1
EM sidechannel information leakage	⊖ ^a	⊖ ^a PoC: [28, 97]	⊖ ^a	⊖ ^a PoC: [28]
insecure key storage on ED M: use secure element (1.1)	● [16]	● [15, 68]; PoC: [43]	●	⊖ ^b [11]
low entropy in HWRNG of SX1272 M: replace nonces by counters (1.1, 1.0.4); P: IDS for entropy monitoring [20], key generation during commissioning [58]	● E: [88, 101]	● [58]; E: [20]	⊖	⊖ [58]
modulation below channel capacity	●	●	●	● E: [39]
unprotected ED	⊖ ^a	⊖ ^a PoC: [43]	⊖ ^a	⊖ ^a [36, 39]
unprotected GW	⊖ ^a	⊖ ^a	⊖ ^a	⊖ ^a
unprotected UART interfaces M: use secure element (1.1)	⊖ ^b [62]; PoC: [2]	⊖ ^b	⊖ ^b	⊖ ^b

^ahardware-dependent. ^bimplementation-dependent.

M: mitigated in specification (with version). P: proposed countermeasure (in publication).

Another attack after compromising is to *disable a GW* to reduce availability in its coverage area. Beltrán-García et al. discuss and then demonstrate the consequences in a network with a single GW, which stops all transmissions. In a multi-GW network, redundancy should reduce the effect of disabling a single GW.

An attacker also may use *selective forwarding*, e.g., to filter out traffic for specific applications or EDs. In contrast to a reactive jamming attack (cf. Section 6.1.2), selective forwarding on the GW can use selection criteria arbitrarily chosen over the entire frame length. Butun et al. [11] mention the possibility of forwarding only selected messages, but assess a minor risk to this kind of attack.

As the GW does not hold any MAC layer keys, frame selection can only regard fields without confidentiality protection. Aras et al. [3] point out that the *FHDR is not encrypted*. If, for example, the DevAddr was encrypted, it would not be possible for the NS to select the matching device key for decryption. Van Es [92] provides a detailed overview of all fields of LoRaWAN messages and the information they reveal for an attacker.

In this overview, van Es also point out that the *FOpts field is not confidential* in LoRaWAN 1.0.2. As the field can contain MAC commands, it reveals network configuration information and allows attackers to deny specific commands and functionality. The LoRaWAN 1.1 specification solves this issue by requiring to *encrypt FOpts with the NwkSEncKey* [49, Section 4.3.1.6]. In LoRaWAN 1.0.x, the only option to send MAC commands confidentially is in the FRMPayload field with FPort set to 0.

5.4 Summary

In this section, we discussed physical access issues of EDs and GWs. These devices make up the field infrastructure of LoRaWAN, which is most vulnerable to on-site attacks. We list related vulnerabilities in Table 5 and attacks based on them in Table 6. We show vulnerabilities and attacks which apply (●), which apply under certain conditions (⊖), and which do not apply (⊖) for each version. Abbreviations for applied methods follow the scheme introduced in Section 4.1, with no prefix referring to a pure discussion of the concept. The applicability for most vulnerabilities strongly depends on the tamper resistance of the hardware and design decisions for the software and network infrastructure implementation. While attacks on the ED impact the device and, if applicable, its multicast groups only, attacks on GWs may affect traffic in a certain region if no other GW can serve it. In addition, compromised GWs may pose further risk to the network infrastructure.

6 LORA LINK THREATS AND MITIGATION

Wireless communication comes with the inherent property of the medium being publicly accessible. Only the location of possible receivers limits access to transmission. That makes the radio

Table 6. Attacks Requiring Physical Access to EDs or GWs

Attack	1.0/1.0.1	1.0.2/1.0.3	1.0.4	1.1
compromised GW V: unprotected ED	● [70]	● [15]; PoC: [7]	●	● [15]
data injection V: unprotected ED; A: ED impersonation	● ^{a,b} [62]; PoC: [2]	● ^{a,b} PoC: [28, 63]	● ^{a,b}	● ^{a,b} [11]; PoC: [28]
destruction, theft, ... of ED V: unprotected ED	●	● PoC: [43]	●	● [11]
disabling GWs A: compromised GW	●	● PoC: [7]	●	●
frame decryption (key compromise) V: EM sidechannel information leakage, no forward secrecy, long-term root keys, key derivation on NS (1.0–1.0.3)	●	● [21]; PoC: [28, 97]	●	● [21, 25, 36]; PoC: [28]
information exfiltration through covert channel V: unprotected ED, modulation below channel capacity	●	●	●	● E: [39]
MIC brute-forcing A: compromised GW	●	● PoC: [15]	●	● [15]
reducing entropy V: low entropy in HWRNG of SX1272	● E: [88, 101]	● E: [20]	○	○
selective forwarding (compromised GW) V: FHDR is not confidential, FOpts are not confidential (1.0–1.0.4, 1.0 on NS, 1.0 on ED); A: compromised GW	●	●	●	● ^c [11]

^aimplementation-dependent. ^bhardware-dependent. ^conly FOpts are encrypted.

V: enabled by this vulnerability (only for selected versions). A: enabled by this attack (only for selected versions).

link between an ED and the—potentially multiple—neighboring GWs an attractive target for an adversary. The likelihood of being detected decreases compared to physically accessing devices, especially for passive attacks. To perform active attacks like dropping, modifying, or replaying frames, the adversary must reach a machine-in-the-middle (MITM) position on the direct link between EDs and GWs. In this section, we first show how a MITM situation can be established in LoRaWAN through jamming and replaying, to then discuss relevant vulnerabilities and attacks for this setup.

6.1 Intercepting the LoRa Link

LoRaWAN not only supports a LoRa-based physical layer but also defines DRs which use frequency-shift keying (FSK) modulation. We focus our discussion of the PHY solely on LoRa. This decision aligns with our results from the literature search that yields studies addressing the LoRa PHY, but none for FSK-based LoRaWAN.

Traffic interception in LoRaWAN comes with a challenge created by the star topology between EDs and GW. The direct link disallows MITM attacks on a higher layer, e.g., by interfering with neighbor discovery or route establishment. Instead, the single-hop communication link splits into two hops, one from the ED to the attacker and one from the attacker to the GW. In the following sections, we first discuss how jamming as an attack against the PHY can be used to interrupt the direct link between both parties. Then we see how an attacker can place transceivers to be still able to receive the jammed traffic. Finally, we relate these findings to the constraints of the LoRaWAN MAC layer and discuss attack scenarios that address them.

6.1.1 Jamming LoRa Frames. The discussion in Section 3.1 shows that the LoRa modulation is most susceptible to interference with other LoRa signals in a non-orthogonal combination of spreading factor, BW, and polarity. Therefore, the apparent choice for constructing a LoRa jammer is an existing LoRa transceiver combined with a detection mechanism for the target frame. Most work presented in this chapter uses COTS LoRa transceivers for jamming. Semtech's SX127x modules are often used, which allow for low-level access to the transceiver functions. Transceivers with an integrated LoRaWAN stack (cf. Figure 11(b)) like the Microchip RN2483 have also been evaluated [40]. However, they are less flexible as only valid LoRaWAN frames can be created. Besides the transceiver, the main difference between jamming approaches is the trigger used to decide when to emit a jamming signals.

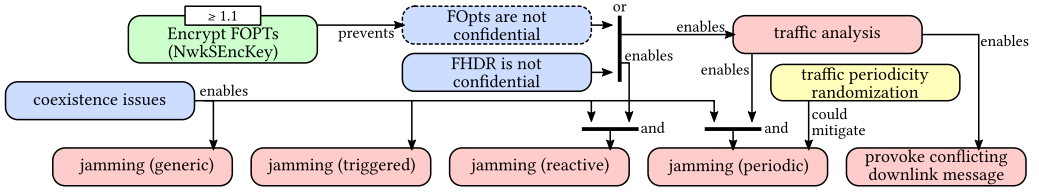


Fig. 14. Attacks, vulnerabilities, and countermeasures for jamming.

The effectiveness of jamming depends on the topology as well as the interference's length and duration. The topology determines the signal to interference plus noise ratio (SINR), which has to be below -3 dB to -6 dB for the same DR to cause invalid demodulation (cf. Section 3.1). If up-link frames are the target, jammers must be placed so that none of the GWs remains in reception range. For targeted attacks, we can assume that the interfering frame will always start later than the legitimate frame because an attacker needs time to sense channel activity. The requirement to prevent frame acceptance at the receiver gives the upper boundary for the attacker's reaction delay. While a fast attacker could interfere with the LoRa PHY header, it is more likely that only the payload is affected by interference. In that case, it is sufficient to interfere with at least one non-padding symbol at the end of the frame, which carries either the LoRa PHY CRC (cf. Section 3) or the LoRaWAN MIC (cf. Section 2.4), causing frame or, respectively, message rejection if invalid.

6.1.2 Jamming Techniques. When modeling jamming within our taxonomy, as shown in Figure 14, we can identify the *coexistence issues* as central vulnerability, enabling different variants of jamming attacks. Contrary to robustness claims by the manufacturer, the LoRa modulation is susceptible to jamming under the preconditions from the previous section. Aras et al. [2] set up a basic COTS jamming experiment based on the discussion about LoRa coexistence raised by Reynders et al. [75], proving that cheap COTS hardware suffices to interrupt communication. In a later publication [3], they extend their experiments on exploiting coexistence issues for targeted attacks and present some first quantifiable results. Croce et al. [19] simulate frame collisions first in MATLAB, then in a USRP software-defined radio (SDR) transmitter and SX1272 LoRa receiver setup. They report a SINR of 1 dB as the threshold for correct reception, which contradicts the -6 dB usually reported from theoretical analysis and practical experiments, e.g., those carried out by Goursaud and Gorce [32]. Another model, built on the assumption of a -6 dB threshold, is presented by Martinez et al. [57] for the ns-3 simulator. Wadtkar et al. [95] present an SDR-based PoC for introducing targeted interference in LoRa networks. Hessel [38], as well as Perković and Sirišćević [71] state the importance of exploiting missing coexistence as a foundation for their jamming attacks. Based on the coexistence issues, we now discuss jamming variants that differ in target accuracy, reaction time, and detectability. Figure 15 shows the collision between the jamming signal and a LoRaWAN frame of 3 byte payload for each variant as a spectrogram.

The most straightforward approach to interfere with LoRa frames is *generic jamming* by transmitting a continuous CSS signal with the same parameters, similar to an eternal preamble, as shown in Figure 15(e). This type of jamming affects all transmissions for which the receiver resides in the transmission range of the malicious interferer. Being an attack solely on the physical layer, it equally affects all versions of the LoRaWAN specification. Aras et al. [2] practically demonstrate this issue by co-locating a LoRaWAN network with a jammer made from COTS hardware, showing that it can drop the receive rate to nearly zero. Martinez et al. [57] simulate a scenario with a “channel-oblivious jammer” which transmits periodically on randomly chosen frequencies, thus reducing the packet receive rate for the whole network. Wadtkar et al. [95] show a PoC

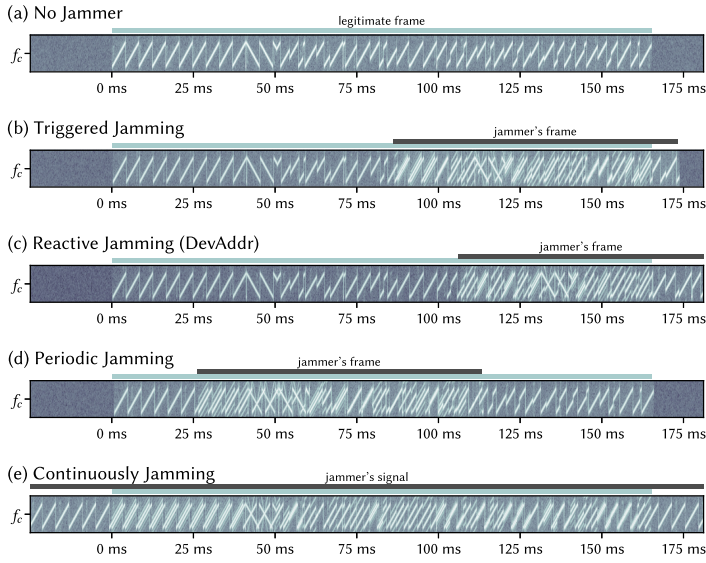


Fig. 15. Spectrogram of different jammer types and their respective temporal offset to the legitimate LoRaWAN message (containing 3 bytes payload and no FOpts).

based on an SDR running Bastille Research's GNU Radio module gr-lora. Their results show a significant drop in the packet receive rate (PRR) for the evaluation in a lab setup. Ingham et al. [40] use an RN2483 transceiver with an integrated LoRaWAN stack (cf. Figure 11(b)) for jamming. The experimental results show a significant drop in the PRR. In contrast to other experiments, the jammer node in this work is registered with the NS, making it look like a malfunctioning device in NS logs. Coman et al. [15] mention this type of jamming as a generic risk for all kinds of LPWANs in their analysis. Finally, Monjur et al. [63] provide a PoC of continuous wideband noise disturbing communication on a block of channels, but without quantifying the success rate of the jammer.

A continuous jamming signal is easy to detect and locate. An attacker can counter this by only activating the jammer briefly when a frame is observed. We call this approach *triggered jamming*, as shown in Figure 15(b). Martinez et al. [57] extend their simulation by adding a "channel-aware" jammer to the channel-oblivious one, which uses sensing on LoRaWAN channels to transmit when a frame is detected. Combined with the channel-oblivious jammer, it further exacerbates the situation at the nearly saturated gateways. Hessel [38] evaluates different frame lengths, DRs, and jammer locations in an experiment. The results show that LoRaWAN messages of all lengths are susceptible to triggered jamming using COTS devices and an optimized LoRa transceiver driver. Perković and Sirišević [71] run experiments on an Arduino-based platform using a standard modem driver. They successfully jam transmissions from node to node for 17 bytes of payload length. Their second setup in a multi-GW topology shows that only GWs closer to the jammer than to the ED are affected by jamming.

They discussed jamming techniques affect all messages equally. Sophisticated attacker can jam frames based on their content by utilizing symbol-wise demodulation during transmission and deciding on jamming using the extracted information. We call this approach *reactive jamming* and show an example in Figure 15(c). Given an attacker model without access to cryptographic keys, the jamming decision has to be made based on public information. Fields close to the beginning of a frame are more critical, as they leave a longer decision and reaction period. The MHDR field in the

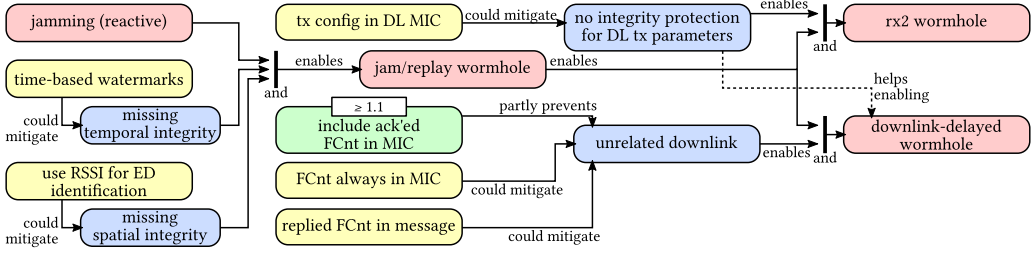


Fig. 16. Attacks, vulnerabilities, and countermeasures around LoRaWAN traffic interception.

first byte allows detection by message type and cannot be encrypted as it is essential for message parsing. Other fields, like FOpts and FHDR, have been introduced with selective forwarding in Section 5.3.

In [2], Aras et al. discuss the possibility of reactive jamming, leading to their implementation and experimental evaluation in [3]. They connect two COTS transceivers via UDP sockets to separate traffic monitoring and jamming, reporting mixed success for SF9 and good results for SF10 and above, given a LoRa frame length of at least 17 bytes and using DevAddr as the jamming criterion. Hessel [38] combines monitoring and jamming in a single COTS transceiver. He evaluates different frame lengths and offsets for the jamming criterion, showing that removing the network overhead allows reliably jamming messages by MHDR or DevAddr for all DRs. Butun et al. [11] assess a minor risk arising from reactive jamming for all protection goals.

Another way of running targeted attacks is *periodic jamming*, which exploits predictable periodic transmission times of EDs or GWs, making periodic sensor readings, receive windows, and beacons possible targets. Using traffic analysis, as further discussed in Section 6.5, an attacker can schedule their interfering frame to match the subsequent transmission, as shown in Figure 15(d). Ingham et al. [40] extend their generic jamming experiment to apply this technique to suppress a periodically transmitting sensor node. The authors report lower attack success rates compared to other jamming techniques for periodic jamming. As a countermeasure, they propose *traffic periodicity randomization*, meaning an attacker can no longer predict the transmission time of the next data message.

Beltrán-García et al. [7] also use traffic analysis to prepare blocking upcoming traffic. They observe uplink traffic to calculate the receive windows and *provoke a conflicting downlink message*. This attack differs from the jamming techniques in this section as it does not use actual frame collisions. However, the effects of collisions in frame schedules are similar to those of periodic jamming. In their PoC for LoRaWAN 1.0.2, they use a Class C ED close to the target ED to generate the traffic. By keeping the transmitter of the GW busy during the receive windows, they block both opportunities for sending downlink messages to the ED under attack. Class C is required because it is the only operation mode that allows requesting frame transmissions unconstrained by receive windows.

6.1.3 Intercepting LoRaWAN Traffic. Jamming alone only blocks communication. More sophisticated attacks rely on the ability to selectively intercept messages, forward them at the right time and location, and handle bidirectional communication. We discuss how reactive jamming, as introduced in Section 6.1.2, and missing metadata integrity for received frames, as shown in Figure 16, are the central enablers for wormhole attacks.

Most LoRaWAN messages have *missing temporal integrity*, as they lack explicit information about their transmission time. Timing of messages is added to messages by GWs, making the

implicit assumption that frames have not been delayed. In [3], Aras et al. state that the consequences of breaking this assumption depend on the underlying application. In [2], they already mention the possibility of constructing a replay-wormhole based on this vulnerability. Hessel et al. [37] validate the timing integrity issue experimentally to motivate a discussion about wormholes.

In addition, LoRaWAN is also *missing spatial integrity*, meaning that EDs are not equipped with a location sensor but rely on the network for locating them. An early discussion on the transmission parameter manipulation due to missing spatial information on EDs is given by Oniga et al. [70], but with a focus on GW compromise instead of jamming. In a simulation based on colored petri-nets, van Es [92] shows that a faster out-of-band connection allows an attacker to forward frames to another GW to affect routing. While the immediate transmission from the simulation might be practically infeasible due to the timing constraints in the single-hop LoRaWAN network, the described risk would hold for a jam-and-replay setup. In [93], the authors extend their work on downlink routing decisions to discuss countermeasures. Sung et al. [87] provide a PoC of how RSSI measurements can be manipulated to evaluate an RSSI-based handshake protocol. Gladisch et al. [30] mention jamming and then replaying a frame in the coverage area of a different NS but without explicitly outlining the expected outcome of such an attack. Hessel et al. [37] demonstrate missing spatial integrity experimentally as the foundation for experiments with LoRaWAN wormholes. Locatelli et al. [47] similarly forward traffic from the coverage area of one GW to another.

By exploiting integrity and coexistence issues, an attacker can create an essential building block for higher-level attacks, the *jam-and-replay wormhole*. Given LoRa's coexistence behavior, as discussed in Section 3.1, a jammer should be located close to the receiver of the target transmission to minimize the SINR at the target. On the contrary, to still extract the target transmission's content for later replay, a sniffer node should be placed closest to the transmitter, maximizing its SINR. In their discussion of reactive jamming, Aras et al. [2] propose the extension of the reactive jammer with a sniffer. In [3], these authors experimentally validate their proposal using an architecture in which the sniffer triggers the jammer via UDP over a network. They report success for longer payloads and higher SFs, but not for short frames due to the network latency. Hessel [38] unties the direct connection between sniffer and jammer for triggering. In his experimental approach, the jammer monitors the channel and starts jamming autonomously, relying on the different SINR at the sniffer, allowing correct reception without coordination. In Hessel et al. [37], the authors call this approach a "unidirectional wormhole" as it does not include downlink forwarding and state that more effort is needed to consider the ED's receive windows. They also point out that including the uplink transmission parameters in the MIC in LoRaWAN 1.1 limits replaying uplink frames to the channel and BW initially selected by the ED. As a special case without jamming and replaying, Locatelli et al. [47] quickly forward an ED's frame to replay it within the deduplication period of the NS to manipulate routing decisions.

The literature describes two approaches to returning the downlink transmissions to the ED: the downlink-delayed wormhole and the rx2 wormhole, each extending the jam-replay wormhole and enabled by a different vulnerability.

The first vulnerability is the *missing integrity protection for downlink transmission parameters*. While the MIC considers the uplink channel and DR (cf. Figure 6(b)) in LoRaWAN 1.1, the downlink remains unprotected in this regard (cf. Figure 6(c)). That means, in LoRaWAN 1.1, uplink messages are rejected when received on any but the intended channel. However, an attacker can change transmission parameters for uplink in LoRaWAN 1.0.x and for downlink in all versions. Hessel et al. [37] verify this experimentally. They then introduce the *rx2 wormhole*, for which the attacker replays the rx1 downlink in the rx2 window. The gained time allows an attacker to store and forward the uplink at a different location or another GW before the ED sleeps again. Timing constraints limit this attack to uplink messages with less than 500 ms airtime, thus excluding SF11

and above for 125 kHz BW. An obvious countermeasure, also proposed by Hessel et al., is to *include the transmission parameters in the MIC* for all message types and directions.

The second vulnerability, caused by *unrelated downlink* messages, enables bidirectional MITM attacks. Given an uplink and downlink data message, it is impossible to decide whether the downlink message is a reply to the uplink message.

LoRaWAN 1.1 standardizes a countermeasure against unrelated downlink messages but limits it to confirmed messages. If the ACK flag is set in reply to a confirmed data message in LoRaWAN 1.1, the *FCnt value of the acknowledged message is considered in the MIC* calculation of the acknowledging message (cf. Figure 6(b), (c)).

For LoRaWAN 1.0.2, Yang [98] mentions unrelated downlink messages as a vulnerability in the context of a PoC for ACK spoofing, an attack based on wormholes. Dönmez and Nigussie [24] examine its applicability in backward-compatible networks, showing that only homogeneous LoRaWAN 1.1 networks in the special case of confirmed data messages are unaffected. Later, Yang et al. [99] discuss their PoC again for LoRaWAN 1.1 and assess it to be applicable for this version, even for confirmed traffic. Hessel [38] practically evaluates the vulnerability by combining multiple LoRaWAN versions and ED and NS software stacks.

Based on the unrelated downlink vulnerability, Hessel et al. [37] constructed another wormhole type, the *downlink-delayed wormhole*. Instead of relaying the downlink message directly, its forwarding is postponed to one of the receive windows following the subsequent uplink message. That allows using this type of wormhole for frames longer than 500 ms, at the cost of downlink traffic delayed by at least one message exchange. By parallelizing downlink message collection and forwarding, the attack can be applied to a series of downlink transmissions. The experiments in [37] show that this attack applies to unconfirmed traffic in LoRaWAN 1.1.

Preventing the downlink-delayed wormhole can be achieved by addressing the underlying vulnerability, the unrelated downlink. Hessel et al. [37] propose a straight-forward measure: Adopting the change made in LoRaWAN 1.1 for all message types, meaning *always include the uplink frame counter in the downlink MIC* calculation, would invalidate downlink messages outside their intended transaction context. Previous to the release of LoRaWAN 1.1 and focusing on ACK Spoofing, Yang [98] proposed to explicitly *add the acknowledged FCnt to the acknowledging message*, which would prevent ACK spoofing, but no other attacks based on unrelated downlink messages.

Another way of preventing wormhole attacks, including the unidirectional jam-and-replay wormhole, is to resolve the temporal and spatial integrity issues. Bigler et al. [8] propose a timing side-channel for periodically transmitting EDs to create *time-based watermarks*. They calculate a rolling checksum over the payload and transmit it encoded in the inter-frame spacing of the periodic uplink messages. With the payload now being bound to the transmission time, the NS can use this to detect anomalies. The authors provide a PoC in simulation for LoRaWAN 1.1. To improve spatial integrity, Sung et al. [87] propose using the *RSSI for ED identification*. In their PoC for LoRaWAN 1.0.2, they customize the join process to learn about the ED's expected RSSI values. If the RSSI deviates beyond a threshold, frames are considered malicious or part of a replay attack. This approach, however, is limited to stationary devices and a stable environment, as mobility or other environmental effects may affect the RSSI measurements, leading to falsely positive attack detection.

6.2 Spoofing Attacks

Reactive jamming is a building block for creating wormhole attacks, wormhole attacks in turn enable higher-layer constructs like the two spoofing attacks discussed in this section: ACK spoofing and ADR spoofing. Figure 17 contains enablers for these attacks and puts them into context known from the wormhole discussion in Figure 16.

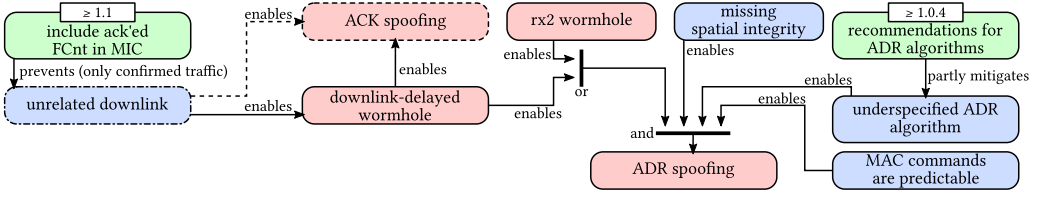


Fig. 17. Attacks, vulnerabilities, and countermeasures related to spoofing. Enablers and proposed countermeasures, which have been discussed in the previous sections are omitted.

The *ACK spoofing* aims at convincing the sender of confirmed messages of their acknowledgment without them being received by the legitimate party. That can be achieved using unrelated downlinks to send a message with the ACK flag set (cf. Section 2.4) from an earlier transaction. The frame must have a higher FCnt value than the last one observed by the receiving party. From an attacker's perspective, they must suppress all communication between capturing a message with an ACK flag and replaying it. That constitutes a special case of the downlink-delayed wormhole. Yang [98] studies this issue for LoRaWAN 1.0.2 and provides a PoC to demonstrate the modified message flow. The implementation uses a compromised GW instead of attacking over-the-air using a jammer. In their follow-up paper, Yang et al. [99] extend the work of [98] and claim ACK Spoofing to still be possible in LoRaWAN 1.1. Dönmez and Nigussie [24] assess the attack to be valid in constellations with at least one LoRaWAN 1.0.x entity, as the changes to include the FCnt in the MIC must be applied to both sides, ED and NS, to maintain interoperability. Hessel [38] applies this attack to various software stacks on the ED and the NS, as long as one side runs on 1.0.x or in 1.0.x compatibility mode. Countermeasures to prevent ACK spoofing are the same as for the unrelated downlink vulnerability in Section 6.1.3. In contrast to the generic downlink-delayed wormhole, ACK spoofing can be effectively prevented by the measure introduced in LoRaWAN 1.1 to include the acknowledged FCnt in the MIC of the acknowledging message for confirmed traffic because ACK spoofing is only applicable to confirmed messages.

The second spoofing attack, *ADR spoofing*, exploits the adaptive data rate (ADR) mechanism to reduce the availability of EDs. ADR is based on signal-to-noise ratio (SNR) measurements from GWs, meaning the attack is also enabled by missing spatial integrity. By forwarding ADR-enabled uplinks using one of the wormholes with an exit close to a GW, an attacker can increase the SNR measurements for an ED. The NS then over-optimizes the DR and transmission power (TP) settings so that the ED no longer reaches any GW and thus becomes disconnected from the network. The ED-side ADR algorithm recovers connectivity only slowly, and an attacker can keep the delivery rate low by forwarding only messages with the ADRACKReq flag set (cf. Section 2.4). Hessel et al. [37] evaluate ADR spoofing experimentally for LoRaWAN 1.1 using both the rx2 wormhole and downlink-delayed wormhole as a basis and report both approaches to work. Their results suggest that even a unidirectional wormhole could work, as only a single high-SNR frame must reach the NS. The current implementations queue the ADR adjustment until it eventually reaches the ED. Locatelli et al. [47] show a similar attack with a unidirectional wormhole for EDs in sparsely covered areas, but without implementing a dedicated retention phase to keep them disconnected when sending ADRACKReqs.

According to [37], the enabling vulnerability for ADR spoofing is the *underspecified ADR algorithm* on the NS side. LoRaWAN only specifies the part of the ADR algorithm used on the ED to regain connectivity. The part used by the NS to assign DR and TP to EDs, however, is left to the implementer. Most NSs use an algorithm initially proposed by Semtech [17], which adjusts parameters based on the maximal SNR observed for an ED and therefore is vulnerable to the ADR spoofing

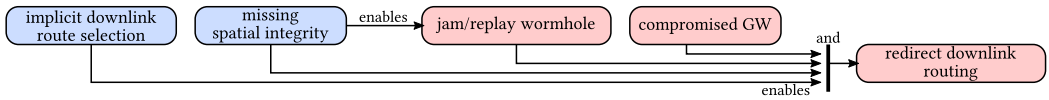


Fig. 18. Routing vulnerabilities and attacks.

attack. In LoRaWAN 1.0.4, the LoRa Alliance reacted to this issue and now *recommends avoiding drastic changes of the DR* and TP of an ED but instead adapting slowly to the better channel conditions [52, Section 5.2, Note 1]. While this is not a complete solution and is only recommended but not enforced in the specification, it still exacerbates exploiting the ADR algorithm for an attacker.

6.3 Routing

Contrary to, e.g., cellular networks, LoRaWAN EDs are not associated with a specific GW. Therefore, the NS heuristically decides for the GW, which relays downlink traffic back to the ED. The NS evaluates metadata gathered by GWs during uplink reception to make the right choice and optimize link quality. Figure 18 shows how an attacker could interfere with this heuristic. The main idea is to *redirect downlink routing* to a GW which is not in range of the ED and, thus, interrupt downlink traffic. The attacker can either use the jam-and-replay wormhole or exploit compromised GWs. The enabling vulnerability is again missing spatial integrity, this time in combination with implicit downlink route selection. The NS uses the untrusted location and signal strength information to decide how to return downlink traffic to the ED.

Van Es [92] formally models downlink routing attacks based on colored petri-nets, showing that an attacker with a fast out-of-band connection can affect downlink routing decisions and redirect them to a GW out of range of the ED. The thesis examines LoRaWAN 1.0.2 and gives an outlook on 1.1. The results are later published in [93], stating that downlink routing attacks still apply to LoRaWAN 1.1. Butun et al. [11] call this attack “wormhole” and “blackhole” and discuss it for LoRaWAN 1.1 based on compromised network infrastructure, mainly using compromised GWs. Only looking at this attack path but ignoring attacks on the LoRa link, they assess a minor risk for all protection goals according to ETSI guidelines. Locatelli et al. [47] conduct a real-world experiment on redirecting downlink routing as a black-box test in an operated network and report success most of the time for lower SFs.

The *implicit downlink route selection* is discussed by van Es et al. [92, 93] and used as part of their formal analysis and simulation to show that the vulnerability affects all LoRaWAN versions. Locatelli et al. [47] explain how most modern NSs mainly rely on the RSSI and SNR values for route selection. They find that only the ChirpStack NS applies randomness in GW selection when multiple GWs reach a high signal quality level.

6.4 Resource Exhaustion

LPWANs usually have strict resources limits: energy, duty cycle, network BW, and more. If an attacker can increase the consumption of these resources, they can reduce the availability of the network. Figure 19 shows two clusters of concepts for such attacks, one for resource exhaustion on the ED based on energy consumption and one for consuming a GW’s duty cycle budget.

For the EDs, *energy depletion* is the biggest threat. Cheap sensors usually have a finite energy budget, limiting their lifetime. If an attacker can quickly drain a significant amount of this budget, the ED’s lifetime decreases.

Mikhaylov et al. [60] show an experiment in which they transmit LoRa frames in the receive windows following an uplink transmission. With the *MIC at the end of the frame*, the ED has to receive the whole frame before it can detect an invalid transmission and discard it. The cryptographically *unprotected PHY header* allows an attacker to pick every value for the frame length, leaving the

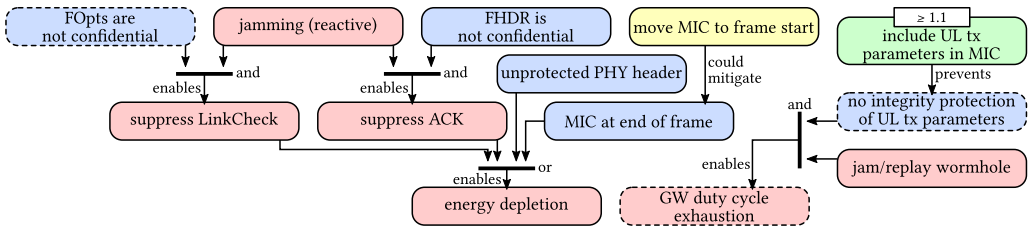


Fig. 19. Attacks, vulnerabilities, and countermeasures related to resource exhaustion.

ED's radio in receive mode for the whole frame transmission period. The results by Mikhaylov et al. show that the lifetime of a frequently transmitting device can be shortened to less than a third using this attack. For sporadically transmitting devices, however, the energy consumption during sleep is dominant, and the effect of the attacker remains small.

Another approach to increase transmissions of an ED is to interfere with mechanisms that use the downlink to deliver feedback to the ED. Using reactive jamming, an attacker can suppress this delivery, which requires the ED to retransmit the same message. Confirmed messages are one of these mechanisms, Nafees et al. [65] experimentally analyze them. If the attacker *suppresses the ACK*-containing message, the ED retransmits the frame several times before giving up. The authors observed a five-fold increase in energy consumption for an ED in their setup when not receiving the expected ACKs. Another option is to *suppress LinkCheckAns* messages. The ED can use the LinkCheck mechanism to assure network connectivity and to retrieve information about the up-link link quality. The strategy taken by the ED in case of a missing reply to a LinkCheckReq is not specified. However, it could include increasing the TP or decreasing the DR, both usually related to ADR, or initiating a rejoin. All of these measures increase the consumption of the energy budget. Beltrán-García et al. [7] show a PoC of attacking the LinkCheck mechanism for LoRaWAN 1.0.2. Instead of attacking over-the-air, they simplify their setup and use direct control of ADR via the NS and a compromised GW to break connectivity and drop messages. Their results show that targeted attacks on network management can indeed increase energy consumption. Partial mitigation, standardized in LoRaWAN 1.1, is the encryption of FOPts, which makes it harder to identify LinkCheck messages.

Talking about countermeasures, Mikhaylov et al. [60] motivate the need to verify the authenticity of the frame already during transmission so that EDs can turn off their receiver if it fails. They suggest a partial MIC including frame length, DevAddr, and frame counter, and *moving that MIC to the frame start*. By checking these parameters early, the impact of injecting malicious frames would be limited, as the radio can be turned off after a fixed time.

In contrast to EDs, GWs are usually not limited by energy constraints, and the incoming data rate from the LoRa link is low enough not to exhaust their Internet uplink. For duty cycle restricted regions, another scarce resource is the transmission time. GWs must follow the same limitations as EDs, which means a maximum transmission duty cycle of 1% to 10% in the EU863-870 region. To perform a *GW duty cycle exhaustion* attack, an attacker tries to force a GW into transmitting as many downlink messages as possible, each with a maximized time on air. Not being able to forge messages, they can only forward previously sniffed messages to the GW. Ideally, those messages should have properties requiring an answer by the network, like the ACK or ADRACKReq flag, or contain MAC command requests needing an answer. To maximize the time on air, the attacker can collect these messages and forward them with the lowest DR possible. As the rx1 receive window usually has the same or a lower DR than the uplink, this will maximize the duty cycle budget spent for answering.

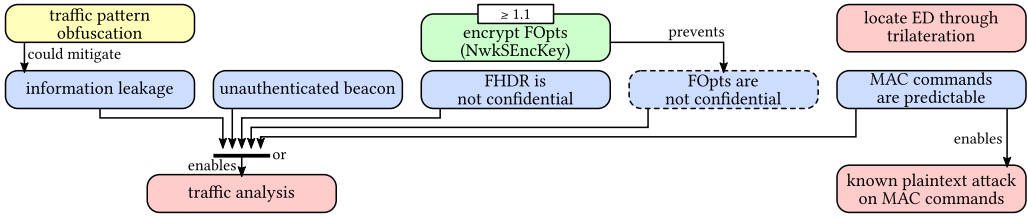


Fig. 20. Attacks, vulnerabilities, and countermeasures based on analyzing LoRaWAN traffic.

Changing the transmission parameters is only possible if *no integrity protection for the uplink tx config* is in place. The protection is missing in all LoRaWAN 1.0.x versions. LoRaWAN 1.1 includes the *uplink tx parameters in the MIC* (cf. Figure 6), meaning the NS will reject the message if it is received on the wrong frequency.

6.5 Passive Attacks

We define passive attacks, as shown in Figure 20, as all actions an attacker takes that do not require transmission on the channel or access to target devices. The main goal is to gain information about the network or specific EDs. Knowing about traffic patterns may, e.g., support jamming attacks, as shown in Figure 14. Depending on the application and sensor type, the timing of messages may reveal the occurrence of underlying events.

We summarize all observations an attacker can make under *traffic analysis*. In his thesis, van Es [92] gives a detailed overview of messages in LoRaWAN 1.0.2. We discuss publicly available information in the context of reactive jamming in Section 6.1.2 to show that the FHDR is not confidential and, for LoRaWAN 1.0.x, also the FOpts are not confidential. Butun et al. [10] briefly mention the threat and illustrate it using an example of a building-wide network that could leak activity information. In [11], Butun et al. mention traffic analysis using compromised GWs impersonating real GWs. They assess a high risk for availability. Coman et al. [15] motivate traffic analysis in a study comparing different LPWAN technologies and also emphasize that metadata can reveal information without breaking payload encryption. Ingham et al. [40] observe traffic to measure the transmission periodicity of EDs in an experiment to schedule transmissions for periodic jamming.

Leu et al. [46] formally define how traffic analysis of transmission timing information can leak data. They define two forms of *information leakage*, namely existential and statistical leakage. For existential leakage, the observation of a single message implies that a specific event has occurred, revealing the existence of that event. Statistical leakage uses deviations in transmission patterns to deduct changes in the data observed by the ED. For both types, an attacker needs information about the application type for interpreting the data but then can infer the content of messages, impairing confidentiality or privacy. The authors propose *traffic pattern obfuscation* to reduce or prevent information leakage. Their solution considers transmission power consumed for the traffic pattern adjustments. In Spadaccino et al. [86] propose DEVIL (DEvice Identification and privacy Leakage), which performs *ED de-anonymization* by combining information leakage based on message intervals and unencrypted FHDR parameters.

Apart from information about application data and network management, an attacker could also be interested in the network's physical topology, e.g., the positions of end devices. The LoRaWAN specification defines methods for network-based device *localization of EDs through trilateration*. The requirements for localizing a specific ED are relatively simple: (1) at least three receivers within reach to receive frames from the ED, (2) a precise clock synchronization to timestamp these frames,

and (3) the possibility to map frames to the ED. The last requirement is easy to achieve, as the DevAddr or DevEUI are transmitted without confidentiality protection. Most COTS GW devices meet the hardware requirements for timestamping. Therefore, an attacker who can distribute GWs over a certain area can localize all EDs in that area. Experimental evaluation has shown a precision of 100 m to 200 m [27]. Gladisch et al. [30] mention the two main principles used for trilateration: RSSI and time-difference-of-arrival. Butun et al. [11] model trilateration based on compromised GWs, using the existing infrastructure of a network. In that case, the attacker needs to be active to compromise the GWs, while the attack as such only requires observation of the channels.

Another focus for traffic analysis can be the network management information in the MAC commands. Even though LoRaWAN 1.1 mandates encryption of FOpts, and MAC commands are encrypted in FRMPayload in all versions, certain states of the protocol allow an outsider to make predictions about the type and payload of transmitted commands. For example, directly after joining a LoRaWAN 1.1 network, an ED will transmit a RekeyInd command and get a RekeyConf reply in the first downlink [12]. Canard and Ferreira [12] discuss the possibility of predicting certain MAC command strings to perform a *known plaintext attack on MAC commands*. If an attacker knows the full or partial ciphertext of the MAC command string and the plaintext representation, they can derive the keystream used for encryption due to operating AES in counter mode. According to Canard and Ferreira, collecting the keystream samples allows for offline guessing of the underlying session key. Hessel et al. [37] use *predictable FOpts* to identify the message containing the LinkAdrReq for their ADR spoofing attack (cf. Section 6.2). Forwarding a frame with the correct length of the MAC command string is a good indicator that the ED will adjust its DR.

Beyond listening to the messages between EDs and GWs, an attacker can also observe *unauthenticated beacon* frames. They lack both confidentiality and integrity protection and can reveal information about the network or the GWs transmitting them. The beacon and an ED's DevAddr determine the timing of Class B downlink windows. Furthermore, beacons can contain the transmitting GW's GPS location, or they can be used to triangulate the GW, similar to locating EDs. Information about GW locations enables an attacker to strategically place jammer and sniffer devices, as illustrated in Section 6.1.2.

6.6 Attacks on the Join Process

The OTAA join process is an essential mechanism for the protocol's security, as it implements the session generation and key derivation. Attacks in this area can affect confidentiality and integrity in case keys are reused or become publicly known, and availability in case NS or ED derive different or unusable session keys. Figure 21 gives an overview of all vulnerabilities identified in the literature. In contrast to other areas, LoRaWAN 1.1 addresses all vulnerabilities in this section, which improves security significantly.

6.6.1 Replay Attacks. Central aspects of OTAA security are the DevNonce and AppNonce/JoinNonce. Before LoRaWAN 1.1 and 1.0.4, the OTAA process relied on unique random numbers for freshness. Keeping track of these arbitrary values to prevent replay attacks is already a resource-intensive task for the NS but impossible to implement with the limited resources of an ED. Consequently, *repeating nonces* can occur, leading to the generation of the same session keys. In his thesis, Zulian [101] discusses this problem and gives detailed insights into the effects and ways to handle detected duplicates. You et al. [100] analyze the join process and identify the problem of keeping track of nonces as a major issue. While admitting that LoRaWAN 1.1 solves the problem, they use their results to propose to replace key derivation with a handshake based on **elliptic curve cryptography (ECC)** for LoRaWAN 1.0.2. Dönmez and Nigussie [24] discuss the impact of this issue in backward compatible setups and assess backward compatible networks as vulnerable

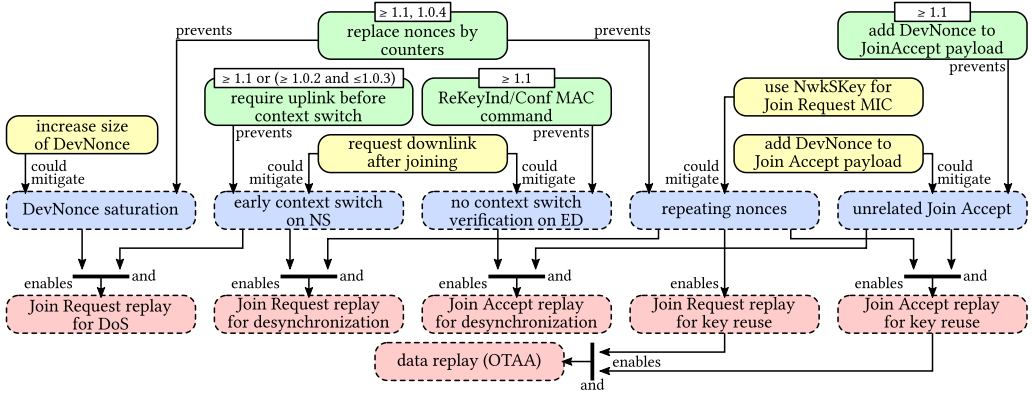


Fig. 21. Attacks, vulnerabilities, and countermeasures related to the join process.

to repeating nonces. Hessel [38] demonstrates experimentally how different ED implementations process nonces. The findings show for LoRaWAN 1.0.3 that none of them, keep track of more than one AppNonce.

One attack directly related to repeating nonces is the *Join Request replay for keystream reuse*. For this attack, the attacker collects Join Requests and data messages exchanged in the subsequent sessions. Afterward, the attacker starts replaying Join Requests. Should the NS answer with a Join Accept using the same AppNonce and the ED gets assigned the same DevAddr, all collected frames become valid again and can be replayed. Additionally, the legitimate ED is disconnected until it attempts rejoining. However, assuming the NS picks the 24-bit AppNonces randomly, a successful attack is highly improbable at $1:2^{24}$, even for the same DevAddr. Avoine and Ferreira [4] analyze the security measures of LoRaWAN 1.0 and identify this attack as an enabler for replay attacks. Wesemeyer et al. [96] support the finding in [4] through their formal analysis of key management in LoRaWAN. Eldefrawy et al. [26] formally analyze the join process using Scyther [18] and find this attack applicable for LoRaWAN 1.0 and mitigated in LoRaWAN 1.1. Dönmez and Nigussie [23] call this attack “replay and eavesdrop via fake session on NS.” They conclude that a LoRaWAN 1.1 network is secure against it. In [24], they discuss backward-compatible setups and conclude that the attack is only applicable if the NS is unaware of LoRaWAN 1.1. If the ED handles backward compatibility, the attack remains possible.

The OTAA process suffers from a vulnerability similar to the unrelated downlink described in Section 6.2, the *unrelated Join Accept*. Figure 4 shows how the payload and MIC of the Join Accept are entirely independent of the Join Request in LoRaWAN 1.0. Multiple authors [4, 92, 101] discuss this as a vulnerability exploitable to enable replay attacks in the join process of LoRaWAN 1.0. Wesemeyer et al. [96] provide a formal analysis concluding that the DevNonce, which guarantees freshness in the handshake, has to be included in the Join Accept message to make the handshake sound. Van Es [93] states that this vulnerability is mitigated in LoRaWAN 1.1. While the conclusion is correct, replacing nonces with counters alone would not be sufficient as attackers can forcefully drop messages and use them for replay. Dönmez and Nigussie [24] extend the applicability assessment stating that also backward-compatible setups are affected by this vulnerability. Eldefrawy et al. [26] provide a formal verification of the vulnerability in LoRaWAN 1.0 and its mitigation in 1.1 using Scyther. Hessel [38] demonstrates the vulnerability experimentally as part of a replay attack, combining different versions of the protocol on ED and NS.

The unrelated Join Accept and repeating nonces result in *Join Accept replay for keystream reuse*. Here, the attacker collects Join Accept messages, including the DevNonce, and downlink frames

of the following session for a specific ED. Even though the Join Accept is mostly encrypted, the timing correlation between Join Request and Accept and the following uplink can be used to match it to DevEUI and DevAddr. The attacker uses reactive jamming for the following Join Request and replays a stored Join Accept to the ED. As the attacker can pick a Join Accept message for the right DevNonce from the collected messages, the attack is much more probable than the Join Request replay. If the ED accepts the replayed Join Accept, the attacker is in the position to pretend to be the NS. Avoine and Ferreira [4] mention this as an enabler also for more sophisticated replay attacks. Eldefrawy et al. [26] provide a formal verification of this attack in LoRaWAN 1.0 and its mitigation in 1.1. In their discussion, Dönmez and Nigussie [23] call this attack “replay and eavesdrop via fake session on ED” and claim it to be inapplicable to LoRaWAN 1.1. In [24], they focus on backward-compatible setups and state that the attack is impossible if the ED is 1.1-aware and connected to a 1.0-only NS, as the ED does not reuse DevNonces then. The attack remains applicable in the opposite direction, with a 1.0-only ED but a 1.1-enabled NS.

Both replay attacks discussed before can serve as enabling attacks for a *data replay attack on OTAA*. Avoine and Ferreira [4] illustrate how the replay attacks for keystream reuse can lead to a whole spoofed session, either towards the ED or the NS.

6.6.2 Context Switch Verification. For the session renewal and key derivation, both communicating parties must agree on the same session context and confirm it with each other. The confirmation missing in LoRaWAN 1.0.x. First, we discuss the *missing context switch verification on the ED* before we get to the NS. After sending a Join Request and receiving a valid Join Accept in return, an ED assumes to have successfully joined the network in LoRaWAN 1.0.x. It directly transitions from the joining state to the joined state and assumes to be able to transmit uplink messages. Van Es [92] briefly mentions this issue for LoRaWAN 1.0.2 in preparation for the following simulations showing session state desynchronization. Dönmez and Nigussie [24] discuss this vulnerability for backward-compatible setups and assess it to be applicable, even though 1.0.x EDs or NSs could use regular data traffic following an OTAA join process as verification.

Together with the repeating nonces, the missing context switch verification on the ED enables *Join Accept replay for desynchronization*, which targets availability instead of exploiting keystream reuse. The idea of this attack is to interfere with OTAA in a way that the NS and ED both assume to have a valid session, but these sessions differ, making communication impossible. To achieve this goal, the attacker can replay any previously captured Join Accept JA1 message in response to a new Join Request JR2 from the ED, as long as the ED does not reject it during AppNonce verification. By jamming the Join Request JR2 on the NS side, the NS stays with the old session based on DevNonce_{JR1} and AppNonce_{JA1}, while the ED uses DevNonce_{JR2} and AppNonce_{JA1} to derive new, different session keys. Zulian [101] describes the attack for LoRaWAN 1.0, then discusses the options from the ED’s perspective: Detecting the underlying Join Accept replay requires keeping track of AppNonces, which is hard, and even if an attack is detected, the ED can do no more but reject the session, also leading to a DoS. Avoine and Ferreira [4] also explain and present this attack as one way to “disconnect an ED from the network.” Naoui et al. [67] provide a formal analysis of the attack for LoRaWAN 1.0 in Scyther to propose a join process using a trusted third party, similar to what would become the JS in LoRaWAN 1.1. Eldefrawy et al. [26] also use Scyther, extending the results of Naoui et al. with insights on LoRaWAN 1.1. According to their findings, the updated specification is no longer affected. Dönmez and Nigussie [23, 24] discuss this attack calling it “DoS on ED via Join Accept replay” for LoRaWAN 1.0.2 and 1.1. They conclude that the attack is impossible in homogeneous 1.1 setups, but all other constellations, including backward-compatible ones, remain affected. Van Es et al. [92, 93] model this attack in their simulation based on colored petri-nets. Their simulation results underline the findings of Dönmez and Nigussie. Hessel [38]

experimentally evaluates this attack by combining different popular software stacks and versions for the ED and the NS. He concludes that only homogeneous 1.1 setups are secure against the attack, even though EDs remain in join state while the attacker is active. In contrast to 1.0 EDs, they notice the missing context verification and try to rejoin directly. He points out that, as none of the ED implementations keeps track of the AppNonce/JoinNonce, a single Join Accept message can be used repeatedly by the attacker in LoRaWAN 1.0.

As mentioned, both communicating parties miss confirming the session context switch mutually in LoRaWAN 1.0, so now we discuss the *early context switch on the NS*. If the NS derives a temporary session context when receiving a valid Join Request and makes it permanent as soon as it sends the Join Accept message, it is unclear if the ED received and applied the new context. Zulian [101] describes this issue from the security perspective, as an attacker can directly modify the agreed session context with a single replayed Join Request if it gets processed.

The early context switch on the NS can, together with the repeating nonces, also be used for *desynchronization by join request replay*. Opposite to the previously discussed desynchronization attack, the invalid session is installed on the NS. To achieve that, an attacker collects Join Requests for an ED, so that they can replay one of them to the NS. The number of Join Requests must be sufficiently large so that at least one of them has a DevNonce no longer stored in the list of used DevNonces on the NS, and thus the Join Request is considered valid again. For LoRaWAN 1.0, multiple authors discuss the attack [4, 64, 101]. Dönmez and Nigussie [23, 24] call it “DoS on ED via Join Request replay” in their work and assess that it does not apply to LoRaWAN 1.1. For backward-compatible scenarios, they state that only a homogeneous LoRaWAN 1.1 network is not affected by the attack. Eldefrawy et al. [26] formally verify the protocol with Scyther and find this vulnerability in LoRaWAN 1.0.x and its mitigation in 1.1. Butun et al. [10] propose a more complicated variant of this attack: Here, the attacker sniffs JR1 and jams it for the NS. Then, when the ED retries with JR2, the attacker replays JR1 at the NS with a stronger signal level and times it so that the Join Accept matches the receive windows for JR2 at the ED. As this requires replaying JR1 nearly synchronized with the transmission of JR2, it demands much higher capabilities from the attacker than replaying the Join Accept itself.

6.6.3 Saturation of Nonces. The last vulnerability in this area is related to the nonces again, particularly the DevNonce. Assuming the NS keeps track of all values, but the ED picks it randomly due to memory constraints, the birthday paradox quickly leads to *DevNonce saturation* as values start repeating. That leaves the NS with two options: (a) ignoring the Join Request, so that the ED tries a different DevNonce or (b) suspecting a replay attack and deactivating the ED. Both approaches enable a *Join Request replay for DoS*. If the NS deactivates the ED completely after observing a DevNonce for the second time, an attacker can trivially replay any captured Join Request to deactivate the respective ED. If the NS has a FIFO-like cache of size n for the last observed DevNonces to be ignored, the attacker can collect $n + 1$ Join Request messages with different DevNonces and rotate replaying them. That requires an early context switch on the NS, as otherwise, the existing session of the ED would not be canceled just by receiving a Join Request. We can assume a NS implements an upper limit on cached DevNonces, as DevNonce saturation would degrade the performance of EDs otherwise. Zulian [101] emphasizes the high probability of repeating DevNonces and the requirement for a strategy to handle when observed by the NS. He shows that if a FIFO-like strategy is used and its size known to the attacker, said attacker could prevent a device from joining the network. Tomasin et al. [88] continue this work by modeling DevNonce repetition mathematically. Thereby, they predict the time of operation after which an ED would be excluded from the network if a repetition would be seen as a potential attack and lead to the deactivation of the ED. Kim and Song [42] discuss DevNonce saturation for LoRaWAN 1.0.2.

They provide predictions for the repetition probability based on measurements with a HWRNG and for various proposed mitigations, like increasing the length of DevNonce to come up with a customized replay prevention scheme.

6.6.4 Countermeasures. We proceed from right to left in Figure 21, starting with measures against the unrelated Join Accept. A solution proposed in [101] is to *add the DevNonce to the payload of the Join Accept* message. While this clearly binds the Join Accept to the issuing Join Request, it would increase its payload by 2 bytes. Instead, the LoRaWAN 1.1 specification uses an approach similar to confirmed data messages by *including the DevNonce in the Join Accept MIC*. As also explained in [23], this binds the validity of the Join Accept to the preceding Join Request. Wesemeyer et al. [96] demonstrate the effectiveness of this measure in their formal protocol analysis.

A central vulnerability is repeating nonces, a precondition for more than half of the attacks presented in this area. Kim and Song [42] propose, based on LoRaWAN 1.0.2, to let the ED sign subsequent Join Requests with the previous session's NwkSKey. Mainly focusing on the *Join Request replay for DoS*, their idea is to require a different MIC in the Join Request even if the DevNonce repeats, making replay attacks impossible. In the revised specification of LoRaWAN 1.1 [49, Section 6.2.3], the former nonces become counters. Thereby, the specification focuses on the primary function of DevNonce and the renamed JoinNonce, to guarantee freshness, as pointed out in [88, 101]. Secrecy or unpredictability is not required, as integrity and confidentiality are established using the root keys during OTAA. The LoRaWAN 1.0.4 specification does not directly include the change [52, Section 6.2.6], but instead refers to the technical recommendations for the join process [51], which then recommends using a counter. Therefore, we consider this vulnerability to be mitigated in 1.0.4 as well. Naoui et al. [66, 67] formally verify a LoRaWAN 1.0.2 extension, which replaces nonces with timestamps, a special case of counters. Their verification shows this can mitigate the vulnerability at the cost of requiring a time source on each ED. [93] mentions this measure as mitigation of the Join Accept replay for desynchronization. However, this measure alone cannot mitigate the attack entirely, as the attacker can replay Join Accept messages that the ED has not seen before, e.g., due to jamming.

Next, we discuss the missing context switch verification on the ED, leaving an ED uncertain whether it has derived the same session as the NS. Any valid downlink message received by the ED in the session can act as an implicit confirmation of having the same session data as the NS. Otherwise, the NS would not answer, or MIC verification would fail. Consequently, Dönmez and Nigussie [24] propose to *request downlink after joining*, and if none is received, to return to the joining state soon after. An ED could achieve this in a LoRaWAN 1.0.x network by starting each session with a confirmed message. LoRaWAN 1.1 enforces an initial bidirectional message exchange by introducing the *RekeyInd* and *RekeyConfMAC* commands [49, Section 5.10]. From the ED's perspective, sending a confirmed uplink or a message containing RekeyInd provides an equal opportunity to reach context switch verification. Therefore, an application developer can include this measure even in homogeneous LoRaWAN 1.0 networks.

The introduction of the RekeyInd and RekeyConf also partly addresses the early context switch on the NS by allowing it to verify the successful session creation on the ED. In addition, the NS must be able to keep multiple pending session contexts for each ED and to *require an uplink message before context switch* confirmation. As soon as the first uplink is received, all other pending contexts for the ED can be dropped. The attacker can then no longer spam the server with Join Requests to force dropping the context of the legitimate ED. The requirement to keep pending contexts is already introduced in LoRaWAN 1.0.2, one of the few security-relevant and non-breaking changes made within the 1.0.x branch of the specification.

The remaining vulnerability, DevNonce saturation, is already addressed when counters replace nonces. Using monotonically increasing numbers optimizes utilization of the whole 16-bit

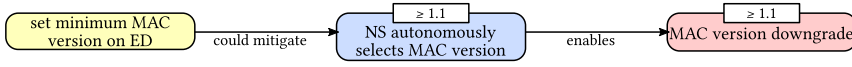


Fig. 22. Downgrade attack scenario.

DevNonce space. The issue was discussed in literature before the mitigation was introduced in LoRaWAN 1.1. For example, Zulian [101] proposes to *increase the size of the DevNonce* to reduce collision probability and thus diminish the effects of saturation.

6.7 Downgrade Attack

The long lifetime of EDs implies that a quick network-wide transition to a new specification version is not feasible. LoRaWAN counters this situation by introducing backward-compatibility measures. That means falling back to the 1.0.x protocol if either the ED or NS does not support LoRaWAN 1.1. As the level of security for LoRaWAN 1.0.x and the backward-compatible operation is lower, deciding on the version to use is also a security-critical task. Figure 22 gives an overview of downgrade attacks geared toward this mechanism.

When an ED joins the network using OTAA, the MAC version should be known to the NS [49, p. 56] to use the proper encryption scheme. The NS then sends a Join Accept message according to the selected MAC version and uses the OptNeg flag to inform the ED whether to use the 1.0 or 1.1 implementation. In the case of 1.1, the minor protocol version is agreed upon in the RekeyInd/RekeyConf exchange, in which the NS makes the final decision. In short, *NS autonomously selects the MAC version* to be used with a certain ED during OTAA. When choosing a fallback to 1.0.x, the ED has to use the single-root-key scheme with only an AppKey [49, Section 6.1.1.3], even if it is commissioned with multiple keys. A malicious NS can use this to run a *MAC version downgrade* attack against EDs. With only a single root key, the strong separation between the application provider and network operator breaks, allowing the network operator to derive and keep the AppSKey and decrypt, encrypt, or forge messages for the ED. Dönmez and Nigussie [23] discuss the possibility of such downgrade attacks in their analysis of the LoRaWAN 1.1 join process, showing that the delegation of key derivation no longer holds in this scenario. Canard and Ferreira [12] also mention downgrade attacks in their discussion of the LoRaWAN 1.1 protocol's cryptographic flaws but put the focus on the ability to run 1.0 attacks against the ED after a downgrade.

A countermeasure proposed in both mentioned publications is to *set the minimum MAC version on the ED*. If the NS selects a version below the minimum, the ED aborts OTAA. While this mitigates downgrade attacks, it prevents the device from joining a legitimate LoRaWAN 1.0 network. Dönmez and Nigussie [23] propose this countermeasure but consider applying it only for security-critical devices. They also briefly propose a scheme in which the AppSKey is always derived from an AppKey known to the device owner, which would not comply with the 1.0 specification. Based on their formal protocol analysis, Canard and Ferreira [12] conclude that all network entities, including the NS, should not fall back to a protocol version below 1.1 for secure operation.

6.8 Frame Counter Repetition and Keystream Reuse

This section discusses circumstances in which the frame counter repeats and how this can be exploited. Figure 23 shows related concepts.

If an activation by personalization (ABP) device without non-volatile memory (NVM) resets, it also reinitializes its frame counter to 0. In addition, ABP devices have no means to refresh their session keys or the DevAddr. Therefore, ABP devices are particularly vulnerable to *counter resets*. If the NS strictly enforces monotonically increasing counters, counter resets also pose a risk to

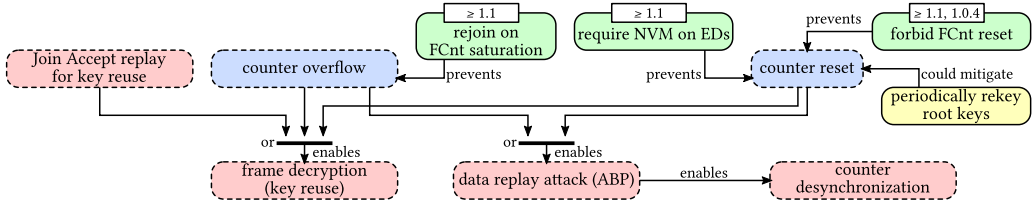


Fig. 23. Attacks, vulnerabilities, and countermeasures related to frame counters.

availability, as messages from an ED are ignored until the former counter value is reached. Yang [98] implements a PoC for exploiting this vulnerability in LoRaWAN 1.0.2 to weaken confidentiality. In [99], Yang et al. extend this PoC and state that the issue is resolved in LoRaWAN 1.1. Aras et al. [2] discuss the problem of reset devices being unable to communicate with the network in LoRaWAN 1.0 if the NS does not accept resetting frame counters. Accepting them, however, comes with the threat of replay attacks. Dönmez and Nigussie [24] discuss the frame counter reset in backward compatibility scenarios and conclude that the issues affect all but homogeneous LoRaWAN 1.1 sessions. Gladisch et al. [30] discuss the risk of counter resets, but refer to the requirement in LoRaWAN 1.1 to store frame counters in persistent storage. Skorpil et al. [82] show a PoC of this vulnerability with a compromised GW so that they can display and record forwarded messages. Naoui et al. [66] mention counter resets as a risk in a smart home case study. They claim gathering plaintext-ciphertext pairs allows an attacker to forge arbitrary packets. However, they overlook that the keystream can only forge payload encryption, not the MIC.

Another cause for frame counter repetitions is *frame counter overflow*, particularly for small counter width of 16 bit. LoRaWAN 1.0.4 and 1.1, therefore, mandate 32-bit counters and explicitly forbids counter overflows. However, in the case of an ABP device, by disallowing counter overflows, counter saturation means the ED reaches its end of life. Yang [98] discusses counter overflows focusing on DoS attacks and breaking confidentiality for LoRaWAN 1.0.2. In [99], Yang et al. claim this issue to be still relevant for LoRaWAN 1.1, ignoring the specification updates. Similarly, Dönmez and Nigussie [24] investigate the vulnerability and claim applicability for backward compatibility scenarios and LoRaWAN 1.1, while the specification does not back the latter. According to Gladisch et al. [30], waiting for a 32-bit counter overflow is unrealistic and also hard to track for the attacker due to encryption. That is only partly true, as the 16 least significant bits (LSBs) of a frame are transmitted in plaintext. However, the 16 most significant bits (MSBs) are not sent over-the-air and must be tracked by an attacker. The authors also state that LoRaWAN 1.1 already addresses this vulnerability by disallowing overflows. Oniga et al. [70] discuss frame counter overflows for LoRaWAN 1.0 and how they can be addressed by rekeying through OTAA when the counter is about to overflow. Therefore, they suggest only using OTAA devices for their proposed secured LoRaWAN deployment. Naoui et al. [66] mention counter overflows and resets in the aforementioned smart home case study.

One attack caused by counter overflow and counter reset is *frame decryption through keystream reuse*. If the session key and DevAddr do not change and the same frame counter c is used, the same keystream k_c is calculated. Miller [62] mentions that this is the case for a counter reset in ABP mode. Given two plaintext-ciphertext pairs (P_1, C_1) and (P_2, C_2) , with $P_1 \otimes k_c = C_1$ and $P_2 \otimes k_c = C_2$, knowing P_1 and C_1 allows deriving k_c . If then C_2 is observed, exploiting $P_1 \otimes P_2 = C_1 \otimes C_2$ allows decrypting P_2 . In this case, it is assumed that the attacker knows (or can guess) the first plaintext payload P_1 , e.g., by knowledge about the application payload or expected MAC commands (cf. Section 6.5). In case of ambiguity of possible plaintexts, collecting more than one plaintext-ciphertext pair for a single k_c combined with knowledge about the application reduces the search

space for the right plaintext. Yang [98] show this in a PoC of their crib-dragging attack. The work is extended in [99], stating that the attack is mitigated for LoRaWAN 1.1. Dönmez and Nigussie [24] agree with that, listing this attack as “replay and eavesdrop.” They also analyze the behavior in backward-compatible setups, claiming the attack to be applicable if no countermeasures are in place. Avoine and Ferreira [4] explain how keystream reuse can be caused by a Join Accept replay attack on LoRaWAN 1.0 (cf. Section 6.6). As they point out, it is important that not only are the Nonces picked during the OTAA process the same, but also the DevAddr must not change. Otherwise, session keys are equal, but keystreams for specific frames still differ. Wesemeyer et al. [96] formally verify keystream reuse in OTAA for LoRaWAN 1.0.2 using Tamarin.

Keystream reuse also enables the *data replay attack*. As shown, even with the knowledge of the encryption keystream k_c , the MIC prevents the forging of arbitrary messages. An attacker can, however, replay messages if frame counter repetitions do not lead to message rejection. By that, the integrity of application data and network management messages can be affected. This attack is enabled through counter reset of ABP devices or overflow with both activation types. Yang [98] present a PoC for this attack as an intermediate step leading to counter desynchronization.

As the data replay attack inserts valid messages into the MAC processing at the NS, it can be used for a *counter desynchronization* attack, affecting the ED’s availability. NSs which enforce frame counter validation will only accept messages with a higher frame counter than observed before. An attacker possessing a large set of recorded frames from an ED can replay selected frames to quickly increase the expected frame counter at the NS after a counter reset or overflow can be observed for the ED. The fresh frames from the ED will then be rejected due to their frame counter being too low. Yang [98] provides an over-the-air PoC for this attack against a TTNv2 NS running LoRaWAN 1.0.2. In [99], Yang et al. extend their work and show that it does not apply to LoRaWAN 1.1 networks. Skorpil et al. [82] provide a PoC of this attack based on a modified forwarder on a *compromised GW*. As GWs just forward messages, they are suitable to create a database of frames for EDs in range and replay those directly to the NS when a counter repetition is detected. Naoui et al. [66] discuss the attack as part of the Smart Home case study. Cerrudo et al. [13] state that an attacker possessing root keys can also easily increase the frame counter on the NS. Coman et al. [15] propose a different approach to achieve counter desynchronization. They use a compromised GW with more BW to the NS to run MIC brute-forcing for a replayed frame. If they succeed and the MIC is accepted, the NS has increased its internal MSBs, meaning the expected frame counter increased by at least 2^{16} .

For ABP devices, the LoRaWAN 1.0.4 and 1.1 specifications contain a dedicated statement that the *FCnt shall not reset* during the whole lifetime of the device [49, 52, Section 4.3.1.5]. LoRaWAN 1.0.3 and below only contain a generic statement that, given the same session keys, the same value of FCntUp shall only be used for retransmissions. Compliance with these requirements mitigates the counter reset vulnerability. Storing frame counters *requires NVM on the EDs*, which is mandatory in LoRaWAN 1.1 [49, Section 6.3], strongly recommended in 1.0.4 [52, Section 6.3], and not required in older versions. Yang et al. [99] state that the introduction of NVM to store frame counters prevents the DoS attack by data replay as frame counters must not reset. They regard a key rollover as a remaining but unlikely issue. Gladisch et al. [30] also mention the requirement for NVM as a countermeasure against counter reset.

OTAA EDs have another option to circumvent counter reuse, as they can *rejoin on FCnt saturation*. In LoRaWAN 1.1, the ED is responsible for rejoining the network before the frame counter is saturated. In his discussion, Yang [98] proposes this as a solution to data replay attacks in LoRaWAN 1.0.2 limited to OTAA. In [99], Yang et al. also suggest this for LoRaWAN 1.1. Oniga et al. [70] mention rejoining as an important step in hardening LoRaWAN networks. Another measure proposed for OTAA EDs is to *periodically refresh root keys*, as discussed in greater detail in Section 6.10.

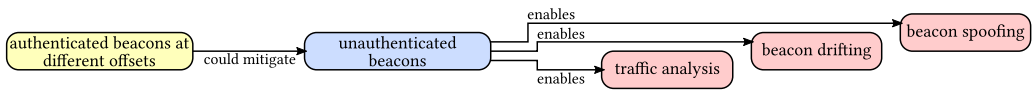


Fig. 24. Attacks, vulnerabilities, and countermeasures related to beaconing.

6.9 Class B and Beaconing

To allow the NS to initiate communication with the ED, the ED must provide additional receive windows. Class B mode (cf. Section 2.4) achieves this by requiring time synchronization on the EDs. Synchronization is provided by periodic beacons, which GWs transmit. In this section, we discuss issues related to this central mechanism for Class B, as shown in Figure 24.

One central idea of Class B beaconing is to have all GWs transmitting the same beacon at the same moment every 128 s. That increases the ED's probability of receiving a beacon and speeds up beacon search, but does not allow for integrity or confidentiality protection of the frames. Keys for beacon protection would have to be shared between all network operators. Therefore, LoRaWAN uses *unauthenticated beacons*, which also allow traffic analysis, as discussed in Section 6.5. The issue was identified early by Miller [62] for the first draft of LoRaWAN 1.0, even before the Class B feature was officially released. In his thesis, Yang [98] explains the missing cryptographic protection of the beacon frames in LoRaWAN 1.0.2 and comments that only non-cryptographic integrity protection exists through the two CRC values. In [99], Yang et al. assess this vulnerability to still exist in LoRaWAN 1.1. Butun et al. support that assessment in their security evaluation of that version [10]. van Es et al. [92, 93] provide a simulation in which they modify the timestamp contained in the beacon. They assume a higher reception probability for the spoofed beacon than for a legitimate one at the target ED. Hessel et al. [37] present experimental results on transmitting beacons at different temporal offsets to desynchronize Class B EDs from legitimate beacons and bind them to attacker-controlled frames.

The Class B beacon contains a time reference and may also contain information about the sending GW. The GW-specific part is separated at the end of the beacon frame, allowing for demodulating the common part even if the GW information collides. As [62] notes, without integrity protection, an attacker can use *beacon spoofing* to manipulate the information the ED receives. Yang et al. [98, 99] discuss this attack for later protocol versions (1.0.2, 1.1) and assess it as still applicable. Butun et al. [10] support this assessment. The simulations by van Es et al. [92, 93] extend the beacon spoofing to a DoS attack by transmitting wrong timestamps. Consequently, the EDs open receive windows at the wrong time, even though the malicious beacons have been transmitted at the right time, together with legitimate beacons. Due to the wrong receive window offsets, downlink traffic is interrupted. Butun et al. [11] also focus on the disruption of the Class B downlink and assess a major risk for availability and a minor risk for all other protection goals.

A variation of beacon spoofing is *beacon drifting*, as introduced and experimentally evaluated in [37]. Instead of constantly competing with legitimate beacons at the ED, the attacker starts by transmitting a beacon synchronous to the network and then slowly shifts the transmission time. EDs follow these small steps and shift their receive windows away from the NS's expectation. In contrast to the spoofing attacks, the effect on availability lasts after stopping transmission, as the ED operates "beaconless" for two hours before falling back to Class A. This attack also affects EDs which synchronized their time using DeviceTimeReq to speed up beacon discovery.

A countermeasure to mitigate the consequences of unauthenticated beacons is transmitting *authenticated beacons at different offsets*, as proposed in [37]. Using a network-wide key for integrity protection of beacons would at least provide some form of protection against spoofing. Transmitting at a network-specific offset allows for the coexistence of multiple LoRaWAN networks

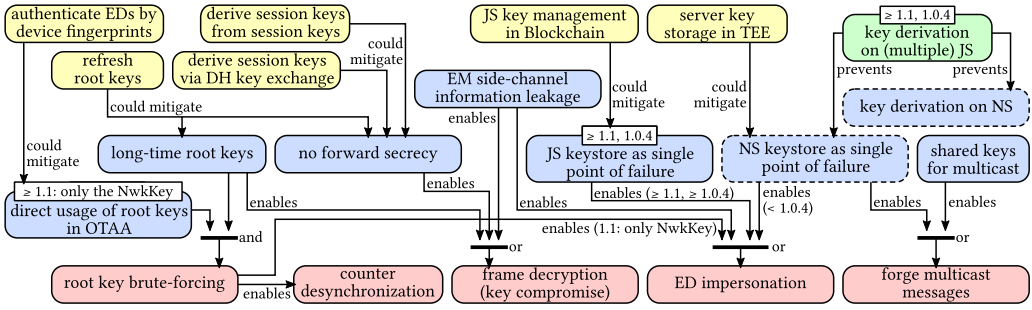


Fig. 25. Attacks, vulnerabilities, and countermeasures related to key management.

without colliding beacons. The key distribution and configuration of the network-specific offset could happen during OTAA or using a dedicated MAC command.

6.10 Key Management

Finally, we discuss issues related to key management, as depicted in Figure 25. A root key compromise lasts for the ED's lifetime if keys are not changed on-site, while a session key compromise of OTAA EDs only affects operation until the next join process. The type of compromised keys also determines the actions an attacker can take. Section 5.1 already discussed some of these implications in the context of getting physical access to an ED. The possession of the AppSKey, e.g., allows an attacker to perform *frame decryption* of the payload. Meanwhile, forging frames and thus a full *ED impersonation* also requires access to the NwkSKeys to calculate the MIC.

Attacks toward a single ED do not create harm for the network as a whole. However, the NS is a *single point of failure* regarding key management in LoRaWAN 1.0–1.0.3, as it stores the AppKey root key of all EDs. As Miller [62] points out, infiltrating the NS keystore means loss of authenticity, integrity, and confidentiality for all EDs, without a chance of recovery. Oniga et al. [70] emphasize that the NS needs optimal protection to prevent unauthorized access to the keystore to create a secure deployment while being a central component of the infrastructure.

The *shared keys for multicast* messages are a special case regarding the key compromise, as the NS manages them directly. Therefore, *forging multicast messages* is enabled by a compromised NS in addition to attacks on EDs.

In LoRaWAN 1.0.4 and 1.1, the key derivation moves from the NS to the JS. While this seems like making the JS a *single point of failure*, using multiple JSs limits the impact of an attack on the keystore to affecting only EDs managed by the individual JS. These EDs, however, remain permanently compromised. Ribeiro et al. [76] discuss the key derivation on the JS for LoRaWAN 1.1, focusing on availability. They find the JS to be a target for DoS attacks, which in turn would prevent EDs from joining the network, thus decreasing their availability. As a solution, they propose to run JS key management in a Blockchain, using Hyperledger Fabric and Smart Contracts, which they evaluate regarding processing speed and node count.

Discussing individual EDs again, an attacker can exploit the usage of root keys in the OTAA join process to try key extraction. Join Request messages are protected using the AppKey (LoRaWAN 1.0.x) or NwkKey (LoRaWAN 1.1). Join Accept messages are also encrypted with the respective root key (cf. Figure 4). Therefore, collecting a pair of these messages allows offline root key brute-forcing, as the Join Request payload is plaintext. An attacker can then compute the MIC with key candidates and compare it to the MIC in the captured frame. Then, to resolve ambiguities caused by the truncated CMAC used as MIC, the attacker can try decrypting a Join Accept to verify

key candidates. However, the key space being all 128-bit AES keys limits the practical applicability. In LoRaWAN 1.1, an attacker can only retrieve the NwkKey using this method. As LoRaWAN 1.0.x uses the AppKey as a single root key, successfully guessing it affects both application and network traffic. Cerrudo et al. [13] propose *root key brute-forcing* as an attack against LoRaWAN 1.0.2. They explain how an attacker can use either a Join Accept or a second Join Request for the same ED to verify key candidates. To reduce the impact of key compromise and make it detectable, Chen et al. [14] suggest to *authenticating EDs by device fingerprints*, effectively using physical layer properties as a second factor for authenticating a message. They conduct experiments and propose a scheme for integrating fingerprint measurements into a LoRaWAN stack. An earlier experimental work by Robyns et al. [78] on fingerprinting LoRa devices supports the results. Both publications show that more research is needed to create stable fingerprints with only negligible environmental effects.

A major issue of compromised key material in LoRaWAN is the *long lifetime of root keys*. All session keys are derived from a single (LoRaWAN 1.0.x) or two (LoRaWAN 1.1) 128-bit AES keys. Changing or refreshing these root keys is not considered by the specification. With a planned lifetime of ten years or more without on-site maintenance and continuously increasing computational power, EDs considered secure today might become more vulnerable before their planned end of life and prone to attacks like brute-forcing. Han and Wang [33] compare the key scheme of LoRaWAN to NIST guidelines [6], pointing out that the expected lifetime exceeds the recommended validity period for symmetric keys. Mårind and Butun [56] also discuss this vulnerability, questioning whether symmetric cryptography is the right choice for the OTAA protocol. Dönmez and Nigussie [25] investigate two specific use cases for LoRaWAN devices: transferable EDs, which change their owner frequently, and personal wearables, which might be left unsupervised from time to time. In both cases, the current device owner can only have limited confidence in the secrecy of the root keys.

All these authors propose different schemes to *refresh root keys*, mitigating the effects of long-term key usage. Han and Wang [33] propose a rekeying scheme based on a pseudo-random function, taking the old root keys and former session keys as input. The pseudo-random function uses the rabbit stream cipher, which the authors claim is efficient. Mårind and Butun [56] show a PoC for root key refreshing based on **elliptic curve Diffie-Hellman (ECDH)** during OTAA. They measure a 77% increase in energy consumption compared to plain LoRaWAN, which they justify by claiming that rekeying is not a frequent process. Milani and Chatzigiannakis [61] propose a backward compatible rejoin mechanism based on ECDH to refresh root keys. Their evaluation in the FIT IoT-Lab testbed shows an acceptable time and energy overhead. For wearables and personal devices, Dönmez and Nigussie [25] propose rekeying root keys through a secure channel, e.g., using physical proximity. They describe adding new key material to the ED as recharging its “security battery,” much like a regular battery has to be recharged. In addition, Naoui et al. [66] propose a measure against long-term root keys while discussing counter resets (cf. Section 6.8). Their approach uses a **one-time password (OTP)** generator, which is not further specified, to change root keys. Hayati et al. [36] warn that unsupervised EDs might become physically compromised and propose a periodic root key update scheme between ED and JS. They verify the security properties of their approach formally using Scyther.

Related to the long-term root keys is the vulnerability of *missing forward secrecy*, as all session keys are directly derived from the ED’s root keys. The DevNonce and AppNonce (or JoinNonce, for LoRaWAN 1.1) introduce freshness during the OTAA key derivation. An attacker in possession of the root keys can retrieve these values from an OTAA message exchange, as the DevNonce is sent in plaintext, and the AppNonce/JoinNonce in the Join Accept message is encrypted only by the AppKey (or NwkKey, for LoRaWAN 1.1). If the attacker observes any OTAA process, they can derive the session keys and decrypt data messages from that session, even years later. In [25],

Dönmez and Nigussie mention achieving forward secrecy as another advantage of their physical rekeying approach. Before, they identified missing forward secrecy as a potential risk in their analysis of the OTAA join process [23]. Hayati et al. [36] state that their periodic rekeying of root keys also addresses the forward secrecy issue.

Forward secrecy is not addressed by any LoRaWAN specification. Research on the matter goes in two main directions, with the first one being *deriving session keys from session keys*. As Dönmez and Nigussie [23] illustrate, deriving session keys from the previous session keys instead of the root keys would create a certain level of forward secrecy. Besides getting access to the root keys, in this scheme, the attacker would have to record all OTAA-related messages back to the initial join process. After the initial join process succeeds, the root keys can be purged from ED and NS to reduce the risk of compromise. The work of Tsai et al. [90] mainly focuses on optimizing the energy consumption of used encryption primitives but also includes a proposal for refreshing session keys based on the current keys. They propose new message types between ED and JS, with the latter informing NS and AS about new keys after the process completes. The proposed rekeying handshake has some functional overlap with the Rejoin Requests in LoRaWAN 1.1.

The second approach towards reaching forward secrecy is using a **Diffie-Hellman (DH)** key exchange and ephemeral keys for session key creation. That implies extending the requirements on EDs capabilities, as the cryptographic primitives have to be extended from only `aes128_encrypt` to support asymmetric cryptography, which has higher demands on memory, computational power, and energy. Sanchez-Iborra et al. [80] propose using an elliptic curve Diffie-Hellman (ECDH) scheme for refreshing the root keys after OTAA. They state that for the analyzed LoRaWAN 1.0, this would prevent session key recovery using a compromised AppKey. You et al. [100] also identify missing forward secrecy as a vulnerability in their analysis of the LoRaWAN 1.0.2 join process. They address it in their solution similar to DTLS, also using elliptic curve cryptography (ECC), but with a considerable communication overhead. Kamkuemah [41] presents a formal analysis of missing forward and future secrecy and shows how ECDH can be used to achieve it in LoRaWAN 1.0.3.

In LoRaWAN 1.0–1.0.3, the separation between the network operator and application provider is incomplete because *key derivation happens on the NS*. Naoui et al. [67] mention how a malicious network server can keep the AppSKey to monitor or modify applications payloads. Bala et al. [5] discuss the separation of NS and AS and the missing end-to-end protection of payload data from ED to AS for LoRaWAN 1.0. They propose a scheme based on two root keys, similar to what LoRaWAN 1.1 introduces but with a dedicated join process between ED and NS to establish end-to-end security guarantees. De Moraes and Da Conceição [21] compare the situation for LoRaWAN 1.0.2 and 1.1, concluding that moving key derivation to the JS in 1.1 at least preserves the confidentiality of the application payload.

To resolve the missing separation of concerns, LoRaWAN 1.0.4 and 1.1 introduce *key derivation on (multiple) JSs*, so that both NS and AS only receive the keys required for their operation. Naoui et al. [67] proposed a similar change before it became part of the specification. They introduce a trusted third party that can directly communicate with the ED and handle key derivation. The proposal is then formally verified using Scyther.

In [16], Coppolino et al. evaluate moving the *server key storage into trusted execution environments (TEEs)* on the NS to make key extraction harder, even by an inside attacker with escalated privileges. In their PoC, they bind **The Things Network (TTN)** stack's key storage against a secure enclave running on Intel SGX.

6.11 Summary

In this section, we discussed vulnerabilities and attacks exploitable using the wireless LoRa link between EDs and GWs. Table 7 summarizes vulnerabilities enabling PHY attacks, while Table 8

Table 7. Vulnerabilities on the PHY Layer

Vulnerability	1.0/1.0.1	1.0.2/1.0.3	1.0.4	1.1
coexistence issues	● S: [19, 57]; PoC: [95]; E: [2, 3]	● [38]	●	● [38, 71]
FHDR is not confidential	● E: [3]	● [92]	●	● [86]
FOpts are not confidential	●	● [92]	●	● G
M: encrypt FOpts (NwkSEncKey) (1.1)				
information leakage	● ^a	● ^a	● ^a	● ^a [86]; F: [46]
P: traffic pattern obfuscation [46, 86]				
missing spatial integrity	● [70]; E: [47]	● S: [92]; PoC: [87]	●	● [30, 92]; S: [93]; E: [37, 47]
P: use RSSI for device identification [87]				
missing Temporal integrity	● [2]; E: [3]	●	●	● E: [37]
P: time-based watermarks [8]				
no integrity protection of DL tx config	●	●	●	● E: [37]
P: tx config in downlink MIC [37]				
unauthenticated beacon	● [62]	● [98, 99]; S: [92]	●	● [10, 92, 99]; S: [93]; E: [37]
P: authenticated beacons at different offsets [37]				
unrelated downlink	●	● [24, 99]; PoC: [98, 99]; E: [38]	●	● ^b [24, 99]; E: [37, 38]
M: include ACK'ed FCnt in MIC (1.1); P: replied FCnt in message [98], always include FCnt in MIC [37]				

^aapplication-dependent. ^bonly solved for confirmed messages.

M: mitigated in specification (with version). P: proposed countermeasure (in publication).

Table 8. Attacks Against the PHY Layer

Attack	1.0/1.0.1	1.0.2/1.0.3	1.0.4	1.1
beacon drifting	●	●	●	● E: [37]
V: unauthenticated beacon				
beacon spoofing	● [62]	● [98, 99]; S: [92]	●	● [10, 11, 92, 99]; S: [93]
V: unauthenticated beacon				
downlink-delayed wormhole	●	●	●	● ^a E: [37]
V: unrelated downlink, no integrity protection of DL tx config; A: jam-and-replay wormhole				
jam-and-replay wormhole	● [2]; E: [3, 47]	● E: [38]	●	● [37]; E: [38, 47]
V: missing Temporal integrity, missing spatial integrity; A: jamming (reactive)				
jamming (generic)	● S: [57]; PoC: [2, 95]	● [15]; PoC: [63]; E: [40]	●	●
V: coexistence issues				
jamming (periodic)	●	● E: [40]	●	●
V: coexistence issues; A: traffic analysis				
jamming (reactive)	● [2]; E: [3]	● E: [38]	●	● ^b [11]; E: [38]
V: FHDR is not confidential, FOpts are not confidential (1.0–1.0.4, 1.0 on NS, 1.0 on ED), coexistence issues				
jamming (triggered)	● S: [57]	● E: [38]	●	● E: [38, 71]
V: coexistence issues				
known plaintext attack on MAC commands	● ^c	● ^c	● ^c	● [12]
V: MAC commands are predictable				
locate ED through trilateration	●	●	●	● [11, 30]
provoke conflicting downlink	●	● PoC: [7]	●	●
A: traffic analysis				
rx2 wormhole	●	●	●	● E: [37]
V: no integrity protection of DL tx config; A: jam-and-replay wormhole				
traffic analysis	●	● [15, 92]; PoC: [7]; E: [40]	●	● ^b [10, 11]; F: [46]; S: [86]
V: FHDR is not confidential, FOpts are not confidential (1.0–1.0.4, 1.0 on ED, 1.0 on NS), unauthenticated beacon, information leakage				

^aonly unconfirmed messages. ^bonly FOpts are encrypted. ^conly in FRMPayload.

V: enabled by this vulnerability (only for selected versions). A: enabled by this attack (only for selected versions).

lists the corresponding attacks. The taxonomy shows that a combination of jamming attacks with sniffing messages allows manipulating the message exchange on higher layers, e.g., by dropping, reordering, or replaying messages. Protection against attacks on the PHY is limited, as the LoRa modulation is not very complex. Multiple authors have practically demonstrated how COTS devices allow even sophisticated jamming setups, showing that attacks on the LoRa link must be considered relevant. As a result, most vulnerabilities and attacks in the graphs for PHY attacks remain unsolved, independently of the LoRaWAN version. That implies that MAC layer protocols must consider such attacks, treat the channel as lossy and untrusted, and implement proper security measures above the PHY.

Table 9. Vulnerabilities on the MAC Layer Exploitable Over the LoRa Link

Vulnerability	1.0/1.0.1	1.0.2/1.0.3	1.0.4	1.1
counter overflow M: rejoin on FCnt saturation (1.1, 1.0.4)	● [70]	● [24, 66, 98, 99]; PoC: [99]	⊖	⊖ [24, 30]
counter reset M: require NVM on EDs (1.1, 1.0.4), forbid reset of FCnt (1.1, 1.0.4); P: (periodically) refresh rootkeys [25, 33, 36, 56, 61, 66, 89]	● [2]	● [24, 66, 99]; PoC: [82, 98, 99]	⊖ ^a	⊖ [24, 30, 99]
DevNonce saturation M: replace nonces by counters (1.1, 1.0.4); P: increase size of DevNonce [101]	● [101]; F: [88]	●	⊖	⊖
direct usage of root keys in OTAA early context switch on NS M: require uplink before context switch (1.0.2, 1.0.3, 1.1); P: request downlink after joining [24]	● [14]	● [13]	●	● ^b
implicit downlink route selection JS keystore as single point of failure P: JS key management in Blockchain [76]	● E: [47]	● S: [92]	●	● [92]; S: [93]; E: [47]
key derivation on NS M: key derivation on (multiple) JSs (1.1, 1.0.4); P: encrypt payload in application [21]	⊖	⊖	●	● E: [76]
long-term root keys P: (periodically) refresh rootkeys [25, 33, 36, 56, 61, 66, 89]	● [5, 67]	● [21]	⊖	⊖ [21]
MAC commands are predictable MIC at end of frame P: move MIC to frame start [60]	●	●	●	● [25, 33, 56, 61, 89]
no context switch verification on ED M: RekeyInd/RekeyConf MAC command (1.1); P: request downlink after joining [24]	●	●	●	● [12]; E: [37]
no forward secrecy P: derive sessions keys from session keys [23, 90], derive session keys via DH key exchange [41, 80], (periodically) refresh rootkeys [25, 33, 36, 56, 61, 66, 89]	● [80]	● [23, 90, 100]; F: [41]	●	● [23, 25, 36, 61, 90]
no integrity protection of UL tx config M: include UL tx parameters MIC (1.1)	●	●	●	⊖
NS autonomously selects MAC version P: set minimum MAC version on ED [12, 23]	⊖	⊖	⊖	● [12, 23]
NS keystore as single point of failure M: key derivation on (multiple) JSs (1.1, 1.0.4); P: server key storage in TEE [16]	● [16, 62, 70]	●	⊖	⊖
repeating nonces M: replace nonces by counters (1.1, 1.0.4); P: use NwkKey for Join Request MIC [42]	● [67, 101]	● [24, 42, 66, 100]; E: [38]	⊖	⊖ [24]; E: [38]
unprotected PHY header unrelated Join Accept M: include DevNonce in Join Accept MIC (1.1); P: add DevNonce to Join Accept payload [101]	●	●	●	⊖ E: [60]
unspecified ADR algorithm M: recommendations for ADR algorithms (1.0.4)	● [101]	● [24]; F: [96]; S: [92, 93]; E: [38]	●	⊖ [24]; F: [26]; S: [93]; E: [38]
	●	●	⊖ ^d	● E: [37]

^a without NVM. ^b Only NwkKey. ^c except for Class C. ^d specifies recommended behavior.

M: mitigated in specification (with version). P: proposed countermeasure (in publication).

Consequently, we studied publications regarding threats and security measures on the MAC layer. Table 9 shows the identified vulnerabilities, and Table 10 gives an overview of attacks based on these vulnerabilities. The graphs show how some areas, like the join process and frame counter issues, have undergone a thorough security analysis for LoRaWAN 1.1. The introduced countermeasures match the identified vulnerabilities well, leading to nearly complete mitigation of known attacks in these areas. Other topics, like Class B security, have barely been addressed.

In between these extremes, we see how standardized countermeasures introduced in LoRaWAN 1.1 only address a specific attack but still allow variations of it under slightly different conditions. A good example of this is the mitigation of the ACK Spoofing attack as discussed in Section 6.2: The frame counter is only included in the MIC of a downlink reply if the ACK flag is set. That indeed prevents the attack itself. However, it does not fully address wormhole attacks which are a building block for all attacks related to message reordering, as unrelated downlink messages remain possible for unconfirmed traffic. We hope our taxonomy supports identifying central vulnerabilities and helps to find countermeasures that fully prevent attacks and keep the protocol's complexity low.

7 BACKEND INFRASTRUCTURE THREATS AND MITIGATION

The last part of this survey discusses threats to the backend infrastructure. We focus on malicious actors who gained access to servers or networks which, contrary to the exposed edge devices

Table 10. Attacks Against the MAC Layer over the LoRa Link

Attack	1.0/1.0.1	1.0.2/1.0.3	1.0.4	1.1
ACK spoofing V: unrelated downlink; A: downlink-delayed wormhole	●	● [24, 99]; PoC: [98, 99]; E: [38]	●	⊗ [24, 99]; E: [38]
ADR spoofing V: missing spatial integrity, unspecified ADR algorithm, MAC commands are predictable; A: rx2 wormhole, downlink-delayed wormhole, compromised GW	⊗ ^a E: [47]	⊗ ^a	⊗ ^a	⊗ ^a E: [37, 47]
counter desynchronization A: data replay attack (ABP) (1.0–1.0.4, 1.0 on NS), MIC brute-forcing, root key brute-forcing	●	● [13, 66, 99]; PoC: [15, 82, 98, 99]	⊗ ^b	⊗ ^c [15, 99]
data replay attack (ABP) V: counter overflow (1.0–1.0.3), counter reset (1.0–1.0.4); A: MIC brute-forcing	●	● [98]; E: [73]	⊗ ^b	⊗
data replay attack (OTAA) A: Join Accept replay for keystream reuse (1.0–1.0.1, 1.0.3), Join Request replay for keystream reuse	●	● E: [73]	⊗	⊗
ED impersonation V: unprotected UART interfaces, insecure key storage on ED, NS keystore as single point of failure (1.0–1.0.3, 1.0 on NS), EM side-channel information leakage, JS keystore as single point of failure (1.0.4, 1.1); A: root key brute-forcing	⊗ ^a [16]; PoC: [2]	⊗ ^a [13]; PoC: [28, 43]	⊗ ^a	⊗ ^a [11]; PoC: [28]
energy depletion V: unprotected PHY header (1.0–1.0.4, 1.0 on ED, 1.0 on NS), MIC at end of frame; A: jamming (reactive), suppress LinkCheck, suppress ACK	● E: [65]	●	●	● E: [60]
forge multicast messages V: shared keys for multicast, insecure key storage on ED, NS keystore as single point of failure (1.0–1.0.3, 1.0 on NS)	⊗ ^d [62]	⊗ ^d	●	●
frame decryption (key reuse) V: counter overflow (1.0–1.0.3), counter reset (1.0–1.0.3); A: Join Accept replay for keystream reuse (1.0–1.0.1, 1.0.3), Join Request replay for keystream reuse	● [62]	● [24, 99]; F: [96]; PoC: [98, 99]	⊗	⊗ [24, 99]
GW duty-cycle exhaustion V: no integrity protection of UL tx config; A: jam-and-replay wormhole	●	●	●	⊗
Join Accept replay for desynchronization V: unrelated Join Accept, no context switch verification on ED	● [47, 101]; F: [26, 67]	● [24, 66]; S: [92, 93]; E: [38]	●	⊗ [23, 24, 47]; F: [26]; S: [93]; E: [38]
Join Accept replay for keystream reuse V: unrelated Join Accept, repeating nonces	● F: [26]	● [24]	⊗	⊗ [23, 24]; F: [26]
Join Request replay for desynchronization V: early context switch on NS (1.0–1.0.1, 1.0.4), repeating nonces (1.0–1.0.1, 1.0 on ED, 1.0 on ED)	● [101]; F: [26]	⊗ [24]	⊗ ^e	⊗ [10, 23, 24]; F: [26]
Join Request replay for DoS V: DevNonce saturation (1.0–1.0.1), early context switch on NS	● [101]	⊗ [42]	⊗	⊗
Join Request replay for keystream reuse V: repeating nonces	● F: [26]	● [24]; F: [96]	⊗	⊗ [23, 24]; F: [26]
MAC version downgrade V: NS autonomously selects MAC version	⊗	⊗	⊗	● [12, 23]
redirect downlink routing V: missing spatial integrity, implicit downlink route selection; A: compromised GW, jam-and-replay wormhole	● E: [47]	● S: [92]	●	● [11, 92]; S: [93]; E: [47]
root key brute-forcing V: direct usage of root keys in OTAA, long-term root keys	● [14]	● [13]	●	⊗ ^f [36]
suppress ACK A: jamming (reactive)	● E: [65]	●	●	●
suppress LinkCheck V: FOpts are not confidential (1.0–1.0.4, 1.0 on ED, 1.0 on NS); A: jamming (reactive)	●	● PoC: [7]	●	⊗ ^g

^aimplementation-dependent. ^bNVM for FCnt recommended. ^cOnly through GW compromise. ^dfeature not fully specified (<1.0.3). ^ere-introduced in 1.0.4. ^fOnly NwkKey. ^gonly ADRackReq.

V: enabled by this vulnerability (only for selected versions). A: enabled by this attack (only for selected versions).

in Section 5, cannot be physically compromised. As ways of infiltrating networked systems are manifold and depend strongly on operational security, the attack path to access such an entity is out of scope for our research. Instead, we discuss vulnerabilities and attacks inherent to the LoRaWAN protocol or its backend interface specification.

7.1 Integrity Issues

In the backend infrastructure, all stakeholders involved in a LoRaWAN network encounter each other, interoperate, but also protect their assets against others, be it inside or outside the network infrastructure. Therefore, the security goals of integrity and authenticity make up the largest part of the discourse in this area. Figure 26 shows concepts related to these goals.

Missing end-to-end integrity protection between ED and AS is broadly discussed as a fundamental vulnerability. When an ED sends an application payload, a keystream is generated and then xor'ed to the payload to create the confidentiality-protected FRMPayload field in the DataUp message

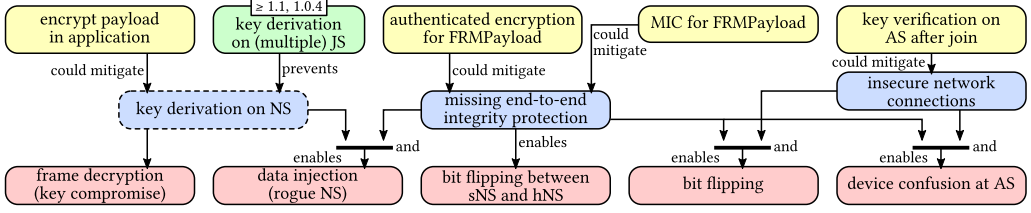


Fig. 26. Vulnerabilities and attacks affecting integrity in the backend infrastructure.

(cf. Figure 6). Combining plaintext and keystream does not provide a means for the receiver to check for the integrity or authenticity of the message. The MIC of LoRaWAN messages provides end-to-end integrity and authenticity protection between ED and NS including the message payload. However, as soon as the payload arrives at the NS, both properties depend on trust in the security of the NS and backend communication channels. Figure 6 shows that the cryptographic method has not changed between different versions.

A direct consequence of this vulnerability is a *bit flipping* attack on the application payload of LoRaWAN messages. If the structure of the transferred data is known, targeted manipulation of individual values or flags might be possible. In one of the earliest works on LoRaWAN security [62], Miller already pointed out that authentication of the payload itself is missing, and the AS would have to be able to check the MIC. Lee et al. [45] discuss payload protection with a focus on the wireless channel but state that if the MIC could be forged, flipping bits within the payload would be possible. Van Es [92] mentions the attack as one of the major issues in LoRaWAN 1.0.2 security. Also, for LoRaWAN 1.0.2, Yang [98] provides the first PoC showing that an adversary in a MITM position between NS and AS can manipulate an arbitrary number transferred as payload. He mentions that missing trust between AS and NS affects other fields, e.g., the device address or frame counter. Changing those values would affect the processing of messages on the AS. In [99], Yang et al. extend this work and show its applicability to LoRaWAN 1.1. Dönmez and Nigussie [24] discuss the vulnerability and bit-flipping in backward-compatibility scenarios, concluding that these are also affected. In [10], Butun et al. mention this attack as a threat to LoRaWAN 1.1 security, and in [11], they assess a minor risk based on a low likelihood for the attack combined with a minor to moderate impact. Missing integrity is also discussed as motivation in publications on cryptographically securing the mutual authentication of ED and AS, e.g., by You et al. [100] and Canard and Ferreira [12]. Pospisil et al. [73] demonstrate experimental validation of bit flipping in a testbed.

The simple bit-flipping attack requires *insecure network connections* between NS and AS, after the MIC protection is removed. If operational measures are taken to protect the data in transit, outsider attacks are prevented, while trust between network operator and application provider is still required. The NS could still apply bit-flipping before the encrypted payload enters the secure channel to the AS. Miller [62] emphasizes the importance of secured channels from GW to NS and NS to AS early on. Wesemeyer et al. [96] point out that the Backend Interface specification does not explicitly require the connection between NS and AS to be secure.

Handover roaming, as introduced in LoRaWAN 1.1, also requires trust relations between Home Network Server (hNS) and sNS. The sNS forwards the FRMPayload content to the hNS, which allows *bit-flipping between sNS and hNS* if no operational measures prevent it. Butun et al. [11] discuss bit-flipping in roaming scenarios and assess a minor risk for all protection goals.

According to Yang [98], the ability to manipulate other frame metadata enables *device confusion at the AS* as an attack. As Wesemeyer et al. present in their formal model [96], an adversary on the channel between AS and NS can switch the DevAddr, leading to decryption with the wrong

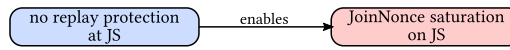


Fig. 27. Resource exhaustion in the backend infrastructure.

keys and processing the data in the context of the wrong device. If the data format is very compact, which is common to reduce time on air, the AS cannot decide whether the presented data is sound or spoofed. In the opposite direction of communication, it might be possible to transfer scrambled data to arbitrary EDs. Therefore, EDs should validate the correct format of the incoming downlink payload before processing it.

A rogue NS is also capable of *data injection* in LoRaWAN versions without a JS. As De Moraes and Da Conceição [21] point out, the NS can keep or derive the AppSKey and use it to forge and forward any application payload to the AS.

Two authentication methods have been proposed to counter the missing end-to-end protection of the application payload. The first proposal, by Naoui et al. [66], is to introduce a *MIC for the FRMPayload* field. AS and ED could mutually authenticate the transmitted payloads by calculating a MIC with the AppSKey and concatenating it to the FRMPayload. De Moraes and Da Conceição [21] propose a similar measure but use a non-cryptographic checksum before encryption. Secondly, Yang et al. [99] propose to use *authenticated encryption for FRMPayload*, which combines the protection of confidentiality and authenticity in a single cryptographic process.

To assure confidentiality of the application payload, De Moraes and Da Conceição [21] suggest also *encrypting the application's payload* with a key not belonging to the LoRaWAN stack. Their focus is on protection against a rogue NS, which could derive the AppSKey before the introduction of the JS.

Addressing the insecure connections between backend components, Wesemeyer et al. [96] propose a *key verification on the AS after joining* the network over OTAA. Their method uses a key confirmation phase to verify the AppSKey after joining. The proposed protocol is formally verified, however, it only solves key establishment. Message-level payload authentication is needed to prevent bit-flipping attacks or device confusion.

7.2 Exhaustion Attacks

Beyond integrity and authenticity, availability is a critical security goal for the backend infrastructure. Figure 27 shows concepts for an exhaustion attack that affects this goal. The exhaustion attacks in the backend differ from those on the edge devices and the LoRa link, as backend servers usually have high-throughput connections to the Internet and a grid-based energy supply.

The only vulnerability in our corpus for this area concerns the *JS, which lacks replay protection* against replayed Join Request messages. If the JS trusts it, a rogue NS can replay such messages and thus consume resources on the JS, making it unavailable for other NSs. Dönmez and Nigussie [23] point out that, according to the specification [49, Section 6.2.2], the NS is in charge of checking the freshness of the DevNonce. If the JS did not check the DevNonce, it would process each Join Request and issue a new Join Accept message. That does not only bind computational resources but also leads to *JoinNonce saturation* for a particular ED, preventing it from joining any network again.

DevNonce validation is discussed not only in the LoRaWAN 1.1 MAC layer specification. The LoRaWAN Backend Interfaces specification [48, Section 4] puts the JS in charge of tracking the DevNonce. In LoRaWAN 1.0.4, the JS is also responsible for checking the DevNonce [52, Section 6.2.5]. The ambiguity about the responsibility for DevNonce tracking in LoRaWAN 1.1 presumably leads to the replay protection vulnerability.

Table 11. Vulnerabilities in the Backend Infrastructure

Vulnerability	1.0/1.0.1	1.0.2/1.0.3	1.0.4	1.1
insecure network connections P: key-verification on AS after join [96]	● ^a [62]	● ^a	● ^a	● ^a [96]
missing end-to-end integrity protection P: authenticated encryption for FRMPayload [99], MIC for FRMPayload [21, 66]	● [62]	● [21, 24, 66, 92, 99, 100]; PoC: [98, 99]	●	● [10–12, 21, 24, 99]
no replay-protection at JS	○	○	○	● ^b [23]

^aimplementation-dependent. ^bambiguous specification.

M: mitigated in specification (with version). P: proposed countermeasure (in publication).

Table 12. Attacks Against the Backend Infrastructure

Attack	1.0/1.0.1	1.0.2/1.0.3	1.0.4	1.1
bit flipping V: missing end-to-end integrity protection, insecure network connections	● ^a	● ^a [24, 92, 99]; PoC: [98, 99]; E: [73]	● ^a	● ^a [10, 24, 99]; E: [73]
bit flipping between sNS and hNS V: missing end-to-end integrity protection	○	○	○	● ^a [11]
data injection (rogue NS) V: key derivation on NS (1.0–1.0.3), missing end-to-end integrity protection	●	● [21]	○	○ [21]
device confusion at AS V: missing end-to-end integrity protection, insecure network connections	● ^a	● ^a F: [96]	● ^a	● ^a F: [96]
Join Nonce saturation on JS V: no replay-protection at JS	○	○	○	● ^b [23]

^aimplementation-dependent. ^bambiguous specification.

V: enabled by this vulnerability (only for selected versions). A: enabled by this attack (only for selected versions).

7.3 Summary

This section discussed vulnerabilities and attacks on the backend infrastructure, mainly affecting authenticity, integrity, and availability. Table 11 summarizes the vulnerabilities, and Table 12 the attacks. From looking at the number of findings in this area, we conclude that the backend infrastructure is less thoroughly researched than the LoRa link. That might be because the specifications tend to use recommendations over explicit requirements in this area, making it harder to study the specification itself instead of a compliant implementation. Furthermore, the assumptions for the attacker model are relatively high by requiring a position within the infrastructure. Roaming, introduced in LoRaWAN 1.1, is barely perceived by the research community, even though it is responsible for most of the cryptographic changes in the protocol and the keying scheme, making this a promising area for further studies.

8 DISCUSSION

The contributions of this article are twofold: First, we present a systematic, comprehensive, and evolvable overview of LoRaWAN security. Second, we propose a research method and taxonomy to maintain up to date knowledge about the protocol and, at the same time, to integrate new findings to expand the body of knowledge. In this section, we summarize and discuss the results for both aspects.

8.1 Security of LoRaWAN

LPWANs, particularly LoRaWAN, strongly gained momentum over the last years. This trend reflects in the number of publications in the area, particularly in security, and the growing size of the LoRaWAN specification. Differences between specification versions can be subtle, as within the 1.0.x branch, or crucial, as in the release of LoRaWAN 1.1. Thus, keeping track of the changes and assessing their implications for the network's security becomes increasingly important. Publications from our corpus often focus on a specific LoRaWAN version. For some articles, even

identifying the analyzed version can be challenging, exacerbating the classification of their results. Combining results from all publications in our corpus makes the bigger picture of LoRaWAN security clearer.

Physical attacks on devices are possible, and their immediate consequences are often hard to prevent. As they mainly affect field entities, their impact is limited: For EDs, usually, only the device itself is compromised. For GWs, most attacks are limited to the respective coverage area. Threats to the backend infrastructure are not thoroughly covered in the literature. Particularly the security implications of new features like roaming are barely discussed. Many issues in the backend infrastructure can only be analyzed in the context of a concrete implementation or software system as the backend specification focuses on interfaces. Issues on the LoRa link outnumber those for backend and physical access and have the best literature coverage. We see that the security in some areas, like the join process and frame counters, improves significantly in LoRaWAN 1.1, and most vulnerabilities are mitigated. Other parts of the specification still contain known vulnerabilities. Many authors propose improvements for key management to reduce trust requirements and introduce properties like forward secrecy. Spoofing and replay attacks are not fully mitigated. For example, the measures against ACK spoofing do not fully prevent message reordering through selective wormholes. Traffic analysis remains partially possible and allows attackers to learn about the network and application metadata.

The primary issue for LoRaWAN security is the slow adaption of new specifications, preventing most of the countermeasures from being implemented yet. Reasons may be the long lifetime of EDs and devices not being certified for the new specification. Currently, network operators, software providers, and device manufacturers must improve the security of their devices and network themselves. The presented results can support this process by facilitating the assessment of the individual risk and suggesting measures to reduce it.

8.2 Evolving the Taxonomy

We used the creation of this publication as an opportunity to develop and optimize software tools supporting the systematic and evolving analysis of network protocols. The ability to add publications and concepts iteratively while getting an immediate and interactive visualization helps handle all aspects of the protocol analysis by providing different views based on the protocol version or applied methodology and supporting the refinement of the initially only roughly defined concepts. LoRaWAN, as a relatively new protocol, allowed to create a comprehensive picture, as the number of publications is still manageable. Again, this emphasizes the requirement for an evolvable process that allows the taxonomy to grow with a maturing protocol.

As a next step, we want to utilize the insights from the process of writing this article to generalize and publish the software tools supporting this methodology, allowing other researchers to apply it as well. Furthermore, we plan to publish the created dataset as a living document, making the process sustainable beyond a single publication. We hope that an evolvable dataset with an accessible way of contributing to it motivates researchers to help keep the data updated and point out further research directions in LoRaWAN security.

9 CONCLUSION

LPWANs are enablers for several novel Internet-of-Things (IoT) applications in urban environments, industrial settings, or critical infrastructures. LoRaWAN is the currently dominating wireless technology tailored to these application domains, and deployments are expected to cover long ranges while operating for multiple years or even decades in partially unattended settings. The security of LoRaWAN is challenged over the entire deployment lifetime by novel attacks and

vulnerabilities. At the same time, the standard evolves, and countermeasures are proposed, standardized, implemented, and rolled out.

This article proposes a novel methodology to create systematic and evolvable surveys on vulnerabilities, attacks, and countermeasures in information and communication systems. While our methodology is general in scope, we instantiated and tailored it for the special case of LoRaWAN security. We performed an extensive and systematic literature search to compile a corpus of representative papers for our survey. Matching the architecture model of LoRaWAN, we study security on three scopes: physical, link-layer, and backend infrastructure.

Based on precise definitions of security concepts and their relationship, we devise a data structure to capture the relationship between vulnerabilities, attacks, and countermeasures for different specification versions. All views on the security of LoRaWAN in our survey are generated based on a linked dataset. Tables summarizing the security of the protocol are generated automatically, and graphs have been created using interactive tool support.

Our methodology supports evolving the survey over time when new security findings are discovered, or the specification progresses and includes novel security mechanisms. We provide manufacturers, developers, and network operators with a powerful methodology to achieve sustainable security for IoT deployments.

In future work, we want to generalize the devised surveying methodology further and apply it to other information and communication systems. We further plan to make our tooling available as open source software to encourage its wider adoption.

REFERENCES

- [1] Ferran Adelantado, Xavier Vilajosana, Pere Tuset-Peiro, Borja Martinez, Joan Melia-Segui, and Thomas Watteyne. 2017. Understanding the limits of LoRaWAN. *IEEE Communications Magazine* 55, 9 (9 2017), 34–40. DOI: <https://doi.org/10.1109/MCOM.2017.1600613>
- [2] Emekcan Aras, Gowri Sankar Ramachandran, Piers Lawrence, and Danny Hughes. 2017. Exploring the security vulnerabilities of LoRa. In *Proceedings of the 2017 3rd IEEE International Conference on Cybernetics (CYBCONF)*. IEEE, 1–6.
- [3] Emekcan Aras, Nicolas Small, Gowri Sankar Ramachandran, Stéphane Delbruel, Wouter Joosen, and Danny Hughes. 2017. Selective jamming of LoRaWAN using commodity hardware. In *Proceedings of the 14th EAI International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services (MobiQuitous'17)*. Association for Computing Machinery, 363–372. DOI: <https://doi.org/10.1145/3144457.3144478>
- [4] Gildas Avoine and Loic Ferreira. 2018. Rescuing LoRaWAN 1.0. In *Proceedings of the 22nd International Conference On Financial Cryptography and Data Security 2018*. Springer, Berlin, 253–271. DOI: https://doi.org/10.1007/978-3-662-58387-6_14
- [5] Suman Bala, Dominique Barthel, and Säid Gharout. 2019. Separate session key generation approach for network and application flows in LoRaWAN. In *Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing (SAC'19)*. Association for Computing Machinery, New York, NY, 870–879. DOI: <https://doi.org/10.1145/3297280.3297366>
- [6] Elaine Barker. 2020. *Recommendation for Key Management, Part 1: General, Special Publication (NIST SP)*. National Institute of Standards and Technology, Gaithersburg, MD. DOI: <https://doi.org/10.6028/NIST.SP.800-57pt1r5>
- [7] Pamela Beltrán-García, Ponciano Jorge Escamilla-Ambrosio, Eleazar Aguirre-Anaya, and Abraham Rodríguez-Mota. 2020. Availability vulnerabilities evaluation to LoRaWAN. In *Proceedings of the Telematics and Computing (Communications in Computer and Information Science)*, Miguel Félix Mata-Rivera, Roberto Zagal-Flores, and Cristian Barria-Huidobro (Eds.). Springer International Publishing, Cham, 333–351. DOI: https://doi.org/10.1007/978-3-030-62554-2_25
- [8] Thomas Bigler, Albert Treytl, and Thilo Sauter. 2020. Side-channel watermarking for LoRaWAN using robust inter-packet timing. In *Proceedings of the 2020 25th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, Vol. 1. 1367–1370. DOI: <https://doi.org/10.1109/ETFA46521.2020.9211875>
- [9] Alex Biryukov, Adi Shamir, and David Wagner. 2001. Real time cryptanalysis of A5/1 on a PC. In *Proceedings of the Fast Software Encryption*, Gerhard Goos, Juris Hartmanis, Jan van Leeuwen, and Bruce Schneier (Eds.). Springer, Berlin, 1–18.
- [10] Ismail Butun, Nuno Pereira, and Mikael Gidlund. 2018. Analysis of LoRaWAN v1.1 security: Research paper. In *Proceedings of the 4th ACM MobiHoc Workshop on Experiences with the Design and Implementation of Smart Objects (SMARTOBJECTS'18)*. ACM, Association for Computing Machinery, 1–6.

- [11] Ismail Butun, Nuno Pereira, and Mikael Gidlund. 2018. Security risk analysis of LoRaWAN and future directions. *Future Internet* 11, 1 (Dec. 2018), 3. DOI : <https://doi.org/10.3390/fi11010003>
- [12] Sébastien Canard and Loïc Ferreira. 2019. Extended 3-party ACCE and application to LoRaWAN 1.1. In *Proceedings of the Progress in Cryptology—AFRICACRYPT 2019*, Johannes Buchmann, Abderrahmane Nitaj, and Tajjeeddine Rachidi (Eds.), Springer International Publishing, Cham, 21–38. DOI : https://doi.org/10.1007/978-3-030-23696-0_2
- [13] Cesar Cerrudo, Esteban Martinez Fayo, and Matias Sequeira. 2020. *LoRaWAN Networks Susceptible to Hacking: Common Cyber Security Problems, How to Detect and Prevent Them*. Technical Report.
- [14] Ning Chen, Aiqun Hu, and Hua Fu. 2021. LoRa radio frequency fingerprint identification based on frequency offset characteristics and optimized LoRaWAN access technology. In *Proceedings of the 2021 IEEE 5th Advanced Information Technology, Electronic and Automation Control Conference (IAEAC)*, Vol. 5. 1801–1807. DOI : <https://doi.org/10.1109/IAEAC50856.2021.9390904>
- [15] Florian Laurentiu Coman, Krzysztof Mateusz Malarski, Martin Nordal Petersen, and Sarah Ruepp. 2019. Security issues in internet of things: Vulnerability analysis of LoRaWAN, sigfox and NB-IoT. In *Proceedings of the 2019 Global IoT Summit (GIOTS)*. 1–6. DOI : <https://doi.org/10.1109/GIOTS.2019.8766430>
- [16] Luigi Coppolino, Salvatore D’Antonio, Giovanni Mazzeo, Luigi Romano, Irene Bonetti, and Elena Spagnuolo. 2021. The protection of LP-WAN Endpoints via TEE: A chemical storage case study. In *Proceedings of the 2021 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. 345–352. DOI : <https://doi.org/10.1109/ISSREW53611.2021.00100>
- [17] Semtech Corporation. 2016. *LoRaWAN—Simple Rate Adaptation Recommended Algorithm*. Technical Report.
- [18] Cas J. F. Cremers. 2008. The scyther tool: Verification, falsification, and analysis of security protocols. In *Proceedings of the Computer Aided Verification*, Aarti Gupta and Sharad Malik (Eds.). Springer, Berlin, 414–418.
- [19] Daniele Croce, Michele Gucciardo, Stefano Mangione, Giuseppe Santaromita, and Ilenia Tinnirello. 2018. Impact of LoRa imperfect orthogonality: Analysis of link-level performance. *IEEE Communications Letters* 22, 4 (2018), 796–799. DOI : <https://doi.org/10.1109/LCOMM.2018.2797057>
- [20] Syed Muhammad Danish, Arfa Nasir, Hassaan Khaliq Qureshi, Ayesha Binte Ashfaq, Shahid Mumtaz, and Jonathan Rodriguez. 2018. Network intrusion detection system for jamming attack in LoRaWAN join procedure. In *Proceedings of the 2018 IEEE International Conference on Communications (ICC)*. 1–6. DOI : <https://doi.org/10.1109/ICC.2018.8422721>
- [21] Poliana De Moraes and Arlindo Flavio Da Conceição. 2021. Protecting LoRaWAN data against untrusted network servers. In *Proceedings of the 2021 IEEE International Conferences on Internet of Things (iThings) and IEEE Green Computing Communications (GreenCom) and IEEE Cyber, Physical Social Computing (CPSCom) and IEEE Smart Data (SmartData) and IEEE Congress on Cybermatics (Cybermatics)*. 99–106. DOI : <https://doi.org/10.1109/iThings-GreenCom-CPSCom-SmartData-Cybermatics53846.2021.00029>
- [22] Joris Delclef. 2018. *Change Request: FOpts Encryption, Usage of FCntDown on the LoRaWAN L2 1.1 Specification*. Technical Report. STMicroelectronics. Retrieved from <https://loro-alliance.org/resource-hub/fopts-encryption-usage-fcntdown-errata-lorawan-l2-11-specification>.
- [23] Tahsin C. M. Dönmez and Ethiopia Nigussie. 2018. Security of join procedure and its delegation in LoRaWAN v1.1. *Procedia Computer Science* 134 (2018), 204–211. DOI : <https://doi.org/10.1016/j.procs.2018.07.202>
- [24] Tahsin C. M. Dönmez and Ethiopia Nigussie. 2018. Security of LoRaWAN v1.1 in backward compatibility scenarios. *Procedia Computer Science* 134 (2018), 51–58. DOI : <https://doi.org/10.1016/j.procs.2018.07.143> The 15th International Conference on Mobile Systems and Pervasive Computing (MobiSPC 2018)/The 13th International Conference on Future Networks and Communications (FNC-2018)/Affiliated Workshops.
- [25] Tahsin C.M. Dönmez and Ethiopia Nigussie. 2019. Key management through delegation for LoRaWAN based health-care monitoring systems. In *Proceedings of the 2019 13th International Symposium on Medical Information and Communication Technology (ISMICT)*. 1–6. DOI : <https://doi.org/10.1109/ISMICT.2019.8743947>
- [26] Mohamed Eldefrawy, Ismail Butun, Nuno Pereira, and Mikael Gidlund. 2019. Formal security analysis of LoRaWAN. *Computer Networks* 148 (2019), 328–339. DOI : <https://doi.org/10.1016/j.comnet.2018.11.017>
- [27] Bernat Carbonés Fargas and Martin Nordal Petersen. 2017. GPS-free geolocation using LoRa in low-power WANs. In *Proceedings of the 2017 Global Internet of Things Summit (GIOTS)*. IEEE, 1–6. DOI : <https://doi.org/10.1109/giots.2017.8016251>
- [28] Kazuhide Fukushima, Damien Marion, Yuto Nakano, Adrien Facon, Shinsaku Kiyomoto, and Sylvain Guilley. 2020. Evaluation of side-channel key-recovery attacks on LoRaWAN end-device. In *Proceedings of the Information Systems Security and Privacy*, Paolo Mori, Steven Furnell, and Olivier Camp (Eds.). Springer International Publishing, Cham, 74–92. DOI : https://doi.org/10.1007/978-3-030-49443-8_4
- [29] FUOTA Working Group of the LoRa Alliance Technical Committee. 2019. *FUOTA Process Summary—Technical Recommendation TR002 v1.0.0*. Technical Report. LoRa Alliance. Retrieved from <https://loro-alliance.org/resource-hub/fuota-process-summary-technical-recommendation-tr002-v100>.

- [30] Alexander Gladisch, Simon Rietschel, Thomas Mundt, Johann Bauer, Johannes Goltz, and Simeon Wiedenmann. 2018. Securely connecting iot devices with LoRaWAN. In *Proceedings of the 2018 Second World Conference on Smart Trends in Systems, Security and Sustainability (WorldS4)*. 220–229. DOI : <https://doi.org/10.1109/WorldS4.2018.8611576>
- [31] Ana Goulart, Anitha Chennamaneni, Damiano Torre, Byul Hur, and Fadhil Y. Al-Aboosi. 2022. On wide-area IoT networks, lightweight security and their applications—a practical review. *Electronics* 11, 11 (Jan. 2022), 1762. DOI : <https://doi.org/10.3390/electronics11111762>
- [32] Claire Goursaud and Jean-Marie Gorce. 2015. Dedicated networks for IoT: PHY/MAC state of the art and challenges. *EAI Endorsed Transactions on Internet of Things* 1, 1 (Oct. 2015), 1–11. DOI : <https://doi.org/10.4108/eai.26-10-2015.150597>
- [33] Jialuo Han and Jidong Wang. 2018. An enhanced key management scheme for LoRaWAN. *Cryptography* 2, 4 (Nov 2018), 34. DOI : <https://doi.org/10.3390/cryptography2040034>
- [34] Jetmir Haxhibeqiri, Eli De Poorter, Ingrid Moerman, and Jeroen Hoebeke. 2018. A survey of LoRaWAN for IoT: From technology to application. *Sensors* 18, 11 (2018), 3995. DOI : <https://doi.org/10.3390/s18113995>
- [35] Jetmir Haxhibeqiri, Floris Van den Abele, Ingrid Moerman, and Jeroen Hoebeke. 2017. LoRa scalability: A simulation model based on interference measurements. *Sensors* 17, 6 (June 2017), 1193. DOI : <https://doi.org/10.3390/s17061193>
- [36] Nur Hayati, Kalamullah Ramli, Susila Windarta, and Muhammad Suryanegara. 2022. A novel secure root key updating scheme for LoRaWANs based on CTR_AES DRBG 128. *IEEE Access* 10 (2022), 18807–18819. DOI : <https://doi.org/10.1109/ACCESS.2022.3150281>
- [37] Frank Hessel, Lars Almon, and Flor Álvarez. 2020. ChirpOTLE: A framework for practical LoRaWAN security evaluation. In *Proceedings of the 13th ACM Conference on Security and Privacy in Wireless and Mobile Networks*. Association for Computing Machinery, New York, NY, 306–316. DOI : <https://doi.org/10.1145/3395351.3399423>
- [38] Frank Philipp Hessel. 2019. *LoRaWAN Security Analysis: An Experimental Evaluation of Attacks*. Master's thesis. Technische Universität Darmstadt.
- [39] Ningning Hou and Yuanqing Zheng. 2020. CloakLoRa: A covert channel over LoRa PHY. In *Proceedings of the 2020 IEEE 28th International Conference on Network Protocols (ICNP)*. 1–11. DOI : <https://doi.org/10.1109/ICNP49622.2020.9259364>
- [40] Max Ingham, Jims Marchang, and Deepayan Bhowmik. 2020. IoT security vulnerabilities and predictive signal jamming attack analysis in LoRaWAN. *IET Information Security* 14, 4 (2020), 368–379. DOI : <https://doi.org/10.1049/iet-ifs.2019.0447>
- [41] Martha N. Kamkuemah. 2021. Epistemic analysis of a key-management vulnerability in LoRaWAN. In *Proceedings of the 2021 18th International Conference on Privacy, Security and Trust (PST)*. 1–7. DOI : <https://doi.org/10.1109/PST52912.2021.9647741>
- [42] Jaehyu Kim and JooSeok Song. 2017. A simple and efficient replay attack prevention scheme for LoRaWAN. In *Proceedings of the 2017 the 7th International Conference on Communication and Network Security*. ACM, 32–36. DOI : <https://doi.org/10.1145/3163058.3163064>
- [43] Reinhard Kloibhofer, Erwin Kristen, and Luca Davoli. 2020. LoRaWAN with HSM as a security improvement for agriculture applications. In *Computer Safety, Reliability, and Security. SAFECOMP 2020 Workshops (Lecture Notes in Computer Science)*, António Casimiro, Frank Ortmeier, Erwin Schoitsch, Friedemann Bitsch, and Pedro Ferreira (Eds.). Springer International Publishing, Cham, 176–188. DOI : https://doi.org/10.1007/978-3-030-55583-2_13
- [44] Franz Kuntke, Vladimir Romanenko, Sebastian Linsner, Enno Steinbrink, and Christian Reuter. 2022. LoRaWAN security issues and mitigation options by the example of agricultural IoT scenarios. *Transactions on Emerging Telecommunications Technologies* 33, 5 (2022). DOI : <https://doi.org/10.1002/ett.4452>
- [45] JungWoon Lee, DongYeop Hwang, JiHong Park, and Ki-Hyung Kim. 2017. Risk analysis and countermeasure for bit-flipping attack in LoRaWAN. In *Proceedings of the 2017 International Conference on Information Networking (ICOIN)*. IEEE, 549–551. DOI : <https://doi.org/10.1109/ICOIN.2017.7899554>
- [46] Patrick Leu, Ivan Puddu, Aanjhan Ranganathan, and Srdjan Čapkun. 2018. I send, therefore i leak: Information leakage in low-power wide area networks. In *Proceedings of the 11th ACM Conference on Security & Privacy in Wireless and Mobile Networks (Stockholm, Sweden) (WiSec'18)*. Association for Computing Machinery, New York, NY, 23–33. DOI : <https://doi.org/10.1145/3212480.3212508>
- [47] Pierluigi Locatelli, Pietro Spadaccino, and Francesca Cuomo. 2021. Hijacking downlink path selection in LoRaWAN. In *Proceedings of the 2021 IEEE Global Communications Conference (GLOBECOM)*. 1–6. DOI : <https://doi.org/10.1109/GLOBECOM46510.2021.9685973>
- [48] LoRa Alliance Technical Committee. 2017. *LoRaWAN™ Backend Interfaces 1.0 Specification*. Technical Report. LoRa Alliance. Retrieved from <https://lora-alliance.org/resource-hub/lorawanr-back-end-interfaces-v10>.
- [49] LoRa Alliance Technical Committee. 2017. *LoRaWAN™ Specification v1.1*. Technical Report. LoRa Alliance. Retrieved from <https://lora-alliance.org/resource-hub/lorawanr-specification-v11>.

- [50] LoRa Alliance Technical Committee. 2018. *LoRaWAN™ Specification V1.0.3*. Technical Report. LoRa Alliance. Retrieved from <https://lora-alliance.org/resource-hub/lorawanr-specification-v103>.
- [51] LoRa Alliance Technical Committee. 2018. *Technical Recommendations for Preventing State Synchronization Issues around LoRaWAN™ 1.0.x Join Procedure*. Technical Report. LoRa Alliance.
- [52] LoRa Alliance Technical Committee. 2020. *LoRaWAN® L2 1.0.4 Specification (TS001-1.0.4)*. Technical Report. LoRa Alliance.
- [53] LoRa Alliance Technical Committee. 2020. *LoRaWAN® Backend Interfaces Technical Specification (TS002-1.1.0)*. Technical Report. Semtech Corporation. 85 pages.
- [54] LoRa Alliance Technical Committee Regional Parameters Workgroup. 2020. *RP002-1.0.1 LoRaWAN® Regional Parameters*. Technical Report. LoRa Alliance. Retrieved from <https://lora-alliance.org/resource-hub/rp2-101-lorawan-regional-parameters>.
- [55] Slim Loukil, Lamia Chaari Fourati, Anand Nayyar, and K.-W.-A. Chee. 2022. Analysis of LoRaWAN 1.0 and 1.1 protocols security mechanisms. *Sensors* 22, 10 (Jan. 2022), 3717. DOI: <https://doi.org/10.3390/s22103717>
- [56] Frederik Mårilind and Ismail Butun. 2020. Activation of LoRaWAN end devices by using public key cryptography. In *Proceedings of the 2020 4th Cyber Security in Networking Conference (CSNet)*. 1–8. DOI: <https://doi.org/10.1109/CSNet50428.2020.9265530>
- [57] Ivan Martinez, Philippe Tanguy, and Fabienne Nouvel. 2019. On the performance evaluation of LoRaWAN under jamming. In *Proceedings of the 2019 12th IFIP Wireless and Mobile Networking Conference (WMNC)*. IEEE. DOI: <https://doi.org/10.23919/WMNC.2019.8881830>
- [58] Ross McPherson and James Irvine. 2020. Secure decentralised deployment of LoRaWAN sensors. *IEEE Sensors Journal* 21, 1 (2020), 1–1. DOI: <https://doi.org/10.1109/JSEN.2020.3013117>
- [59] Kais Mekki, Eddy Bajic, Frederic Chaxel, and Fernand Meyer. 2019. A comparative study of LPWAN technologies for large-scale IoT deployment. *ICT Express* 5, 1 (2019), 1–7.
- [60] Konstantin Mikhaylov, Radek Fujdiak, Ari Pouutu, Voznak Miroslav, Lukas Malina, and Petr Mlynek. 2019. Energy attack in LoRaWAN: Experimental validation. In *Proceedings of the 14th International Conference on Availability, Reliability and Security (ARES'19)*. ACM, New York, NY, Article 74, 6 pages. DOI: <https://doi.org/10.1145/3339252.3340525>
- [61] Stefano Milani and Ioannis Chatzigiannakis. 2021. Design, analysis, and experimental evaluation of a new secure rejoin mechanism for LoRaWAN using elliptic-curve cryptography. *Journal of Sensor and Actuator Networks* 10, 2 (June 2021), 36. DOI: <https://doi.org/10.3390/jsan10020036>
- [62] Robert Miller. 2016. Lora security: Building a secure LoRa solution. *MWR Labs Whitepaper* (2016).
- [63] Mohammad Mezanur Rahman Monjur, Joseph Heacock, Rui Sun, and Qiaoyan Yu. 2021. An attack analysis framework for LoRaWAN applied advanced manufacturing. In *Proceedings of the 2021 IEEE International Symposium on Technologies for Homeland Security (HST)*. 1–7. DOI: <https://doi.org/10.1109/HST53381.2021.9619847>
- [64] SeungJae Na, DongYeop Hwang, WoonSeob Shin, and Ki-Hyung Kim. 2017. Scenario and countermeasure for replay attack using join request messages in LoRaWAN. In *Proceedings of the 2017 International Conference on Information Networking (ICOIN)*. IEEE, 718–720. DOI: <https://doi.org/10.1109/ICOIN.2017.7899580>
- [65] Muhammad Nouman Nafees, Neetesh Saxena, Pete Burnap, and Bong Jun Choi. 2020. Impact of energy consumption attacks on LoRaWAN-enabled devices in industrial context. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS'20)*. Association for Computing Machinery, New York, NY, 2117–2119. DOI: <https://doi.org/10.1145/3372297.3420022>
- [66] Sarra Naoui, Mohamed Elhoucine Elhdhili, and Leila Azouz Saidane. 2020. Novel enhanced LoRaWAN framework for smart home remote control security. *Wireless Personal Communications* 110, 4 (Feb. 2020), 2109–2130. DOI: <https://doi.org/10.1007/s11277-019-06832-x>
- [67] Sarra Naoui, Mohamed Elhoucine Elhdhili, and Leila Azouz Saidane. 2017. Trusted third party based key management for enhancing LoRaWAN security. In *Proceedings of the 2017 IEEE/ACS 14th International Conference on Computer Systems and Applications (AICCSA)*. 1306–1313. DOI: <https://doi.org/10.1109/AICCSA.2017.73>
- [68] Jorge Navarro-Ortiz, Natalia Chinchilla-Romero, Juan J. Ramos-Munoz, and Pablo Munoz-Luengo. 2019. Improving hardware security for LoRaWAN. In *Proceedings of the 2019 IEEE Conference on Standards for Communications and Networking (CSCN)*. 1–6. DOI: <https://doi.org/10.1109/CSCN.2019.8931397>
- [69] Hassan Noura, Tarif Hatoum, Ola Salman, Jean-Paul Yaacoub, and Ali Chehab. 2020. LoRaWAN security survey: Issues, threats and possible mitigation techniques. *Internet of Things* 12 (Dec. 2020), 100303. DOI: <https://doi.org/10.1016/j.iot.2020.100303>
- [70] Bogdan Oniga, Vasile Dadarlat, Eli De Poorter, and Adrian Munteanu. 2017. Analysis, design and implementation of secure LoRaWAN sensor networks. In *Proceedings of the 2017 13th IEEE International Conference on Intelligent Computer Communication and Processing (ICCP)*. IEEE, 421–428. DOI: <https://doi.org/10.1109/ICCP.2017.8117042>
- [71] Toni Perković and Dino Sirišević. 2020. Low-cost LoRaWAN jammer. In *Proceedings of the 2020 5th International Conference on Smart and Sustainable Technologies (SpliTech)*. 1–6. DOI: <https://doi.org/10.23919/SpliTech49282.2020.9243739>

- [72] Sumesh J. Philip, James M. McQuillan, and Oluwatoba Adegbite. 2020. LoRaWAN v1.1 Security: Are we in the clear yet? In *Proceedings of the 2020 IEEE 6th International Conference on Dependability in Sensor, Cloud and Big Data Systems and Application (DependSys)*. 112–118. DOI: <https://doi.org/10.1109/DependSys51298.2020.00025>
- [73] Ondrej Pospisil, Radek Fujdiak, Konstantin Mikhaylov, Henri Ruotsalainen, and Jiri Misurec. 2021. Testbed for LoRaWAN Security: Design and validation through man-in-the-middle attacks study. *Applied Sciences* 11, 16 (Jan. 2021), 7642. DOI: <https://doi.org/10.3390/app11167642>
- [74] Andri Rahmadhani and Fernando Kuipers. 2018. When LoRaWAN frames collide. In *Proceedings of the 12th International Workshop on Wireless Network Testbeds, Experimental Evaluation & Characterization (ACM WiNTECH'18)*. DOI: <https://doi.org/10.1145/3267204.3267212>
- [75] Brecht Reynders, Wannes Meert, and Sofie Pollin. 2016. Range and coexistence analysis of long range unlicensed communication. In *Proceedings of the 2016 23rd International Conference on Telecommunications (ICT)*. IEEE, 1–6. DOI: <https://doi.org/10.1109/ICT.2016.7500415>
- [76] Victor Ribeiro, Raimir Holanda, Alex Ramos, and Joel J. P. C. Rodrigues. 2020. Enhancing key management in LoRaWAN with permissioned blockchain. *Sensors* 20, 11 (May 2020), 3068. DOI: <https://doi.org/10.3390/s20113068>
- [77] Alessandra Rizzardi, Sabrina Sicari, and Alberto Coen-Porisini. 2022. Analysis on functionalities and security features of internet of things related protocols. *Wireless Networks* 28, 7 (2022), 2857–2887. DOI: <https://doi.org/10.1007/s11276-022-02999-7>
- [78] Pieter Robyns, Eduard Marin, Wim Lamotte, Peter Quax, Dave Singelée, and Bart Preneel. 2017. Physical-layer fingerprinting of LoRa devices using supervised and zero-shot learning. In *Proceedings of the 10th ACM Conference on Security and Privacy in Wireless and Mobile Networks (WiSec'17)*. Association for Computing Machinery, New York, NY, 58–63. DOI: <https://doi.org/10.1145/3098243.3098267>
- [79] Jan Ruge, Jiska Classen, Francesco Gringoli, and Matthias Hollick. 2020. Frankenstein: Advanced wireless fuzzing to exploit new bluetooth escalation targets. In *Proceedings of the 29th USENIX Security Symposium (USENIX Security 20)*. 19–36.
- [80] Ramon Sanchez-Iborra, Jesús Sánchez-Gómez, Salvador Pérez, Pedro Fernández, José Santa, José Hernández-Ramos, and Antonio Skarmeta. 2018. Enhancing LoRaWAN security through a lightweight and authenticated key management approach. *Sensors* 18, 6 (2018), 1833. DOI: <https://doi.org/10.3390/s18061833>
- [81] Altaf Shaik, Ravishankar Borgaonkar, N. Asokan, Valtteri Niemi, and Jean-Pierre Seifert. 2016. Practical attacks against privacy and availability in 4G/LTE mobile communication systems. In *Proceedings of the 2016 Network and Distributed System Security Symposium*. Internet Society. DOI: <https://doi.org/10.14722/ndss.2016.23236>
- [82] Vladislav Skorpil, Vaclav Oujezsky, and Ludek Palenik. 2018. Internet of things security overview and practical demonstration. In *Proceedings of the 2018 10th International Congress on Ultra Modern Telecommunications and Control Systems and Workshops (ICUMT)*. 1–7. DOI: <https://doi.org/10.1109/ICUMT.2018.8631198>
- [83] Nicolas Sornin, Miguel Luis, Thomas Eirich, Thorsten Kramp, and Olivier Hersent. 2015. *LoRaWAN™ Specification v1.0*. Technical Report. LoRa Alliance. Retrieved from <https://lora-alliance.org/resource-hub/lorawanr-specification-v10>.
- [84] Nicolas Sornin, Miguel Luis, Thomas Eirich, Thorsten Kramp, and Olivier Hersent. 2016. *LoRaWAN™ Specification v1.0.1*. Technical Report. LoRa Alliance. Retrieved from <https://lora-alliance.org/resource-hub/lorawanr-specification-v101>.
- [85] Nicolas Sornin, Miguel Luis, Thomas Eirich, Thorsten Kramp, and Olivier Hersent. 2016. *LoRaWAN™ Specification v1.0.2*. Technical Report. LoRa Alliance. Retrieved from <https://lora-alliance.org/resource-hub/lorawanr-specification-v102>.
- [86] Pietro Spadaccino, Domenico Garlisi, Francesca Cuomo, Giorgio Pillon, and Patrizio Pisani. 2022. Discovery privacy threats via device de-anonymization in LoRaWAN. *Computer Communications* 189 (May 2022), 1–10. DOI: <https://doi.org/10.1016/j.comcom.2022.02.017>
- [87] Woo-Jin Sung, Hyeong-Geun Ahn, Jong-Beom Kim, and Seong-Gon Choi. 2018. Protecting end-device from replay attack on LoRaWAN. In *Proceedings of the 2018 20th International Conference on Advanced Communication Technology (ICACT)*. IEEE, 167–171. DOI: <https://doi.org/10.1109/ICACT.2018.8323683>
- [88] Stefano Tomasin, Simone Zulian, and Lorenzo Vangelista. 2017. Security analysis of LoRaWAN join procedure for internet of things networks. In *Proceedings of the Wireless Communications and Networking Conference Workshops (WCNCW)*. IEEE, 1–6. DOI: <https://doi.org/10.1109/WCNCW.2017.7919091>
- [89] Kun-Lin Tsai, Li-Woei Chen, Fang-Yie Leu, Hsiung-Chieh Hsu, and Chuan-Tian Wu. 2022. Secure LoRaWAN root key update scheme for IoT environment. In *Proceedings of the Mobile Internet Security (Communications in Computer and Information Science)*. Ilseun You, Hwankuk Kim, Taek-Young Youn, Francesco Palmieri, and Igor Kottenko (Eds.). Springer Nature, 3–15. DOI: https://doi.org/10.1007/978-981-16-9576-6_1
- [90] Kun-Lin Tsai, Fang-Yie Leu, Shuo-Wen Chang, Jiun-Yi Lin, and Huei-Tang Luo. 2020. A LoRaWAN based energy efficient data encryption method. In *Proceedings of the Innovative Mobile and Internet Services in Ubiquitous Computing*,

- Leonard Barolli, Fatos Xhafa, and Omar K. Hussain (Eds.). Springer International Publishing, Cham, 493–502. DOI : https://doi.org/10.1007/978-3-030-22263-5_47
- [91] Floris Van den Abeele, Jetmir Haxhibeqiri, Ingrid Moerman, and Jeroen Hoebeke. 2017. Scalability analysis of large-scale lorawan networks in NS-3. *IEEE Internet of Things Journal* 4, 6 (Dec. 2017), 2186–2198. DOI : <https://doi.org/10.1109/JIOT.2017.2768498>
 - [92] Eef van Es. 2018. *LoRaWAN Vulnerability Analysis – (In)validation of Possible Vulnerabilities in the LoRaWAN Protocol Specification*. Master's thesis. Open University of the Netherlands.
 - [93] Eef van Es, Harald Vranken, and Arjen Hommersom. 2018. Denial-of-service attacks on LoRaWAN. In *Proceedings of the 13th International Conference on Availability, Reliability and Security*. ACM, 17. DOI : <https://doi.org/10.1145/3230833.3232804>
 - [94] Mathy Vanhoef and Frank Piessens. 2017. Key reinstallation attacks: Forcing nonce reuse in WPA2. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, 1313–1328. DOI : <https://doi.org/10.1145/3133956.3134027>
 - [95] Prachi V. Wadatar, Bharat S. Chaudhari, and Marco Zennaro. 2019. Impact of interference on LoRaWAN link performance. In *Proceedings of the 2019 5th International Conference On Computing, Communication, Control And Automation (ICCUBEA)*. 1–5. DOI : <https://doi.org/10.1109/ICCUBEA47591.2019.9128417>
 - [96] Stephan Wesemeyer, Ioana Boureanu, Zach Smith, and Helen Treharne. 2020. Extensive security verification of the LoRaWAN key-establishment: Insecurities patches. In *Proceedings of the 2020 IEEE European Symposium on Security and Privacy (EuroSP)*. 425–444. DOI : <https://doi.org/10.1109/EuroSP48549.2020.00034>
 - [97] Jiming Xu, You Tang, Yujian Wang, and Xinán Wang. 2019. A practical side-channel attack of a LoRaWAN module using deep learning. In *Proceedings of the 2019 IEEE 13th International Conference on Anti-Counterfeiting, Security, and Identification (ASID)*. 17–21. DOI : <https://doi.org/10.1109/ICASID.2019.8925203>
 - [98] Xueying Yang. 2017. *LoRaWAN: Vulnerability Analysis and Practical Exploitation*. Master's thesis. Delft University of Technology.
 - [99] Xueying Yang, Evgenios Karampatzakis, Christian Doerr, and Fernando Kuipers. 2018. Security vulnerabilities in LoRaWAN. In *Proceedings of the 2018 IEEE/ACM Third International Conference on Internet-of-Things Design and Implementation (IoTDI)*. IEEE, 129–140. DOI : <https://doi.org/10.1109/IoTDI.2018.00022>
 - [100] Ilsun You, Soonhyun Kwon, Gaurav Choudhary, Vishal Sharma, and Jung Seo. 2018. An enhanced LoRaWAN security protocol for privacy preservation in IoT with a case study on a smart factory-enabled parking system. *Sensors* 18, 6 (Jun 2018), 1888. DOI : <https://doi.org/10.3390/s18061888>
 - [101] Simone Zulian. 2016. *Security Threat Analysis and Countermeasures for LoRaWAN Join Procedure*. Master's thesis. Università degli Studi di Padova.

Received 18 March 2022; revised 19 July 2022; accepted 15 August 2022